UNIVERSITÄT BONN

Masterarbeit
zur Erlangung des Grades
Master of Science (M. Sc.)
der Landwirtschaftlichen Fakultät
der Rheinischen Friedrich-Wilhelms-Universität Bonn
Institut für Geodäsie und Geoinformation

Leveraging Vision Foundation Models for Semantic Implicit Multi-Sensor SLAM

von

Shuyi Wu

aus
Ganzhou, China



Bonn 2026

Supervisor:
Prof. Dr. Cyrill Stachniss, University of Bonn, Germany

Second Supervisor:
Dr. Louis Wiesmann

Statement of Authorship

I hereby certify that this master thesis has been composed by myself. I have not made use of the work of others or presented it here unless it is otherwise acknowledged in the text. All references and verbatim extracts have been quoted, and all sources of information have been specifically acknowledged.

Bonn, 04/11/2026 Shaoji Wu

Place, Date (Signature)

Acknowledgments

To start with, I would like to give my heartfelt thanks to all the people who have supported me throughout my master's studies and during the completion of this thesis. Working on this thesis has been a valuable learning experience, allowing me to deepen my knowledge, develop my research skills, and approach problems with greater independence. Throughout this process, I have greatly appreciated the guidance, encouragement, and understanding of those around me.

I would like to thank Prof.Dr Cyrill Stachniss and Dr. Louis Wiesmann for giving me the opportunity to work on this topic and to conduct my thesis within StachnissLab. I am deeply grateful for the inspiring research environment, the valuable discussions, and the opportunity to learn from the work carried out in the group.

I would especially like to thank my supervisor, Dr.Louis Wiesmann, for his kindness, illuminating guidance and profound knowledge. His feedback helped me not only to improve the technical quality of this work, but also to develop a more critical and structured way of approaching research problems.

I would like to express my appreciation to all teachers who have lectured me for their insightful lectures which are of great benefit to me in my research.

My acknowledgements also go to my friends, who have supported me through some of the more difficult moments of this journey. I am deeply grateful to Sonali for her love and support, for always cooking delicious Indian food, and for helping me out of woods. My sincere thanks also go to Shiqi Qiu, who has always supported me unconditionally and has a special talent for bringing everyone together around a table for a good meal. I would also like to thank Junjie Zhao and Kangze Yuan for the incredibly delicious Chinese food and beautifully crafted desserts they have shared with us.

A special thanks goes to Linglong Li and Heidi, who have been both my friends and classmates throughout this journey. We have supported and encouraged one another through our studies, shared many challenges and memorable moments, and helped each other keep going along the way. I am also grateful to Xuefei Zhong, Jennie, Haohan He, Yuting Yang, Mia, Radu, and all the others with whom I shared countless enjoyable evenings and some of the best Mahjong

moments of my time in Bonn. These seemingly ordinary moments have become some of my most cherished memories.

I would also like to thank the friends who were once an important part of my life, even though our paths have since diverged and we may no longer be in contact. The experiences we shared, both good and difficult, have shaped me in ways I did not always realize at the time. Thank you for being part of my journey and for helping me grow into the person I am today.

Finally, I would like to express my deepest gratitude to my family for their unconditional and selfless support. I am especially grateful to my mother for raising me with love and strength, to my stepfather for caring for and supporting me as if I were his own daughter, and to my younger sister for always being one of my strongest sources of emotional support.

I would also like to thank myself. There were countless moments when I wanted to give up, but I kept moving forward. Reaching this point has not always been easy, and I am proud that I chose to persevere through all the uncertainty, setbacks, and difficult moments along the way.

Abstract

In simultaneous localization and mapping problem beyond pose estimation and geometric reconstruction, a good semantic understanding of the scene is also crucial part to support navigation, interaction, and decision-making. SLAM provides a key foundation for estimating of the system's motion, position, and together with a representation of vehicle's surroundings. In particular, self-driving cars or robots need a fine-grained understanding of the surfaces and objects in their vicinity. Geometric maps describe the spatial structure of the environment, while semantic information identifies the categories of its objects and regions. This additional context can support navigation, interaction, and decision-making by helping autonomous systems interpret their surroundings.

This thesis addresses the integration of semantic scene understanding into an efficient implicit SLAM pipeline. The objective is to exploit vision foundation model features during training while enabling semantic segmentation from LiDAR scans alone at inference time, and to incorporate the resulting predictions into an incrementally updated map. The proposed approach takes advantage of DINOv3 feature during training of a semantic segmentation model. Once trained, the model predicts semantic labels directly from LiDAR input, without requiring images for segmentation. The segmentation pipeline is integrated into PIN-SLAM, where successive scans update the same map, allowing scene geometry and semantic categories to be queried and visualized together.

Our method on SemanticKITTI achieves a mean intersection over union (mIoU) of 64.13% on the validation set and 63.5% on the test set, with semantic segmentation inference running at 16.5 frames per second. On the IPB dataset validation split, the model trained on manually annotated local data achieves 56.87% mIoU. Compared with geometry-only mapping, the semantic configuration achieves lower aligned absolute pose error. The integrated system demonstrates live semantic map updates in PIN-SLAM using LiDAR input alone.

Zusammenfassung

In beim Problem der simultanen Lokalisierung und Kartierung (SLAM) ist neben der Posenschätzung und der geometrischen Rekonstruktion auch ein gutes semantisches Verständnis der Szene von entscheidender Bedeutung, um Navigation, Interaktion und Entscheidungsfindung zu unterstützen. SLAM bildet eine wichtige Grundlage für die Schätzung der Bewegung und Position des Systems sowie für die Darstellung der Fahrzeugumgebung. Insbesondere benötigen selbstfahrende Autos oder Roboter ein detailliertes Verständnis der Oberflächen und Objekte in ihrer Umgebung. Geometrische Karten beschreiben die räumliche Struktur der Umgebung, während semantische Informationen die Kategorien ihrer Objekte und Bereiche identifizieren. Dieser zusätzliche Kontext kann Navigation, Interaktion und Entscheidungsfindung unterstützen, indem er autonomen Systemen hilft, ihre Umgebung zu interpretieren.

Diese Arbeit befasst sich mit der Integration des semantischen Szenenverständnisses in eine effiziente implizite SLAM-Pipeline. Das Ziel besteht darin, während des Trainings die Merkmale des Vision-Foundation-Modells zu nutzen und gleichzeitig zur Inferenzzeit eine semantische Segmentierung allein anhand von LiDAR-Scans zu ermöglichen sowie die daraus resultierenden Vorhersagen in eine inkrementell aktualisierte Karte einzubinden. Der vorgeschlagene Ansatz nutzt DINOv3-Merkmale während des Trainings eines semantischen Segmentierungsmodells. Nach dem Training sagt das Modell semantische Labels direkt aus den LiDAR-Eingaben voraus, ohne dass Bilder für die Segmentierung benötigt werden. Die Segmentierungspipeline ist in PIN-SLAM integriert, wo aufeinanderfolgende Scans dieselbe Karte aktualisieren, wodurch Szenengeometrie und semantische

Unsere Methode auf SemanticKITTI erreicht einen mittleren „Intersection over Union“-Wert (mIoU) von 64,13 % auf dem Validierungsdatensatz und 63,5 % auf dem Testdatensatz, wobei die semantische Segmentierung mit 16,5 Bildern pro Sekunde ausgeführt wird. Auf dem Validierungssplit des IPB-Datensatzes erreicht das auf manuell annotierten lokalen Daten trainierte Modell einen mIoU von 56,87 %. Im Vergleich zur rein geometrischen Kartierung weist die semantische Konfiguration einen geringeren ausgerichteten absoluten Positionsfehler auf. Das integrierte System demonstriert Live-Aktualisierungen der semantischen Karte in

PIN-SLAM unter ausschließlicher Verwendung von LiDAR-Eingaben.

Task Descriptiion

Before you start working on your thesis you will receive a task description. Here is the place to put it into your thesis document. Uncomment the command below for that.

Uni Bonn · Stachniss · Nussallee 15 · 53115 Bonn

Rheinische
Friedrich-Wilhelms-
Universität Bonn

Prof. Dr. Cyrill Stachniss
Professor

Institute of Geodesy
and Geoinformation

Photogrammetry
& Robotics Lab

Nussallee 15
53115 Bonn

Tel.: +49-228-73-2714
Fax: +49-228-73-2712
cyrill.stachniss@igg.uni-bonn.de

Secretary:
Marie Köller
Tel.: +49-228-73-2713
Fax: +49-228-73-2712

Bonn, April 29, 2026

To whom it may concern

Task Description for M.Sc. thesis for Shuyi Wu:
Leveraging Vision Foundation Models
for Semantic Implicit Multi-Sensor SLAM

Task Description

Simultaneous localization and mapping is a central problem in robotics and autonomous driving. Beyond pose estimation and geometric reconstruction, most downstream tasks require a semantic understanding of the scene to support navigation, interaction, and decision-making. Combining geometric reconstruction and semantic interpretation into a single, globally consistent map representation has therefore become an active research direction.

Recent works on implicit neural representations have shown that point-based neural maps can provide accurate and globally consistent reconstructions in LiDAR SLAM. PIN-SLAM [4] is a state-of-the-art point-based neural SLAM system that allows accurately estimating the motion of the LiDAR while creating a neural signed distance field to represent the environment. The follow-up work PINGS [3] extends the representation by incorporating camera data to enable photorealistic mapping. Having a photorealistic representation allows humans to interpret the environment. Instead of adding the raw photometric camera information, in this work we are interested in fusing semantic interpretations from the camera data to obtain a geometric and semantic map.

Image foundation models like DINOv3 [5] have been shown to provide dense semantic features that generalize well over unseen data. First works [1] investigated already how to transfer those features from 2D images to a 3D model for segmentation. In this work we are going to explore how the 2D to 3D semantic fusion [2] can be made robust against per-frame segmentation flicker, occlusion, and calibration inaccuracies. In the end, the goal is to extend PIN-SLAM [4] by semantic features from a vision foundation model to enable geometric and semantic mapping.

Point Of Departure:

In this thesis, we aim to extend PIN-SLAM towards a semantic map representation by integrating image-based semantic information into its point-based implicit neural map while leaving its geometric core unchanged. Calibrated [6] LiDAR and camera data from an urban environment is provided. The work will consist of three main parts. First, extracting dense features from onboard cameras using DINOv3 [5]. Second, the development of a 3D label fusion module that integrates the projected features into the

UNIVERSITÄT BONN

Rheinische
Friedrich-Wilhelms-
Universität Bonn

neural point map and fuses subsequent information over time, drawing inspiration from existing works [1, 2] and by this achieving a consistent semantic map. Third, the goal would be to extend the mesh extraction so that the semantic field is queried jointly with the SDF, producing a semantically colored mesh. Fourth, since no ground-truth semantic labels exist for the data, one further task is to obtain the necessary data and perform a quantitative evaluation highlighting the targeted properties of the method.

Kind regards,



Cyrill Stachniss

[1] K. Knaebel, K. Yilmaz, D. de Geus, A. Hermans, D. Adrian, T. Linder, and B. Leibe. Dino in the room: Leveraging 2d foundation models for 3d segmentation. *arXiv preprint*, arXiv:2503.18944, 2025.

[2] J. McCormac, A. Handa, A. Davison, and S. Leutenegger. SemanticFusion: Dense 3D Semantic Mapping with Convolutional Neural Networks. In *Proc. of the IEEE Intl. Conf. on Robotics & Automation (ICRA)*, 2017.

[3] Y. Pan, X. Zhong, L. Jin, L. Wiesmann, M. Popović, J. Behley, and C. Stachniss. PINGS: Gaussian Splatting Meets Distance Fields within a Point-Based Implicit Neural Map. In *Proc. of Robotics: Science and Systems (RSS)*, 2025.

[4] Y. Pan, X. Zhong, L. Wiesmann, T. Posewsky, J. Behley, and C. Stachniss. PIN-SLAM: LiDAR SLAM Using a Point-Based Implicit Neural Representation for Achieving Global Map Consistency. *IEEE Trans. on Robotics (TRO)*, 40:4045–4064, 2024.

[5] O. Siméoni, H.V. Vo, M. Seitzer, F. Baldassarre, M. Oquab, C. Jose, V. Khalidov, M. Szafraniec, S. Yi, M. Ramamonjisoa, et al. Dinov3. *arXiv preprint*, arXiv:2508.10104, 2025.

[6] L. Wiesmann, T. Läbe, L. Nunes, J. Behley, and C. Stachniss. Joint Intrinsic and Extrinsic Calibration of Perception Systems Utilizing a Calibration Environment. *IEEE Robotics and Automation Letters (RA-L)*, 9(10):9103–9110, 2024.

UNIVERSITÄT BONN

Master's Program Geodetic Engineering

ANNOUNCEMENT

of a Master's Thesis according to exam regulations for the
MASTER'S PROGRAM GEODETIC ENGINEERING
of the Rheinischen Friedrich-Wilhelms-Universität Bonn
(in the currently valid version)

To

Surname, First name: Wu , Shuyi

the Master's Thesis with the title

Leveraging Vision Foundation Models for Semantic Implicit Multi-Sensor SLAM

should be issued at the date 30.04.2026.

The Master's Thesis will be supervised by

Mr. Prof. Dr. Cyrill Stachniss

and

Mr. Dr.-ing Louis Wiesmann

Bonn, dated 28.04.2026

Signature and stamp of the scientific representative

Institut für Geodäsie und Geoinformation
Photogrammetrie
Universität Bonn
Nussallee 15, 53115 Bonn

Attachment

- One or two-sided statement of task of the Master's Thesis

Contents

1 Introduction **1**
- 1.1 Motivation . . . 1
- 1.2 Goal and Main Contributions . . . 2
- 1.3 Overview of the Thesis . . . 3

2 Related Work **5**
- 2.1 3D Semantic segmentation . . . 5
- 2.2 Semantic SLAM . . . 9

3 Basic Techniques **13**
- 3.1 Vision Foundation Model and DINOv3 . . . 13
- 3.2 LiDAR-Camera calibration . . . 18
- 3.3 3D semantic segmentation evaluation . . . 21
- 3.4 Odometry Evaluation with evo . . . 23

4 Approach **25**
- 4.1 Problem formulation and notation . . . 26
- 4.2 2D-3D correspondence . . . 27
- 4.3 DINO-baesd LiDAR semantic segmentation . . . 29
- 4.4 Semantic Neural point map . . . 34

5 Experiments **37**
- 5.1 Evaluation Criteria . . . 37
- 5.2 Experiments Setup . . . 38
- 5.3 Semantic Segmentation Results . . . 40
- 5.4 Semantic Mapping and Trajectory Estimation . . . 44

6 Conclusion **49**
- 6.1 Summary of Key Contributions . . . 49

Appendices **63**

A Paper **63**

B Poster **73**

Chapter 1

Introduction

Robots can assist people by performing repetitive and physically demanding tasks and operating in environments that are difficult for humans. For example, autonomous vehicles can provide strong safeguards for people's daily commutes and reduce the occurrence of traffic accidents. Realizing these benefits requires vehicles to estimate their own position and motion while maintaining an informative representation of the surrounding road environment. Simultaneous localization and mapping (SLAM) provides a spatial foundation for autonomous robots. From the early formulation of spatial uncertainty, "On the Representation and Estimation of Spatial Uncertainty" [37] to particle-filter-based approaches such as FastSLAM [27] and GMapping [15], and on to underpin visual system such as MonoSLAM [9], PTAM [18], ORB-SLAM [28], as well as LiDAR and multi-sensor approaches such as LOAM [51] and FAST-LIVO2 [55], the main thread of SLAM has actually remained quite consistent: state estimation, data association, local odometry, loop detection, and global consistency. For intelligent agents such as domestic robots and autonomous vehicles, interaction with the environment also requires an understanding of what the mapped structures represent. Incorporating semantics provides a richer scene representation that not only enhances localization accuracy in SLAM but also enables advanced cognitive functionalities for downstream navigation and planning tasks.

1.1 Motivation

Traditional SLAM system provides mapping environment to intelligent agent but mapping doesn't equal to understanding the latent meaning of the environment. A sparse point cloud mapping can help a robot localize itself but it is difficult for it to answer the question, " Is there a biker driving in front of the car?" An occupancy map can tell a robot where space is occupied but it doesn't necessarily

know whether that's a road or a sidewalk. To help intelligent agents to progress towards human level intelligence, maps are no longer just for localization, it must also support semantic information connects these spatial representations to meaningful scene categories, allowing a robot to distinguish a road from a sidewalk, a vehicle from vegetation, or a door from a wall. (Figure 1.1)

In this thesis, we develop a semantic mapping framework that brings geometric reconstruction and semantic interpretation into a common 3D representation. Building on PIN-SLAM [30], we enrich its point-based implicit neural map with semantic information while retaining its geometric reconstruction and global map optimization pipeline. To learn scene semantics on 3D point clouds, we draw on DINOv3 [36], a vision foundation model that provides high-quality dense features that achieve superior performance on various vision downstream tasks, including semantic segmentation. Our approach transfers DINO-derived semantic knowledge to a LiDAR semantic predictor during training and integrates its predictions into the neural point map during mapping. This design combines visual knowledge acquired during training with LiDAR-only semantic inference, associating reconstructed geometry with semantic categories in a persistent map.

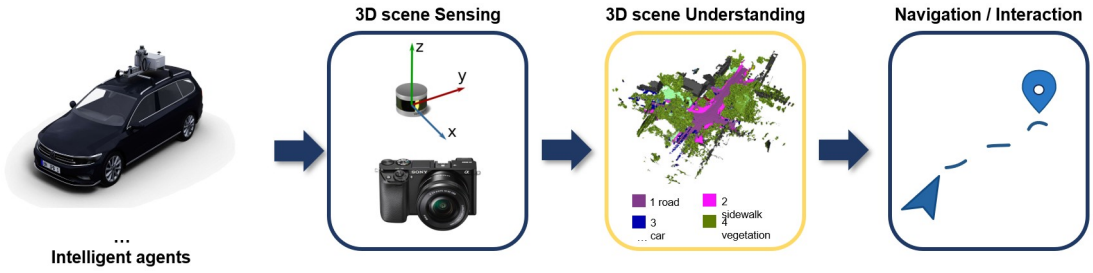


Figure 1.1: The aim of this thesis lies in pushing intelligent agents towards human level intelligence on interaction and navigation

1.2 Goal and Main Contributions

In this thesis, we aim to extend PIN-SLAM towards a semantic map representation by integrating DINO-based semantic information into its point-based implicit neural map while leaving its geometric core unchanged. Given a sequence of scans, it combines PIN-SLAM pose estimation with semantic predictions from an LiDAR-only semantic segmentation module. Paired images, calibration, and point labels support offline pretraining part; deployment uses LiDAR measurements alone. The main contributions are as follows:

- **DINOv3-assisted LiDAR semantic segmentation.** We develop a training pipeline that associates frozen DINOv3 features with LiDAR points

and combines them with geometric features in a multi-scale fusion teacher model. The resulting student predicts 19 semantic classes only using LiDAR.

- **Integration with PIN-SLAM's neural point map.** We connect the pretrained semantic segmentation model to the online mapping pipeline and preserve the correspondence between semantic predictions and points. The predictions supervise the semantic field through map features shared with the SDF decoder, while pose estimation follows PIN-SLAM's geometric registration.
- **Semantic map queries and visualization.** We use the integrated representation to query and display semantic categories in the reconstructed scene. The system provides live semantic map visualization in PIN-SLAM and uses its existing surface reconstruction tools to assign semantic colors to mesh vertices.
- **Data preparation and experimental evaluation.** We prepare calibrated image–LiDAR correspondences and consistent labels for IPB and SemanticKITTI dataset. We manually annotated the semantic labels for IPB dataset for evaluation. We evaluate segmentation accuracy, semantic segmentation model inference speed, and trajectory errors, and study the effects of visual inputs, class weighting, and targeted augmentation. The IPB experiments use manually annotated labels with separate training and validation frames.

Our approach achieves a mean intersection over union (mIoU) of 64.13% on the SemanticKITTI validation set and 63.5% on the test set, with semantic segmentation model inference at 16.5 frames per second. On IPB dataset, our approach achieves 56.87% mIoU. The mapping experiments demonstrate semantic map updates within PIN-SLAM. Across KITTI sequences 08 and 09 and the IPB sequence, the semantic configuration achieves lower aligned absolute pose error.

1.3 Overview of the Thesis

Chapter 2 reviews related work on 3D semantic segmentation and semantic SLAM. Chapter 3 introduces vision foundation models, camera–LiDAR calibration, semantic map representations, and segmentation metrics. Chapter 4 presents the proposed training pipeline, the LiDAR student and fusion teacher, and their integration with the neural point map. Chapter 5 describes the datasets and experimental settings, reports segmentation and ablation results, and evaluates semantic mapping and trajectory estimation. Chapter 6 summarizes the work and its main contributions.

Chapter 2

Related Work

In this chapter, we first review different 3D semantic segmentation approaches and related work on 3D semantic segmentation. Then the second concerns how individual semantic predictions are incorporated into a persistent 3D map. These approaches motivate a semantic extension of PIN-SLAM [30] that leveraging vision foundation model for implicit multi-sensor SLAM.

2.1 3D Semantic segmentation

The goal of 3D semantic segmentation is to assigns each point in a point cloud with its corresponding semantically meaningful category. The outcome of semantic segmentation supports several forms of interaction with a 3D environment. In autonomous driving, label such as road, sidewalk, car, and vegetation provide information for interpreting the surrounding and constructing semantic maps. In indoor robotics, categories such as floor, wall, table, and chair support navigation and object-directed tasks. In augmented and virtual reality (AR and VR), labeled surfaces help associate digital content with meaningful parts of a environment. For this thesis, the application is semantic LiDAR mapping.

Early methods combined explicitly designed geometric descriptors with statistical classifiers. Weinmann *et al.*[43] organized point cloud interpretation around neighbourhood selection, feature extraction, feature selection, and classification. The development of deep learning prompted researchers to enter a new plateau, where specific features can be learned from data. To organize this development, Zhao [54] distinguishes methods that can be divided into two categories: indirect methods and direct methods. The indirect methods attempt to alleviate the irregular and unordered problems of point clouds via conversion of the point clouds into alternative forms of representation. And also further partition this class of works into two tpyes: 2D images projection based and 3D voxel grids

transformation based methods. Recent methods combine these representations with attention and pretraining.

Projection-based methods transforms the point cloud into one or more 2D representations, predicts labels on those representations, and transfers the predictions back to 3D. SnapNet [5] constructs multiple RGB and geometric snapshots, applies image segmentation networks, and back-projects their predictions onto the point cloud. RangeNet++ [26] adapts this principle to rotating LiDAR, it uses a spherical range image and a convolutional network, followed by nearest neighbour post-processing to recover labels for the original points. More recently, RangeFormer [21] combines a transformer architecture with range-view data augmentation and post-processing, and introduces a training strategy for lower resolution range images. The route has thus developed from reusing image convolutional networks to designing the learning pipeline around the sampling and projection characteristics of LiDAR. A range image is derived from laser measurements, so this representation can support LiDAR only inference. The illustration of range image shows in Figure 2.1

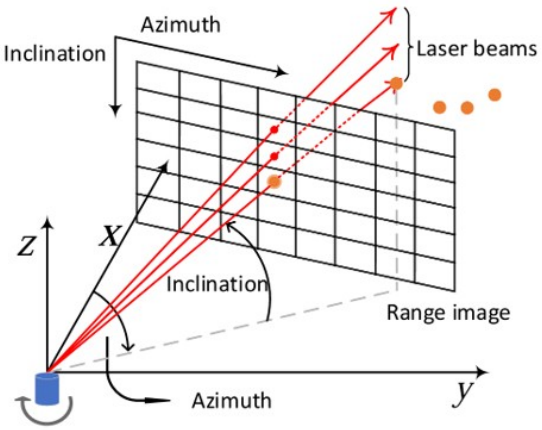


Figure 2.1: The illustration of the native range image.

In addition to projecting 3D point clouds into images, a number of approaches [40, 14, 59, 39, 12] transform point into 3D voxel grid, followed by 3D deep networks for feature extraction and segmentation. SEGCloud [40] combines a volumentric convolutional network with trilinear interpolation to return voxel predictions to points, followed by conditional random field refinement. Submanifold sparse convolutions [14] concentrate feature computation on active sites while preserving sparsity through the corresponding convolutional layers. For outdoor scans, Cylinder3D [59] uses cylindrical partitioning and asymmetrical convolutions to accout for the variation of point density with range. Hybrid designs combine the spatial aggregation of voxels with point-level detail. The sparse point-voxel convolution introduced by Tang *et al.*[39] joins a sparse voxel branch

with a high-resolution point branch, and SPVNAS searches architectures built from this operation. Based on SPVCNN, LSK3DNet [12] combines spatial-wise dynamic sparsity (SDS) and channel-wise weight selection (CWS) to achieve a efficient and effective large sparse kernels.These developments show how voxel processing has evolved towards sparse computation, sensor-aware partitioning, and exchange between point and voxel features.

Point-based methods focused on directly exploring deep neural networks on raw point clouds. PointNet [31] learns shared point-wise features and combines them through symmetric pooling. It segmentation network uses a global descriptor together with local point features to predict a label for each input point. PointNet++ [32] extends this design by hierarchically sampling and grouping neighborhoods, allowing successive layers to learn local patterns over increasing spatial scales. DGCNN [42] introduces EdgeConv and recomputes neighbourhood graphs in feature space, so that the relations used for feature aggregation evolve with the learned representation. These methods develop the treatment of context from global pooling to local hierarchies and learned neighborhoods relationships.

Point Transformer [53] applies self-attention with positional information within point neighborhoods, allowing feature aggregation to depend on their content and spatial relationship. Point Transformer V3 [47] places greater emphasis on scalable processing. It organizes points through serialization and uses the resulting neighborhoods for attention, making larger receptive fields practical without repeated precise nearest neighbor searches for attention grouping. The progression is therefore also a change in how context is accessed: from explicitly defined local groups to content dependent aggregation over efficiently organized point sequences. Alongside the development of geometric representation, recent research investigates how 3D models acquire semantic knowledge before or during training. Sonata [46] studies self-supervised point cloud representations through self-distillation. It training design obscures readily available spatial information and encourages the model to use input features, producing representations that can be adapted through linear probing or fine-tuning.

Cross-modal learning provides a complementary route by exploiting paired images and point clouds. A useful distinction concerns that availability of images at deployment. In 2D-3D fusion, image and point clouds features jointly contribute to each prediction. In distillation, a model learns from the paired observations and subsequently predicts from its designated modality.

2DPASS [49] develops this arrangement through auxiliary modality fusion and multi-scale fusion-to-single knowledge distillation. During training, image and point cloud features form fused predictions that supervise the 3D branch at multiple scales. At inference, the trained branch performs segmentation from LiDAR alone. The method establishes a direct predecessor for transferring mul-

Table 2.1: Overview of 3D semantic segmentation approaches.

Category	**Representative methods**	**Main idea**
Projection-based	SnapNet [5]; RangeNet++ [26]; RangeFormer [21]	Predict labels in projected views and map them back to 3D points.
Voxel-based	SEGCloud [40]; submanifold sparse CNNs [14]; Cylinder3D [59]	Apply dense or sparse 3D convolutions to voxelized point clouds.
Point–voxel	SPVNAS [39]; LSK3DNet [12]	Combine voxel aggregation with point-level processing.
Point-based	PointNet [31]; PointNet++ [32]; DGCNN [42]; Point Transformer [53]; Point Transformer V3 [47]	Learn directly from points using pooling, graphs, or attention.
Pretraining and transfer	Sonata [46]; 2DPASS [49]; DINO in the Room [19]	Learn reusable features through 3D pretraining or cross-modal knowledge transfer.

timodal semantic information into the LiDAR predictor used in this thesis.

Vision foundation models extend this direction by supplying representations learned from large image collections. DINOv3 [36] provides dense visual features learned through self-supervision, which can be adapted to a downstream semantic label space. DINO in the Room [19] studies two ways to bring pretrained visual features into 3D segmentation: projecting and injecting them into a point cloud model, and distilling them into a 3D backbone during pretraining. Its distillation route supports subsequent segmentation when images are unavailable at inference. Together with direct 3D pretraining, these approaches broaden the sources of information available to a segmentation network beyond its task-specific labeled point clouds. Table 2.1 summarizes the approaches discussed above.

Building on these approaches, this thesis connects pretrained visual knowledge with persistent semantic LiDAR mapping. We combine DINOv3 image features and point cloud features in a fusion teacher, and transfer the resulting semantic knowledge to a LiDAR student through multi-scale distillation. At inference, the student predicts point-wise class probabilities from LiDAR observations alone. These labels supervise a semantic decoder and shared map features as successive scans update the neural point map of PIN-SLAM.

2.2 Semantic SLAM

Semantic simultaneous localization and mapping (SLAM) incorporates semantic information into the estimation of sensor motion and the construction of an environmental map. Semantic information can describe category labels, individual objects, or learned scene features. Its role depends on the system: it may enrich the map, support data association and pose estimation, or guide the selection of observations used for reconstruction. For the present thesis, the central question is how semantic observations become persistent evidence in a map as the sensor moves through the environment.

Fusion++ [25] develops object-level mapping through incremental reconstruction. It uses instance segmentations from RGB-D observations to initialize a separate truncated signed distance function (TSDF) volume for each object. Figure 2.2 explains what is TSDF. The reconstructed objects form an optimizable pose graph and support tracking, relocalization, and loop closure. Semantic information and object existence probabilities are updated over time. This organization connects object identity, reconstructed shape, and persistent map state.

SemanticFusion [24] combines convolutional image segmentation with the surfel reconstruction of ElasticFusion. Geometric correspondences associate predictions from successive RGB-D views with map surfels, where class probabilities are fused. The map therefore accumulates semantic observations across viewpoints.

Semantics can also guide which observations contribute to localization and mapping. DynaSLAM [4] extends ORB-SLAM2 with dynamic object detection and background inpainting. It uses instance segmentation, multi-view geometry, or their combination to identify dynamic scene content and reconstruct the static background. This approach connects semantic recognition to the selection of measurements used for tracking, making scene dynamics an explicit consideration in the SLAM pipeline.

For LiDAR, SuMa++ [7] integrates point-wise semantic predictions into a surfel-based SLAM system. A convolutional network processes spherical projections of laser scans, and the resulting labels are associated with 3D measurements and map surfels. Semantics supports both moving-object filtering and projective scan matching through semantic constraints. SuMa++ is particularly relevant to this thesis because it connects LiDAR segmentation to persistent map construction and shows how semantic information can participate in geometric estimation.

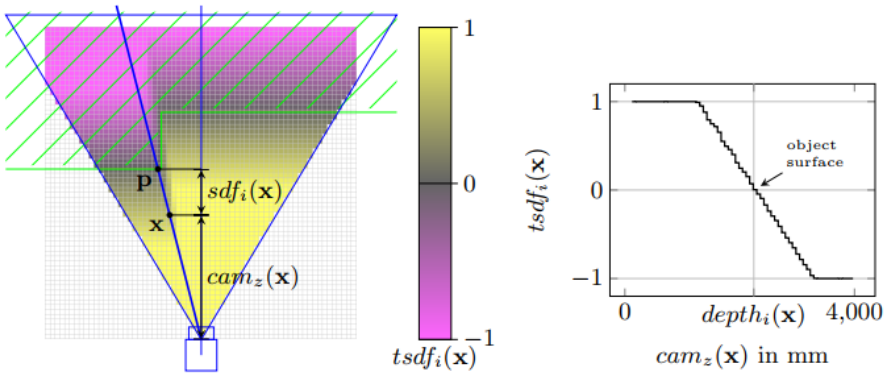


Figure 2.2: The TSDF value is derived from the signed distance function (SDF) value. As shown in the figure, the white and grap squares represent the individual voxels in the TSDF map. The bule triangles represent the camera's field of view. A green cross-section line in the center of the figure represents the cross-section of an object. The depth map from a depth camera can display depth information for the object's cross-section. We need to calculate the SDF and TSDF values for the voxels, namely SDF(x) and TSDF(x) .

These systems highlight a distinction between maintaining a semantic map and using semantics to influence the estimated trajectory. Both can occur within one system, and each involves a specific mechanism: associating predictions with map elements, selecting measurements, or introducing semantic constraints into registration. This distinction provides a useful basis for comparing systems with different sensors and map representations.

Neural implicit representations introduce learned functions for encoding scene properties. Semantic-NeRF [56] jointly represents semantics, appearance, and geometry and uses multi-view consistency to propagate and refine semantic labels. Developed for scene reconstruction with known camera poses, it provides a foundation for learning semantic scene representations. Its relevance to SLAM lies in representing semantic evidence within the same learned scene model as geometry and appearance.

SNI-SLAM [58] brings this joint representation into online tracking and mapping. It uses hierarchical semantic features and combines appearance, geometry, and semantics through cross-attention. A fusion-based decoder predicts color, signed distance, and semantic information, while a feature-level loss supervises the learned representation. The system illustrates how semantic reasoning can be incorporated into the optimization of a neural scene model during SLAM.

Recent systems also use explicit 3D Gaussian primitives. SGS-SLAM [23] combines appearance, geometry, and semantic information through multi-channel optimization. Its semantic feature loss and semantic-guided keyframe selection connect scene interpretation to reconstruction and map updates. This formulation extends the attributes of the Gaussian map beyond color and spatial structure, making semantic information part of the representation used by the SLAM

system.

Across these developments, the map representation determines where semantic evidence is stored, while the optimization strategy determines how that evidence interacts with motion estimation. This thesis adopts the neural points of PIN-SLAM [30] as the common spatial support for geometric reconstruction and semantic map learning. PIN-SLAM provides geometric alignment, and the LiDAR student described in the preceding section supplies semantic observations informed.

Chapter 3

Basic Techniques

THIS chapter provides an introduction to the task of how to use VFM, LiDAR-Camera calibration, feature association, semantic fusion in SLAM and 3D semantic segmentation evaluation. Section 3.1 introduces what is vision foundation model and why to choose DINOv3 as the VFM which providing high-quality dense features. Section 3.2 explain how to establishes correspondences between DINO features and LiDAR point clouds. Section 3.3 provides briefly discusses the evaluation of semantic segmentation in SLAM.

3.1 Vision Foundation Model and DINOv3

This section introduces vision foundation models (VFM), explains how the DINO family learns reusable visual features, and describes the dense features supplied by DINOv3 to the proposed learning pipeline.

A VFM is pretrained on massive and diverse visual dataset to provide representations that can be reused across downstream tasks [36]. An image encoder transforms pixel values into features describing visual appearance and scene structure. A downstream component then uses these features for a particular prediction. The pretrained encoder can be kept fixed while this component is trained, or adapted together with it.

Common image encoders use convolutional neural networks (CNNs) [22] or Vision Transformers (ViTs) [11]. CNNs apply shared filters over local image regions and build increasingly contextual feature maps through successive layers. A ViT divides an image into patches and processes their representations as a sequence of tokens. Self-attention allows each token to gather information from other tokens.

The surrounding architecture depends on how these features are used. A visual encoder can supply a classifier, a retrieval system, or a decoder for se-

mantic segmentation and depth estimation. CLIP [33] combines image and text encoders in a shared embedding space, enabling image-text retrieval and classification through text descriptions. The Segment Anything Model (SAM) [17] combines an image encoder, a prompt encoder, and a mask decoder to produce segmentation masks from prompts such as points or boxes. Figure 3.1 summarizes these representative arrangements. In this thesis, we use DINOv3 [36] as a visual backbone to extract dense image features. Using camera intrinsics and LiDAR-to-camera extrinsics, we establish correspondence between LiDAR points and image locations, then extract the corresponding image features from patches and associate them with the 3D points.

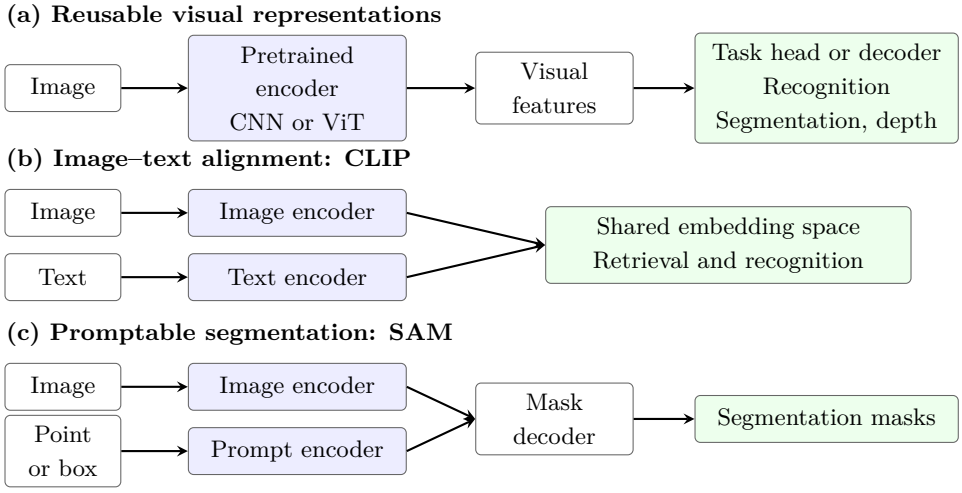


Figure 3.1: Representative architectures using pretrained visual encoders: (a) visual features reused by task-specific heads; (b) image–text alignment in CLIP [33]; (c) prompt-conditioned segmentation in SAM [17].

The first version of DINO, self-distillation with no labels (DINO) [6], learns image representations by matching prediction from different views of the same image. A student network is optimized to match a teacher, whose parameters follow an exponential moving average of the student:

$$\boldsymbol{\theta}_T \leftarrow m\boldsymbol{\theta}_T + (1-m)\boldsymbol{\theta}_S, \qquad 0 < m < 1. \tag{3.1}$$

Here, m is the EMA momentum coefficient controlling how slowly the teacher follows the student, in other words, m controls the update rate. Centering and sharpening the teacher outputs help prevent a constant representation for all inputs. The original DINO study showed that self-supervised ViT features can encode object structure, even though the training objective uses image-level predictions.

DINOv2 [29] develops this principle into broadly reusable features through curated data and training at larger scale. The student predicts teacher targets at

masked patch locations, while the teacher processes the corresponding unmasked input. KoLeo regularization and efficient training support this combination. The result is a representation evaluated for both global recognition and dense prediction.

DINOv3 [36] continues from the DINOv2 weights but adds an important loss term, Gram anchoring. The Gram-anchoring loss is

$$\mathcal{L}_{\text{Gram}} = \left\| \mathbf{X}_S \mathbf{X}_S^\top - \mathbf{X}_G \mathbf{X}_G^\top \right\|_F^2 . \tag{3.2}$$

Here, $\mathbf{X}_S, \mathbf{X}_G \in \mathbb{R}^{N \times D}$ contain the L_2-normalized patch features of the student and the reference Gram teacher, respectively, where N is the number of patches and D is the feature dimension.

This term interacts mostly with the iBOT loss which works on patch level and was introduced to keep a similar patch ranking between the teacher and student. Most importantly, this helped with the dense feature representation as they show in the paper and with adjustments to crop sizes it also scales to higher input resolutions. Figure 3.2 shows the visualization PCA results of features obtained from images with DINOv3. Table 3.1 summarizes the progression relevant here.

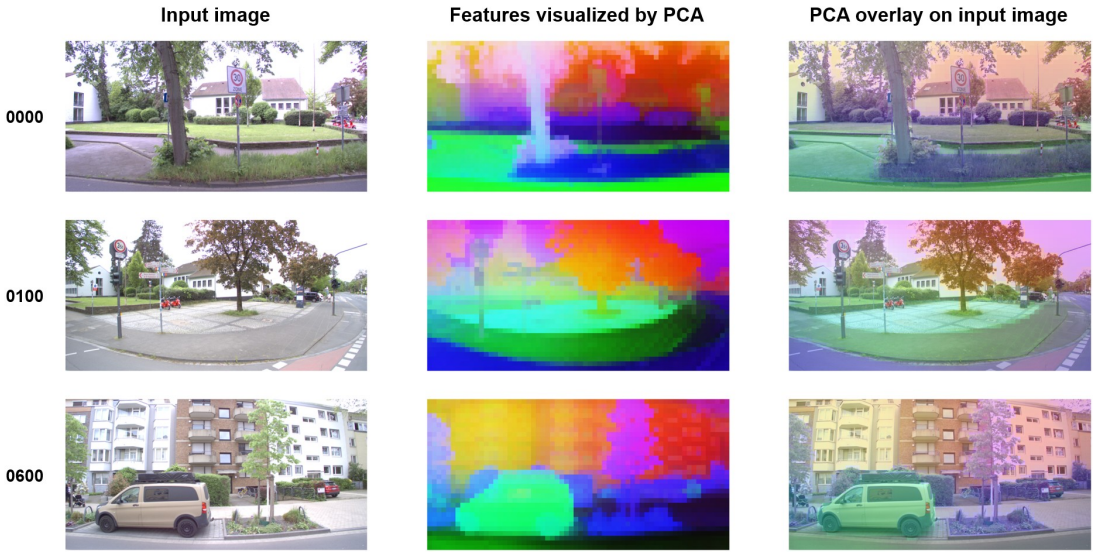


Figure 3.2: Visualization of dense features of DINOv3 by mapping the first three components of a PCA computed over the feature space to RGB.

The DINOv3 encoder used in this thesis is ViTL/16 [36]. After padding the processed image dimensions H and W to multiples of 16, it divides the RGB image into non-overlapping 16×16 patches. A learned linear projection maps each flattened patch to a $D = 1024$ dimensional token. With $H_p = H/16$ and $W_p = W/16$, the image provides $N = H_p W_p$ patch tokens. One class token and four auxiliary storage tokens are added, giving a sequence of $N + 5$ tokens.

Table 3.1: Comparison of DINO, DINOv2, and DINOv3.

Aspect	**DINO** 2021 [6]	**DINOv2** 2024 [29]	**DINOv3** 2025 [36]
Contribution	Cross-crop self-distillation with an EMA teacher	Reusable features through data curation and scaling	Further scaling with improved dense-feature quality
Objectives	Image-level cross-entropy; centering and sharpening	DINO + iBOT; KoLeo regularization	DINO + iBOT; distributed KoLeo; Gram anchoring during large-model refinement
Data	ImageNet-1K, without class labels	LVD-142M: 142M curated images	Data mixture centred on LVD-1689M (1.689B curated images)
Models	Main ViTs: ViT-S (21M) and ViT-B (85M)	ViT-g teacher (1.1B); distilled ViT-S, ViT-B, and ViT-L	ViT-7B teacher ($\sim$6.7B); distilled ViT and ConvNeXt models
Features	Recognition features; object structure in attention maps	Transferable global and patch-level features	Global descriptors and improved dense features

M and B denote million and billion, respectively. Parameter counts are approximate. DINOv2 was released in 2023; 2024 is its journal publication year.

The token sequence is processed by 24 transformer blocks, in which self-attention allows each patch token to aggregate information from other tokens [36]. The resulting patch descriptors combine local visual information with broader image context while retaining their correspondence to the original patch locations. These spatially indexed features can then be associated with LiDAR points through image projection.

After the final block and output layer normalization, the N patch tokens are retained and reshaped into a spatial feature map:

$$\mathbf{F} = E_{\text{DINO}}(\mathbf{I}) \in \mathbb{R}^{H_p \times W_p \times 1024}, \qquad \mathbf{d}_{a,b} = \mathbf{F}[a, b, :] \in \mathbb{R}^{1024}. \tag{3.3}$$

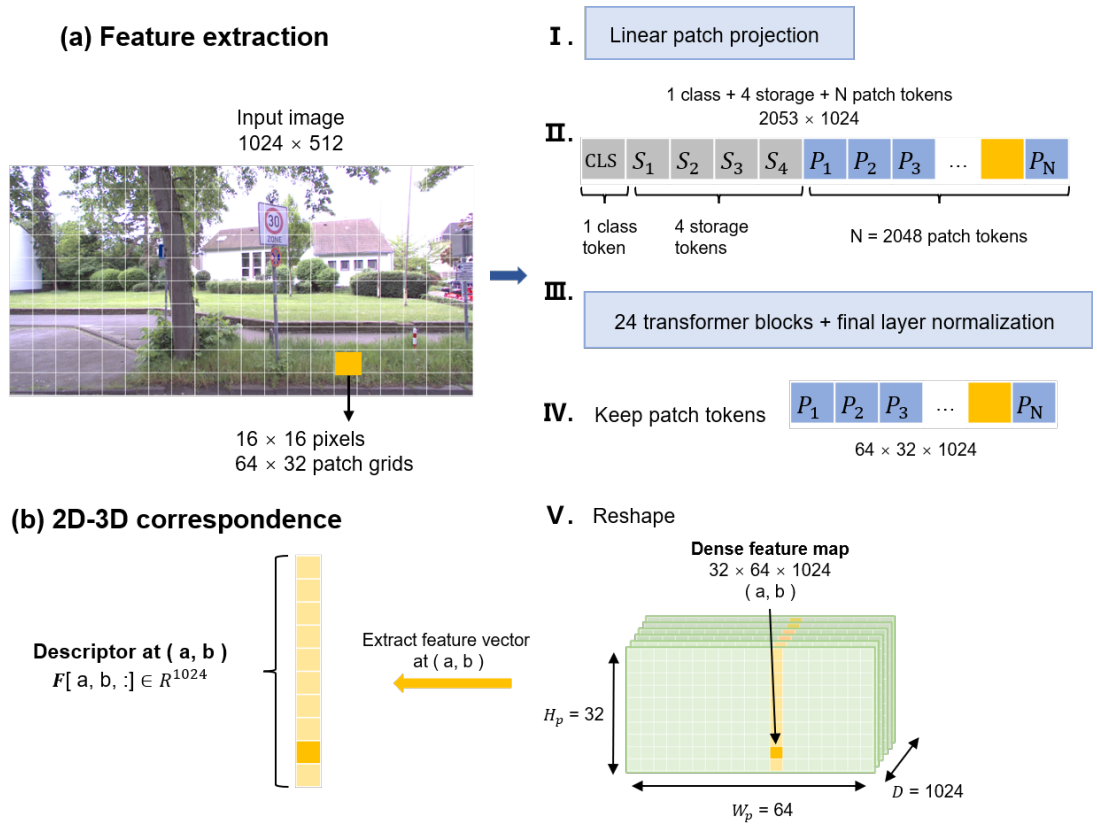


Figure 3.3: Schematic of patch feature extraction with frozen DINOv3 ViT-L/16. The final patch descriptors retain their spatial indices and form the grid used for LiDAR–image association.

The class and storage tokens contribute to processing but are excluded from this map. The indices (a, b) retain the association with an image patch, whereas the channel dimension encodes its learned descriptor. Thus, dense features form a regular patch grid. Semantic class probabilities are obtained later through a semantic segementation prediction head. Figure 3.3 illustrates this transformation.

3.2 LiDAR-Camera calibration

A camera sensor measures the environment of a scene in image coordinates and a LiDAR sensor measures 3D positions relative to its own sensor coordinate system. This section provide a summary of how LiDAR-Camera calibration works which is essential for achieve 2D-3D projection in this thesis. Calibration establishes the geometric relationship between LiDAR and camera. Camera intrinsics describe the process of projecting points from camera coordinate system to the sensor coordinate system. Sensor extrinsics describe the pose of the camrea with respect to the world. Together, these parameters allow a 2D image observation to be associated with a measured 3D point. [44] The following discussion considers a rigidly mounted sensor pair and spatially corresponding observations.

Let L and C denote the LiDAR and camera coordinate frames, respectively.The same physical point has coordinates $\mathbf{p}_L$ in the LiDAR frame and $\mathbf{p}_C$ in the camera frame. Their relationship is

$$ {}^{C}p = {}^{C}\mathbf{R}_L\mathbf{p}_L + {}^{C}\mathbf{t}_L, \qquad ({}^{C}\mathbf{R}_L)^{\mathsf{T}}{}^{C}\mathbf{R}_L = \mathbf{I}, \quad \det({}^{C}\mathbf{R}_L) = 1. \tag{3.4} $$

In ${}^{C}\mathbf{T}_L$, the left superscript C denotes the destination frame and the right subscript L denotes the source frame. The same convention is used for rotation and translation. The rotation matrix ${}^{C}\mathbf{R}_L \in \mathbb{R}^{3\times3}$ aligns the coordinate axes, and the translation vector ${}^{C}\mathbf{t}_L \in \mathbb{R}^3$ accounts for the displacement of their origins. Rotation and translation contribute 3 degrees of freedom each.

Using homogeneous coordinates, rotation and translation can be written as a single matrix multiplication:

$$ \begin{aligned} &\begin{bmatrix} {}^{C}p \\ 1 \end{bmatrix} = {}^{C}\mathbf{T}_L \begin{bmatrix} \mathbf{p}_L \\ 1 \end{bmatrix}, \quad {}^{C}\mathbf{T}_L = \begin{bmatrix} {}^{C}\mathbf{R}_L & {}^{C}\mathbf{t}_L \\ \mathbf{0}^{\mathsf{T}} & 1 \end{bmatrix}, \\ &{}^{L}\mathbf{T}_C = ({}^{C}\mathbf{T}_L)^{-1}, \qquad {}^{L}\mathbf{T}_C = \begin{bmatrix} ({}^{C}\mathbf{R}_L)^{\mathsf{T}} & -({}^{C}\mathbf{R}_L)^{\mathsf{T}}{}^{C}\mathbf{t}_L \\ \mathbf{0}^{\mathsf{T}} & 1 \end{bmatrix}. \end{aligned} \tag{3.5} $$

The inverse maps camera coordinates back to LiDAR coordinates. Transformations through an intermediate frame compose by matrix multiplication: ${}^{A}\mathbf{T}_C = {}^{A}\mathbf{T}_B{}^{B}\mathbf{T}_C$, with the rightmost transformation applied first.

The superscripts k, c, and s denote the camera coordinate system, image plane coordinate system, and sensor coordinate systems, respectively. The object coordinate system serves as the reference frame. Taking the distance from the camera origin to the image plane is the constant c, which is $c < 0$. The scalar c specifies the position of the image plane relative to the projection center, it is distinct from the coordinate system superscript c.

Let $\mathcal{P}$ be a 3D point and $\overline{\mathcal{P}}$ its projection onto the image plane, the coordinates

are:

$$\begin{aligned} {}^{c}x_{\overline{\mathcal{P}}} &:= {}^{k}X_{\overline{\mathcal{P}}} = c\,\frac{{}^{k}X_{\mathcal{P}}}{{}^{k}Z_{\mathcal{P}}}, \\ {}^{c}y_{\overline{\mathcal{P}}} &:= {}^{k}Y_{\overline{\mathcal{P}}} = c\,\frac{{}^{k}Y_{\mathcal{P}}}{{}^{k}Z_{\mathcal{P}}}. \end{aligned} \tag{3.6}$$

The quantities ${}^{k}X_{\mathcal{P}}$, ${}^{k}Y_{\mathcal{P}}$, and ${}^{k}Z_{\mathcal{P}}$ are the 3D point's camera coordinates. The projection lies on the image plane, whose third camera coordinate is fixed:

$$c = {}^{k}Z_{\overline{\mathcal{P}}} = c\,\frac{{}^{k}Z_{\mathcal{P}}}{{}^{k}Z_{\mathcal{P}}}. \tag{3.7}$$

Following the camera projection formulation of Wiesmann et al. [44], a 3D point expressed in the camera coordinate system is first projected into normalized image coordinates. The projection function depends on the camera model. For a point in the camera frame $\mathbf{P}_c = [X_c, Y_c, Z_c]^\top$, the pinhole and equidistant fisheye projections are given by

$$\begin{aligned} \mathbf{x}_{\mathrm{u}} &= \mathrm{project}(\mathbf{P}_c) \\ &= \begin{cases} \begin{bmatrix} \frac{X_c}{Z_c} \\ \frac{Y_c}{Z_c} \end{bmatrix}, & \text{pinhole model,} \\ \frac{\theta}{r_{xy}} \begin{bmatrix} X_c \\ Y_c \end{bmatrix}, & \text{equidistant fisheye model,} \end{cases} \end{aligned} \tag{3.8}$$

where

$$r_{xy} = \sqrt{X_c^2 + Y_c^2}, \qquad \theta = \mathrm{atan2}(r_{xy}, Z_c). \tag{3.9}$$

Here, r_{xy} is the radial distance of the 3D point from the camera's optical axis, and θ is the angle between the corresponding camera ray and the positive optical axis. The vector $\mathbf{x}_{\mathrm{u}} = [x_{\mathrm{u}}, y_{\mathrm{u}}]^\top$ contains the undistorted normalized image coordinates.

In this formulation, radial and tangential distortion are described by

$$\begin{aligned} \mathbf{x}_{\mathrm{d}} &= \mathrm{distort}(\mathbf{x}_{\mathrm{u}}) \\ &= \mathbf{x}_{\mathrm{u}} \left(1 + \sum_{n=1}^{N} k_n r^{2n} \right)^{\tau} \\ &\quad + 2\mathbf{x}_{\mathrm{u}}\mathbf{x}_{\mathrm{u}}^\top \mathbf{p} + r^2 \mathbf{p}, \end{aligned} \tag{3.10}$$

where $r^2 = x_{\mathrm{u}}^2 + y_{\mathrm{u}}^2$, $\{k_n\}_{n=1}^{N}$ are the radial distortion coefficients, and $\mathbf{p} = [p_2, p_1]^\top$ contains the tangential distortion coefficients. The choice $\tau = 1$ corresponds to the Brown distortion model, while $\tau = -1$ gives the division model.

The distorted normalized coordinates $\mathbf{x}_{\mathrm{d}} = [x_{\mathrm{d}}, y_{\mathrm{d}}]^\top$ are then mapped to pixel coordinates (u, v) by

$$\begin{bmatrix} u \\ v \end{bmatrix} = \begin{bmatrix} f_x & 0 \\ 0 & f_y \end{bmatrix} \begin{bmatrix} x_{\mathrm{d}} \\ y_{\mathrm{d}} \end{bmatrix} + \begin{bmatrix} c_x \\ c_y \end{bmatrix}. \tag{3.11}$$

Here, f_x and f_y are the focal lengths expressed in pixel units along the horizontal and vertical image axes, respectively, and (c_x, c_y) are the principal point coordinates in pixels.

The equivalent homogeneous representation uses the intrinsic matrix

$$\mathsf{K} \doteq \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}, \tag{3.12}$$

such that $[u, v, 1]^\top = \mathsf{K}[x_\mathrm{d}, y_\mathrm{d}, 1]^\top$. This parameterization fixes skew to zero and uses four parameters for the intrinsic matrix, with lens distortion modeled separately.

Combining the extrinsic transformation, camera projection, lens distortion, and conversion to pixel coordinates, the complete mapping of a LiDAR point into the image is

$$\begin{aligned} \mathbf{p}_C &= {}^C\mathbf{R}_L \mathbf{p}_L + {}^C\mathbf{t}_L, \\ \mathbf{x}_\mathrm{u} &= \mathrm{project}(\mathbf{p}_C), \\ \mathbf{x}_\mathrm{d} &= \mathrm{distort}(\mathbf{x}_\mathrm{u}), \\ \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} &= \mathsf{K} \begin{bmatrix} \mathbf{x}_\mathrm{d} \\ 1 \end{bmatrix}. \end{aligned} \tag{3.13}$$

The projection function is selected according to the camera model, and lens distortion is applied in normalized image coordinates before conversion to pixels. In the figure 3.4 shows the overview of mapping process from object coordinate system to sensor system.

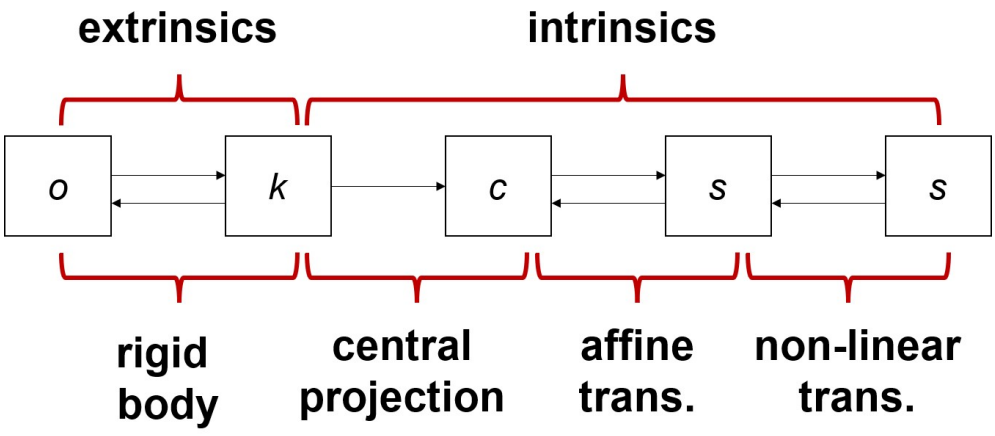


Figure 3.4: The overview of mapping

3.3 3D semantic segmentation evaluation

In this section, we introduce the evaluation metric for semantic segmentation results. 3D semantic segmentation assigns a semantic class to each point in a point cloud. Evaluation compares these predictions with reference labels at the same point locations. In this thesis, evaluation is performed on the prediction output from semantic segmentation module.

The evaluation follows the 19-class semantic label mapping of SemanticKITTI [2]. Let $\mathcal{C} = \{1, \ldots, C\}$ denote the evaluated classes, with $C = 19$. Label 0 denotes the ignored category. $\widehat{y}_{t,i}$ is the prediction result for point i in scan t. $y_{t,i}$ is the value from ground truth label for point i in scan t.

Let $\mathcal{V}$ contain the points inside the spatial region of interest whose reference labels belong to $\mathcal{C}$. These points form the evaluation set. Points with ignored reference labels are excluded, while a prediction of label 0 for a valid reference point counts as an error.

Prediction counts are accumulated in a confusion matrix $\mathbf{M} \in \mathbb{N}^{(C+1)\times(C+1)}$, with reference classes in rows and predicted classes in columns:

$$M_{ab} = \sum_{(t,i)\in\mathcal{V}} \mathbf{1}[y_{t,i} = a]\,\mathbf{1}[\widehat{y}_{t,i} = b], \qquad a, b \in \{0, \ldots, C\}. \tag{3.14}$$

Here, $\mathbf{1}[\cdot]$ is one when its condition holds and zero otherwise. The matrix is accumulated over all validation scans before the metrics are computed. This gives a dataset-level evaluation based on point counts.

For semantic class $c \in \mathcal{C}$, the true-positive, false-positive, and false-negative counts are

$$\begin{aligned} \mathrm{TP}_c &= M_{cc}, \\ \mathrm{FP}_c &= \sum_{a=0}^{C} M_{ac} - M_{cc}, \\ \mathrm{FN}_c &= \sum_{b=0}^{C} M_{cb} - M_{cc}. \end{aligned} \tag{3.15}$$

True positives are correctly classified points of class c. False positives are points from other classes predicted as c. False negatives are reference points of class c assigned another label, including label 0. Figure 3.5 shows an example of TP, FP, FN on a true scan of SemanticKITTI dataset.



Figure 3.5: The explanation of how to calculate TP, FP, FN on prediction and ground truth label. In this case, LiDAR point clouds of SemanticKITTI sequence 09, 000888 frame was projected on it correspondence image, compare the ground truth label and prediction point by point.

Intersection over union (IoU) measures the overlap between the predicted and reference point sets of each class. The class IoU and mean IoU are defined as

$$\begin{aligned} \mathrm{IoU}_c &= \frac{\mathrm{TP}_c}{\mathrm{TP}_c + \mathrm{FP}_c + \mathrm{FN}_c}, \\ \mathrm{mIoU} &= \frac{1}{|\mathcal{C}_+|} \sum_{c \in \mathcal{C}_+} \mathrm{IoU}_c. \end{aligned} \tag{3.16}$$

The set $\mathcal{C}_+$ contains the evaluated classes with a nonzero union. When all 19 classes have a nonzero union, mIoU is their arithmetic mean. Each class receives equal weight, allowing the metric to reflect performance on both frequent and less frequent classes. Per class IoU further identifies which semantic categories contribute to the overall result.

Accuracy (ACC) in semantic segmentation refers to overall accuracy (OA). For each valid LiDAR point, evaluate whether the categories of prediction is same with the ground truth label. Here is the equation of ACC:

$$\mathrm{ACC} = \frac{\sum_{c=1}^{C} M_{cc}}{|\mathcal{V}|}. \tag{3.17}$$

Compare with IoU or mIoU, ACC do not need to choose one specific class of true positive. Every point only evaluate for once. Figure 3.6 illustrates how to calculate acc.

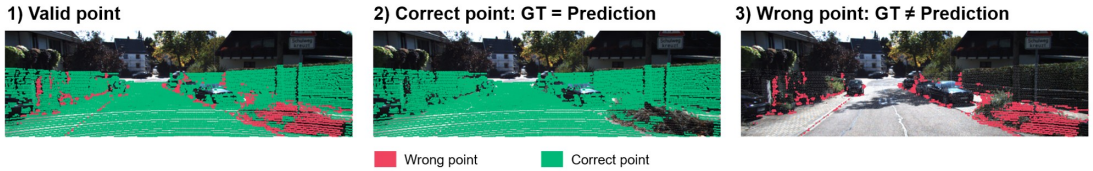


Figure 3.6: The explanation of how to calculate ACC on prediction and ground truth label. In this case, LiDAR point clouds of SemanticKITTI sequence 09, 000888 frame was projected on it correspondence image. The color only means whether corret or not, do not mean any semantic class in this case. 1) shows the origin points; 2) the green color point means the true positive prediction point of all classes; 3) the red color points means the prediction point of all classes are different with ground truth labels.

3.4 Odometry Evaluation with evo

Odometry evaluation measures the agreement between an estimated trajectory and a reference trajectory. It provides information about both the overall consistency of the estimated path and the accuracy of local motion estimates. In this thesis, trajectory evaluation is performed using `evo`, a software package for evaluating odometry and SLAM systems [16]. The evaluation consists of preparing corresponding trajectories, aligning their coordinate frames where required, and computing absolute and relative errors.

Let $\mathbf{T}_i \in \mathrm{SE}(3)$ and $\hat{\mathbf{T}}_i \in \mathrm{SE}(3)$ denote the reference and estimated poses, respectively. Before comparison, both trajectories must describe the same sensor frame and use a consistent pose convention. The trajectory files can be supplied in formats supported by `evo`, including KITTI and TUM. In KITTI format, each row contains the first three rows of a homogeneous pose matrix, stored as twelve values in row-major order. Since this format contains no timestamps, the two files must contain corresponding poses in the same order. In TUM format, each row contains a timestamp, a three-dimensional position, and a quaternion in the order (q_x, q_y, q_z, q_w). Timestamped trajectories can be associated using nearest timestamps within a specified maximum time difference.

An estimated trajectory may be expressed in a different global coordinate frame from the reference. For absolute error evaluation, a single rigid transformation $\mathbf{S} = (\mathbf{R}_s, \mathbf{t}_s) \in \mathrm{SE}(3)$ is therefore estimated from the corresponding trajectory positions. In `evo`, the `--align` option performs this alignment using Umeyama's method. The aligned estimated position is

$$\hat{\mathbf{t}}_i^{\,a} = \mathbf{R}_s \hat{\mathbf{t}}_i + \mathbf{t}_s, \tag{3.18}$$

where $\hat{\mathbf{t}}_i$ is the translation component of $\hat{\mathbf{T}}_i$. The same transformation is applied to the entire estimated trajectory. Since LiDAR odometry provides metric scale, the evaluation uses rigid alignment with scale correction disabled.

Following the trajectory evaluation formulation of Sturm et al. [38], the aligned absolute error at the i-th corresponding pose is defined as

$$\mathbf{F}_i = \mathbf{T}_i^{-1}\mathbf{S}\hat{\mathbf{T}}_i. \tag{3.19}$$

Its translational magnitude measures the distance between the aligned estimated position and the reference position. For N corresponding poses, the absolute trajectory error is

$$\begin{aligned} \mathrm{ATE}_{\mathrm{RMSE}} &= \sqrt{\frac{1}{N}\sum_{i=1}^{N} \|\mathrm{trans}(\mathbf{F}_i)\|_2^2} \\ &= \sqrt{\frac{1}{N}\sum_{i=1}^{N} \left\|\mathbf{R}_s\hat{\mathbf{t}}_i + \mathbf{t}_s - \mathbf{t}_i\right\|_2^2}, \end{aligned} \tag{3.20}$$

where $\mathrm{trans}(\cdot)$ extracts the translation component of a rigid transformation, and $\mathbf{t}_i$ and $\hat{\mathbf{t}}_i$ are the reference and estimated positions. Thus, ATE in this thesis denotes the RMSE of the translational absolute pose errors after rigid alignment, expressed in meters.

Relative pose error evaluates the discrepancy between the estimated and reference motions over a specified interval Δ [38]. The relative transformations $\mathbf{T}_i^{-1}\mathbf{T}_{i+\Delta}$ and $\hat{\mathbf{T}}_i^{-1}\hat{\mathbf{T}}_{i+\Delta}$ describe the reference and estimated motions between the same two evaluated frames. Their discrepancy is

$$\mathbf{E}_{i,\Delta} = \left(\mathbf{T}_i^{-1}\mathbf{T}_{i+\Delta}\right)^{-1}\left(\hat{\mathbf{T}}_i^{-1}\hat{\mathbf{T}}_{i+\Delta}\right). \tag{3.21}$$

If the two relative motions agree, $\mathbf{E}_{i,\Delta}$ is the identity transformation. Let $\mathbf{t}_{i,\Delta}^E$ and $\mathbf{R}_{i,\Delta}^E$ denote its translation and rotation components. The translational error and rotational error are

$$\begin{aligned} e_{i,\Delta}^{\mathrm{trans}} &= \left\|\mathbf{t}_{i,\Delta}^E\right\|_2, \\ e_{i,\Delta}^{\mathrm{rot}} &= \frac{180}{\pi}\arccos\left(\frac{\mathrm{tr}\left(\mathbf{R}_{i,\Delta}^E\right)-1}{2}\right), \end{aligned} \tag{3.22}$$

where $\mathrm{tr}(\cdot)$ denotes the matrix trace. The rotational error is the angle of the error rotation, expressed in degrees. Using all $N-\Delta$ pose pairs separated by Δ frames, the corresponding RMSE values are

$$\begin{aligned} \mathrm{RPE}_{\mathrm{trans,RMSE}} &= \sqrt{\frac{1}{N-\Delta}\sum_{i=1}^{N-\Delta}\left(e_{i,\Delta}^{\mathrm{trans}}\right)^2}, \\ \mathrm{RPE}_{\mathrm{rot,RMSE}} &= \sqrt{\frac{1}{N-\Delta}\sum_{i=1}^{N-\Delta}\left(e_{i,\Delta}^{\mathrm{rot}}\right)^2}. \end{aligned} \tag{3.23}$$

In this thesis, $\Delta = 1$ compares adjacent poses in the evaluated trajectory. Translational and rotational RPE are reported in meters and degrees, respectively.

Chapter 4

Approach

This approach combines leveraging DINOv3 high quality dense feature semantic learning with persistent semantic LiDAR mapping. During training, a 2D-3D fusion teacher uses calibrated DINO feature and geometry of point clouds to predict KITTI semantic categories. Its class distributions guide a LiDAR-only semantic segmentation student model. During mapping, PIN-SLAM estimates the current pose through geometric registration. The aligned student predictions then supervise the semantic decoder and the neural point features shared with the SDF decoder.

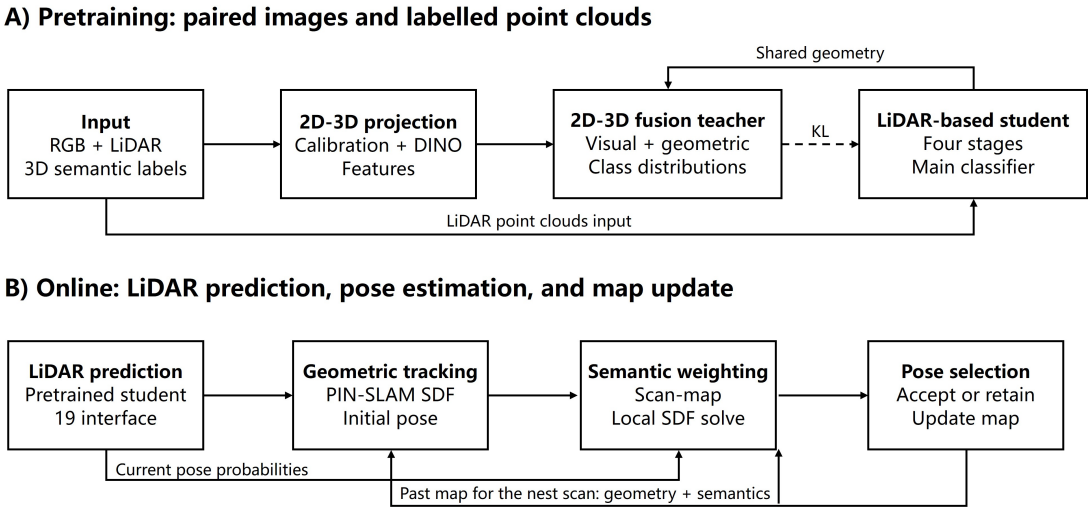


Figure 4.1: Overview of the all pipeline. A) pretraining part connects frozen DINO dense visual featrues to the LiDAR backbone through a 2D-3D fusion teacher. B) the online part combines geometric registration with optional semantic weighting, then updates the map at the selected pose.

Figure 4.1 shows the main components of the method. Section 4.1 defines the inputs and notation. Section 4.2 explains how DINO dense visual features are associated with LiDAR measurement. Section 4.3 describes the LiDAR point

semantic segmentation student model, 2D-3D fusion teacher model, and their joint training.Section 4.4 the online procedure in PIN-SLAM linking prediction, registration and mapping.

4.1 Problem formulation and notation

A training sample consists of a LiDAR scan $P = \{(\mathbf{x}_i^L, i_i, b_i^r)\}_{i=1}^N$, a paired RGB image $\mathbf{I}$, point labels $\{y_i\}_{i=1}^N$, and camera–LiDAR calibration. Here, $\mathbf{x}_i^L \in \mathbb{R}^3$ is a LiDAR-frame coordinate, i_i is the supplied intensity, and b_i^i indicates whether intensity is available and valid. Following the SemanticKITTI taxonomy [2], learning IDs $1, \ldots, 19$ are mapped to network targets $0, \ldots, 18$; ignored learning ID 0 becomes target -1. Figure 4.2 shows the 19 classes of Semantic KITTI. Every prediction head outputs $C = 19$ channels in this order. The same channel order is used by the semantic segmentation model, semantic map, and pose-refinement module.

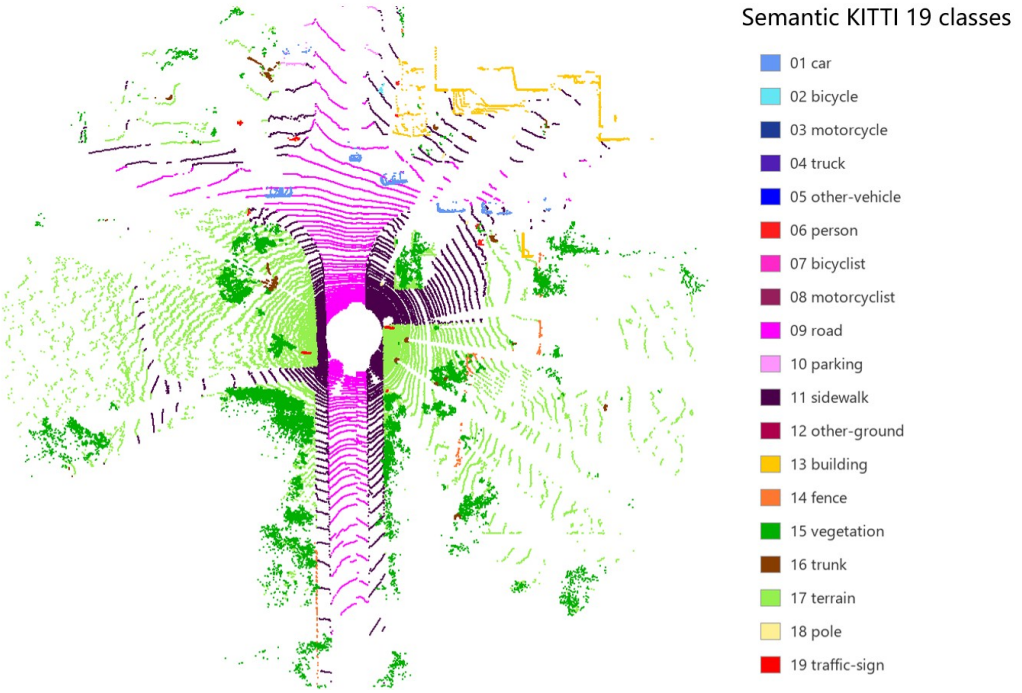


Figure 4.2: Semantic KITTI sequence 08/000001 with 19 classes

We use i for scan points, j for map nodes, $l = 1, \ldots, K$ for the $K = 4$ network stages, and t for scan time. Feature matrices and DINO features are denoted by $\mathbf{F}$ and $\mathbf{f}$; logits and class probabilities are $\mathbf{z}$ and $\mathbf{p}$. Superscripts 2D, 3D, and fuse identify the visual, geometric, and fused representations. We denote training losses by L and pose objectives by E.

A transform $\mathbf{T}_{AB}$ maps coordinates from frame B to frame A. Thus, $\mathbf{T}_{C_0L}$ is the fixed LiDAR to reference camera transform, while $\mathbf{T}_{WL_t}$ is the world pose

estimated for scan t. The signed distance residual is e_i. These conventions are used throughout calibration, learning, and online registration.

Table 4.1: Main notation.

Symbol	Meaning	Size or unit
P, N	Raw LiDAR scan and its point count	N rows
N_{ret}, N_{vis}, M	Retained, image-paired, and valid paired counts	Counts
$\mathbf{x}_i^L$, $\mathbf{x}_i^{\text{aug}}$	Original and augmented point coordinates	$\mathbb{R}^3$, metres
i_i, $\tilde{i}_i$, b_i^r	Supplied intensity, input intensity, valid flag	Scalar; $[0,1]$; $\{0,1\}$
C, K	Semantic categories and network stages	19, 4
$\mathbf{F}_l^{3D}$, $\mathbf{F}_l^{\text{fuse}}$	Geometric and fused feature matrices	$N_{\text{ret}}\times 64$, $N_{\text{vis}}\times 64$
$\mathbf{f}_i^{2D}$, $\tilde{\mathbf{f}}_{l,i}^{2D}$	Frozen and stage adapted visual descriptors	$\mathbb{R}^{1024}$, $\mathbb{R}^{64}$
$\mathbf{F}_{\text{vis}}^{2D}$	Stacked point aligned frozen descriptors	$N_{\text{vis}}\times 1024$
$\mathbf{z}_{h,i}^{3D}$, $\mathbf{z}_{h,i}^{\text{fuse}}$	Per point classification logits	$\mathbb{R}^{19}$
$\mathbf{Z}_h^{3D}$, $\mathbf{Z}_h^{\text{fuse}}$	Stacked student and teacher logits	$N_{\text{ret}}\times 19$, $N_{\text{vis}}\times 19$
$\mathbf{T}_{AB}$	Rigid transform from frame B to frame A	4×4
τ, h	Distillation temperature; head index	2; $0,\ldots,4$
$\mathbf{n}_j$, $\mathbf{f}_j^{\text{map}}$	Map node position and trainable feature	$\mathbb{R}^3$ (m), $\mathbb{R}^8$
$s_{\mathcal{M}}(\mathbf{q})$, e_i	Map SDF and tracking residual	Metres
L, E	Learning loss and pose objective	Scalars

4.2 2D-3D correspondence

The purpose of 2D-3D correspondence is to associate a 3D LiDAR measurement with its location in an image. This requires 2 operations: transforming the point from the LiDAR coordinate frame L into a camera coordinate frame C_k, and projecting the transformed point onto the image plane. Following our notation, $\mathbf{T}_{C_kL}$ maps LiDAR coordinates into camera k, while the intrinsic matrix $\mathbf{K}_k$ determines

the perspective projection:

$$\begin{aligned} \overline{\mathbf{x}}_i^{C_k} &= \mathbf{T}_L^{C_k}\, \overline{\mathbf{x}}_i^{L}, \\ d_{ik} \begin{bmatrix} u_{ik} \\ v_{ik} \\ 1 \end{bmatrix} &= \mathbf{K}_k \begin{bmatrix} \mathbf{I}_3 & \mathbf{0} \end{bmatrix} \overline{\mathbf{x}}_i^{C_k}. \end{aligned} \tag{4.1}$$

Here, $\overline{\mathbf{x}} = [\mathbf{x}^\top, 1]^\top$ denotes a homogeneous 3D point, d_{ik} is its depth in the camera frame, and (u_{ik}, v_{ik}) is its image position in pixels.

For SemanticKITTI dataset, we use the rectified `image_2` view. The supplied calibration first transforms a LiDAR point into the reference camera frame C_0 using $\mathbf{T}_{C_0 L}$. The projection matrix $\mathbf{P}_2 \in \mathbb{R}^{3\times 4}$ then maps that reference camera point to the rectified image, accounting for the intrinsic projection and the offset of camera 2:

$$d_i \begin{bmatrix} u_i \\ v_i \\ 1 \end{bmatrix} = \mathbf{P}_2\, \mathbf{T}_L^{C_0}, \overline{\mathbf{x}}_i^{L}. \tag{4.2}$$

For IPB dataset, 4 undistorted images provide front, left, rear, and right views. Each camera has its own intrinsic matrix $\mathbf{K}_k$ and LiDAR-to-camera transformation $\mathbf{T}_L^{C_k}$, obtained from the sensors' extrinsic calibrations in their common reference frame. Each LiDAR point is transformed into the four camera frames and projected using the corresponding intrinsics. If several cameras provide valid projections, we select the view in which the projected point lies closest to the image center in normalized image coordinates. Thus, each matched point receives one image correspondence.

We use frozen DINOv3 ViT-L/16 patch descriptors [36]. Each image is resized with its aspect ratio preserved and its long side set to 1248 pixels. After RGB normalization, zero padding on the right and bottom makes both dimensions divisible by 16. For padded dimensions $H' \times W'$, the final normalized patch tokens form

$$\mathbf{F}_{\text{grid}}^{2D} = E_{\text{DINO}}(I) \in \mathbb{R}^{H_p \times W_p \times 1024}, \qquad H_p = H'/16, \quad W_p = W'/16. \tag{4.3}$$

During semantic training, DINOv3 features which extract from images stay fixed.

For SemanticKITTI dataset, we bilinearly sample the token grid. Let s_x, s_y be the actual resize factors after rounding the image dimensions. With the image coordinate,the continuous token indices are

$$q_i^u = \frac{s_x(u_i + 1/2)}{16} - \frac{1}{2}, \qquad q_i^v = \frac{s_y(v_i + 1/2)}{16} - \frac{1}{2}. \tag{4.4}$$

Set $j_i^u = \lfloor q_i^u \rfloor$, $j_i^v = \lfloor q_i^v \rfloor$, $\alpha_i = q_i^u - j_i^u$, and $\beta_i = q_i^v - j_i^v$. The descriptor is

$$\mathbf{f}_i^{2D} = \sum_{a=0}^{1} \sum_{b=0}^{1} \omega_a(\alpha_i)\omega_b(\beta_i) \mathbf{F}_{\text{grid}}^{2D}[\text{clip}(j_i^v + b), \text{clip}(j_i^u + a), :], \tag{4.5}$$

where $\omega_0(q) = 1 - q$, $\omega_1(q) = q$, and clipping keeps each index within the token grid.

For the IPB dataset takes the token containing the selected image position. Its horizontal index is $\text{clip}(\lfloor s_x(u_i + 1/2)/16 \rfloor, 0, W_p - 1)$, with the corresponding vertical expression.

Both datasets produce a 1024 dimensional descriptor $\mathbf{f}_i^{2D}$ for each paired point. Stacking these vectors in the order $i_1, \ldots, i_{N_{\text{vis}}}$ gives $\mathbf{F}_{\text{vis}}^{2D} \in \mathbb{R}^{N_{\text{vis}} \times 1024}$.

4.3 DINO-baesd LiDAR semantic segmentation

Following the multi-scale fusion-to-single knowledge distillation strategy of 2DPASS [49], this module uses a LiDAR-only student and a multi-modal fusion teacher. The student model receives 5 channels point attributes comprising XYZ coordinates, remission, and a remission validity indicator, and extracts 64-dimensional point features at four spatial scales. At each scale, the fusion teacher combines these LiDAR features which are computed by student model with corresponding 1024-dimensional image features extracted by a frozen DINOv3 encoder. Figure 4.3 shows the overall semantic segmentation module pipeline.

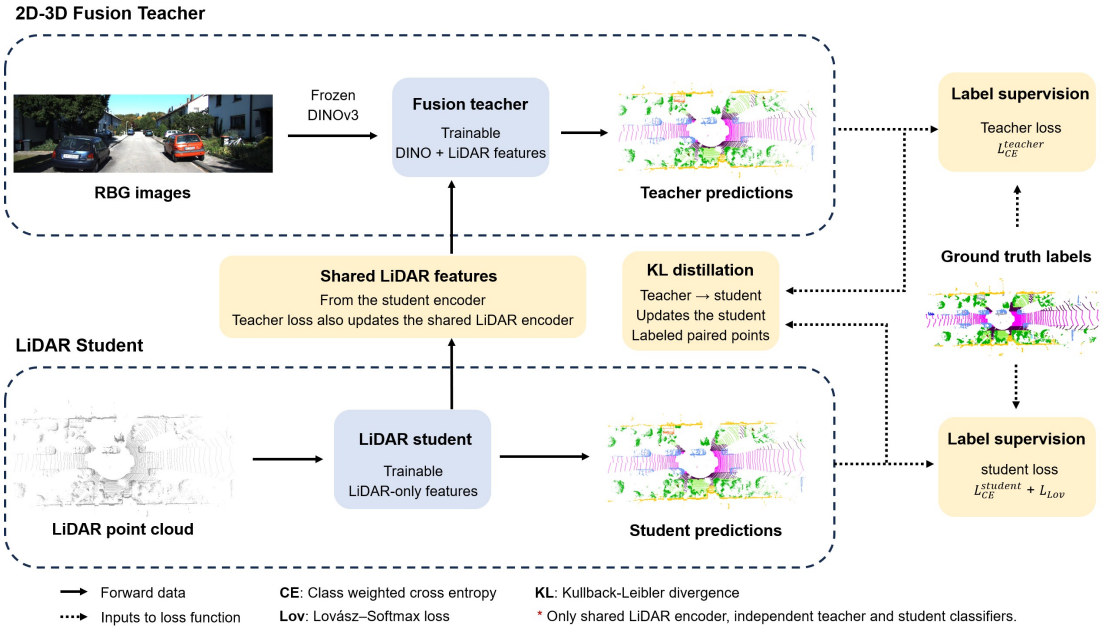


Figure 4.3: Overview of the teacher–student training pipeline for LiDAR semantic segmentation. The fusion teacher combines image features from a frozen DINOv3 encoder with LiDAR features from the student encoder. Groundtruth point labels supervise teacher and student model, while KL distillation aligns the student's class probabilities with those of the teacher on image paired points with valid labels. Teacher and student model share the LiDAR encoder and use separate classifiers. Only the LiDAR student is used at inference.

Both teacher and student model produce 19-class logits through a main clas-

sifier and four auxiliary classifiers: the student predicts labels for all retained points, while the teacher predicts labels for points with image correspondences.

The fusion teacher model reuses the student's LiDAR features, so a single LiDAR encoder serves both branches, while their classifiers have independent parameters.

Ground truth point labels supervise the student on all valid annotated points and the teacher on valid image paired points. The teacher's supervised loss also updates the shared LiDAR encoder because the student features are not detached before fusion. The teacher's adapters, fusion layers, and classifiers are trained jointly with the student, while DINOv3 remains frozen.

For knowledge distillation, KL divergence aligns the student's main and auxiliary class distributions with the corresponding teacher distributions on valid image-paired points. The teacher distributions are detached when computing this loss, ensuring unidirectional distillation from the fusion branch to the LiDAR branch.

At inference, only the LiDAR encoder and the student's main classifier are used. The following paragraphs describe the student architecture and the fusion teacher in detail, followed by the supervised and distillation losses used for joint training.

In this thesis, because of the design of 2DPASS is too compute expensive, design of the student model and teacher model is different from 2DPASS which will give more detail in following context.

The student uses a base voxel size $h_0 = 0.10$ m. For point i, let $\bar{\mathbf{x}}_{v(i)}$ be the mean coordinate in its base voxel and let $\mathbf{c}_{v(i)} = h_0(\lfloor \mathbf{x}_i^{\text{aug}}/h_0 \rfloor + \frac{1}{2}\mathbf{1})$ be the voxel center. The input 3D point feature is

$$\mathbf{f}_{i,\text{entry}}^{3D} = \text{MLP}_{11\to64\to64}\left([\mathbf{a}_i; \mathbf{x}_i^{\text{aug}} - \bar{\mathbf{x}}_{v(i)}; \mathbf{x}_i^{\text{aug}} - \mathbf{c}_{v(i)}]\right). \tag{4.6}$$

The 11 input values comprise five point attributes, which is

$$\mathbf{a}_i = \begin{bmatrix} x_i^{\text{aug}} \\ y_i^{\text{aug}} \\ z_i^{\text{aug}} \\ \tilde{i}_i \\ b_i^r \end{bmatrix} \in \mathbb{R}^5. \tag{4.7}$$

and two 3D offsets. ReLU is placed between the two linear layers.The MLP places ReLU between its two linear layers.

The encoder contains 4 point–voxel blocks with the same structure. Each block has a sparse convolution branch and an MLP branch, as shown in Figure 4.4. The sparse branch uses two residual blocks with sparse convolutions [14] to collect information from neighboring occupied voxels.

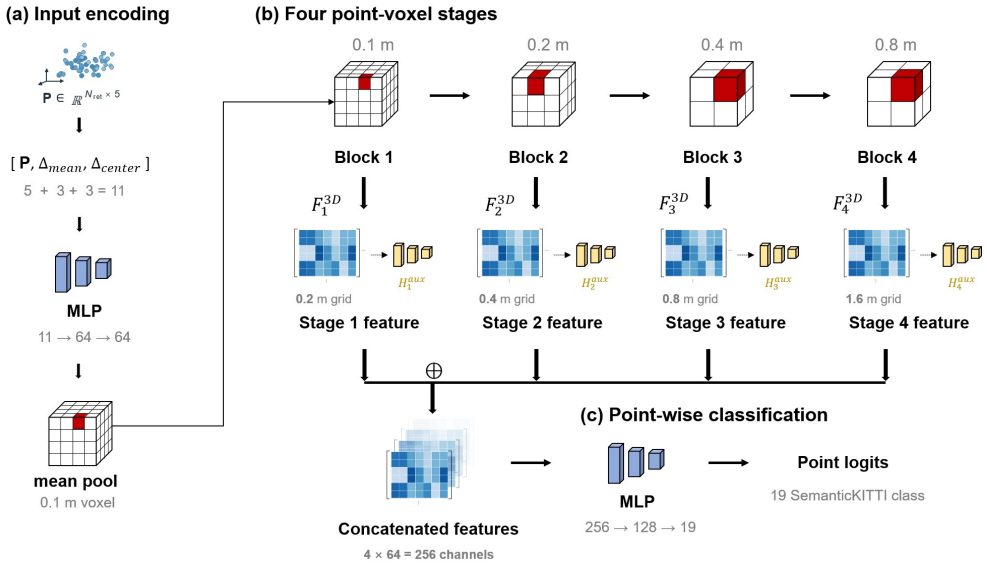


Figure 4.4: LiDAR-only student model architecture. The 11-dimensional point input is embedded into 64 channels, averaged within the initial voxels, and processed by four point–voxel blocks. Each block returns one 64-dimensional feature for every retained point. Concatenating the four outputs gives the 256-dimensional input to the main classifier. The dashed box describes the auxiliary classifiers used during training.

At the end of each block, each point takes the feature of its current voxel from each branch. These point features are then averaged within the next, larger voxels. Adding the two pooled results gives the input to the next block. For the stage output $\mathbf{F}_l^{3D}$, each retained point takes the pooled MLP branch feature of its new voxel. The four stages produce $\mathbf{F}_1^{3D}, \ldots, \mathbf{F}_4^{3D}$ at output voxel sizes of 0.20, 0.40, 0.80, and 1.60 m, respectively. Every output has N_{ret} rows and 64 channels, preserving the retained point order. Table 4.2 lists the voxel sizes.

The main classifier concatenates the four stage outputs along the channel dimension. Each point therefore has a 256D feature that combines the four spatial scales. A $256 \to 128 \to 19$ MLP converts this feature into class logits. During training, each stage also has its own $64 \to 128 \to 19$ auxiliary classifier:

$$\begin{aligned} \mathbf{Z}_0^{3D} &= H_0^{3D}([\mathbf{F}_1^{3D}; \mathbf{F}_2^{3D}; \mathbf{F}_3^{3D}; \mathbf{F}_4^{3D}]), \\ \mathbf{Z}_l^{3D} &= H_l^{3D}(\mathbf{F}_l^{3D}), \qquad l = 1, \ldots, 4. \end{aligned} \tag{4.8}$$

All classifiers use ReLU between their two linear layers. The auxiliary classifiers supervise the intermediate point features during training. At deployment, the encoder and main classifier produce the point-wise predictions.

Table 4.2: Voxel sizes and output dimensions of the student encoder. Every stage returns one feature vector for each retained point.

Stage l	Input voxel size	Output voxel size	Point feature size
1	0.10 m	0.20 m	$N_{\text{ret}} \times 64$
2	0.20 m	0.40 m	$N_{\text{ret}} \times 64$
3	0.40 m	0.80 m	$N_{\text{ret}} \times 64$
4	0.80 m	1.60 m	$N_{\text{ret}} \times 64$
Main classifier: concatenate the four stage outputs			$256 \to 128 \to 19$
Auxiliary classifier: one for each stage			$64 \to 128 \to 19$

Following the fusion-to-single learning strategy of 2DPASS [49], we combine image and LiDAR features at each of the four student stages. The fusion teacher predicts point labels from both modalities and provides supervision for the LiDAR student during training.

Let $\mathcal{V}$ denote the index set of LiDAR points with image correspondences, and let $N_{\text{vis}} = |\mathcal{V}|$. For each paired point $i \in \mathcal{V}$, a separate adapter at stage l reduces the frozen DINO descriptor $\mathbf{f}_i^{2D}$ from 1024 to 64 channels. We concatenate it with the corresponding 64D point feature $\mathbf{f}_{l,i}^{3D}$ and apply a linear layer. A sigmoid then weights the resulting feature channels:

$$\begin{aligned} \tilde{\mathbf{f}}_{l,i}^{2D} &= \mathrm{ReLU}\big(\mathrm{LN}(A_l \mathbf{f}_i^{2D} + \mathbf{b}_l^A)\big)\,, \\ \tilde{\mathbf{f}}_{l,i}^{\text{fuse}} &= B_l[\tilde{\mathbf{f}}_{l,i}^{2D}; \mathbf{f}_{l,i}^{3D}] + \mathbf{b}_l^B, \\ \mathbf{f}_{l,i}^{\text{fuse}} &= \mathrm{ReLU}\Big(\tilde{\mathbf{f}}_{l,i}^{\text{fuse}} \odot \sigma(Q_l \tilde{\mathbf{f}}_{l,i}^{\text{fuse}} + \mathbf{b}_l^Q)\Big)\,. \end{aligned} \tag{4.9}$$

Here A_l, B_l, and Q_l are learned weights with the corresponding biases; LN denotes LayerNorm, σ denotes sigmoid, and $\odot$ denotes element wise multiplication. The same DINO descriptors are used at all four stages, with separate adapters and fusion weights at each stage. Figure 4.5 shows the architecture of teacher model.

Point labels supervise both the student and the fusion teacher. The student uses all valid annotations in $\mathcal{A}$, while the fusion teacher uses the valid paired points in $\mathcal{I}_{\text{pair}}$. To account for class imbalance, all classification heads use weighted cross-entropy. For a valid point set $\mathcal{B}$, it is

$$\mathrm{CE}(\mathbf{Z}, \mathbf{y}; \mathcal{B}) = -\frac{\sum_{i \in \mathcal{B}} w_{y_i} \log p_{i,y_i}}{\sum_{i \in \mathcal{B}} w_{y_i}}, \qquad w_c = \left(\frac{n_{\max}}{n_c}\right)^{1/3}, \tag{4.10}$$

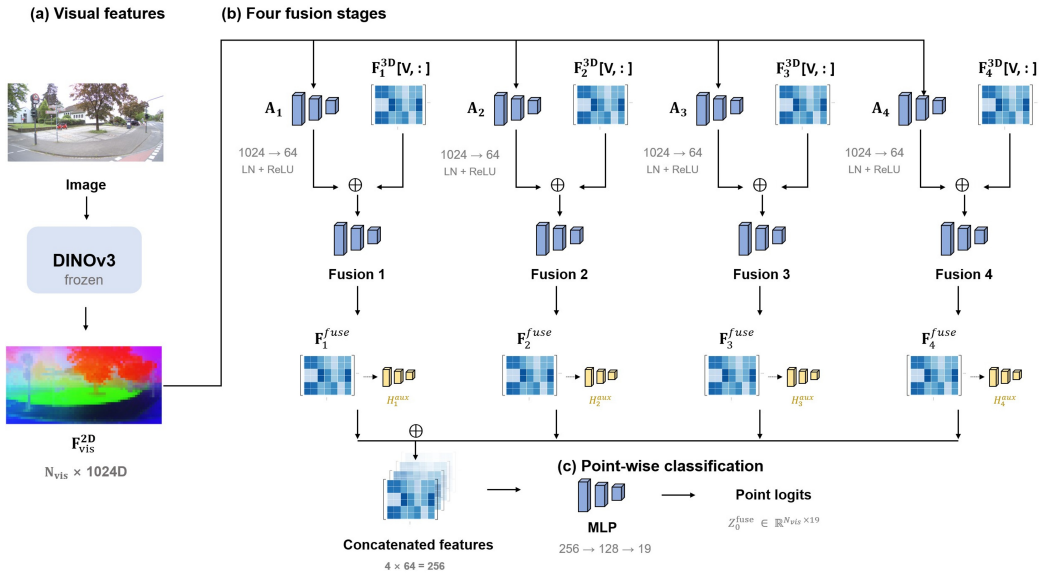


Figure 4.5: 2D-3D fusion teacher model architecture. The fusion teacher combines visual features from a frozen DINOv3 encoder with 3D geometric features from the LiDAR student. Its inputs are 1024D DINO descriptors for image paired points and four 64D student point features. At each stage, a trainable adapter reduces the DINO descriptor to 64D. This descriptor is concatenated with the corresponding student feature, then processed by a linear layer, a sigmoid, and ReLU to produce a 64D fused feature. The four fused features are concatenated and passed to the main classifier, which predicts 19 class logits per image paired point. Each stage also has an auxiliary classifier. The teacher and student use the same classifier architectures with separate weights. During training, the teacher's predictions guide the student through knowledge distillation.

where $\mathbf{p}_i = \mathrm{softmax}(\mathbf{z}_i)$, n_c is the fixed SemanticKITTI class count, and $n_{\max} = \max_c n_c$. The student main head also uses the Lovász–Softmax loss L_{Lov} [3], evaluated over the classes present in the valid annotations. The supervised losses are

$$\begin{aligned} L_{3D} &= \mathrm{CE}(\mathbf{Z}_0^{3D}, \mathbf{y}; \mathcal{A}) + \lambda_{\mathrm{Lov}} L_{\mathrm{Lov}} + \frac{\lambda_{\mathrm{aux}}}{4} \sum_{l=1}^{4} \mathrm{CE}(\mathbf{Z}_l^{3D}, \mathbf{y}; \mathcal{A}), \\ L_{\mathrm{fuse}} &= \mathrm{CE}(\mathbf{Z}_0^{\mathrm{fuse}}, \mathbf{y}; \mathcal{I}_{\mathrm{pair}}) + \frac{1}{4} \sum_{l=1}^{4} \mathrm{CE}(\mathbf{Z}_l^{\mathrm{fuse}}, \mathbf{y}; \mathcal{I}_{\mathrm{pair}}), \end{aligned} \tag{4.11}$$

with $\lambda_{\mathrm{Lov}} = 0.10$ and $\lambda_{\mathrm{aux}} = 0.25$. The DINO descriptors remain frozen, while the fusion teacher and student are trained jointly. The teacher's label loss also updates the student encoder through the shared LiDAR features.

The fusion predictions guide the student through KL divergence, following the fusion-to-single direction of 2DPASS [49]. We match the main predictions and the four corresponding auxiliary predictions on the same valid paired points.

For head $h = 0, \ldots, 4$, the class distributions at temperature $\tau = 2$ are

$$\begin{aligned} \mathbf{p}_{h,i}^{3D}(\tau) &= \mathrm{softmax}(\mathbf{z}_{h,i}^{3D}/\tau), \\ \bar{\mathbf{p}}_{h,i}^{\mathrm{fuse}}(\tau) &= \mathrm{sg}\big[\mathrm{softmax}(\mathbf{z}_{h,i}^{\mathrm{fuse}}/\tau)\big] . \end{aligned} \tag{4.12}$$

The stop gradient operator sg keeps the teacher distribution fixed when computing this loss. Averaging over the five heads and the M valid paired points gives, for $M > 0$,

$$L_{\mathrm{KD}} = \frac{\tau^2}{5M} \sum_{h=0}^{4} \sum_{i \in \mathcal{I}_{\mathrm{pair}}} D_{\mathrm{KL}}\big(\bar{\mathbf{p}}_{h,i}^{\mathrm{fuse}}(\tau) \,\|\, \mathbf{p}_{h,i}^{3D}(\tau)\big) , \tag{4.13}$$

where $D_{\mathrm{KL}}(\mathbf{p}\|\mathbf{q}) = \sum_{c=0}^{18} p_c \log(p_c/q_c)$. The selected points have equal distillation weight; the frequency of class weights apply to cross-entropy.

The complete training objective combines label supervision and distillation:

$$\begin{aligned} L_{\mathrm{train}} &= L_{3D} + \lambda_{\mathrm{fuse}} L_{\mathrm{fuse}} + \lambda_{\mathrm{KD}}(n_{\mathrm{ep}}) L_{\mathrm{KD}}, \\ \lambda_{\mathrm{KD}}(n_{\mathrm{ep}}) &= 0.10 \,\mathrm{clip}\left(\frac{n_{\mathrm{ep}} - 5}{5}, 0, 1\right), \qquad \lambda_{\mathrm{fuse}} = 1. \end{aligned} \tag{4.14}$$

Here n_{ep} is the epoch number. Distillation starts after 5 epochs and reaches its full weight at epoch ten. DINO feature information thus enters the student through supervised fusion and prediction distillation. At deployment, the LiDAR student produces predictions from LiDAR-only inputs.

4.4 Semantic Neural point map

We connect the LiDAR student to the neural point representation of PIN-SLAM [30]. Each map point stores a world position $\mathbf{n}_j \in \mathbb{R}^3$, an orientation $\mathbf{R}_j \in SO(3)$, and a trainable feature $\mathbf{f}_j^{\mathrm{map}} \in \mathbb{R}^8$. Two separate decoders use these shared features to predict geometry and semantics. The geometric decoder represents a signed distance field (SDF), while the semantic decoder represents scene categories.

Both decoders have one 64 unit ReLU hidden layer. Let g_{θ_g} return a scalar geometry output and $\mathbf{s}_{\theta_s}$ return semantic logits, with respective parameters θ_g and θ_s. The semantic head has 20 channels: reserved index zero and the 19 scene categories. The implemented decode before interpolation rule is

$$s_j(\mathbf{x}) = \beta_s g_{\theta_g}(\mathbf{h}_j(\mathbf{x})), \qquad s_{\mathcal{M}}(\mathbf{x}) = \sum_j \alpha_j(\mathbf{x}) s_j(\mathbf{x}), \tag{4.15}$$

$$\ell_{j,c}(\mathbf{x}) = \log \mathrm{softmax}\big(\mathbf{s}_{\theta_s}(\mathbf{h}_j(\mathbf{x}))\big)_c, \qquad \ell_c(\mathbf{x}) = \sum_j \alpha_j(\mathbf{x}) \ell_{j,c}(\mathbf{x}). \tag{4.16}$$

$s_{\mathcal{M}}$ is the interpolated SDF in metres, and ℓ_c is the aggregated log score of class c. The map label is $\hat{y}_{\mathcal{M}}(\mathbf{x}) = \arg\max_{c \in \{0,\ldots,19\}} \ell_c(\mathbf{x})$.

The frozen student runs on each raw scan while geometric preprocessing and tracking proceed. Original row indices preserve correspondence through point selection. At the Mapper boundary, the integration retrieves the result for the same frame and attaches the retained points' hard labels. A student index $c \in \{0, \ldots, 18\}$ becomes target $\hat{y}_i^{\text{map}} = c+1$; unusable predictions receive zero. These labels supervise the map after the current pose has been estimated.

Semantic supervision uses the aggregated log scores from Equation 4.16:

$$L_{\text{sem}} = -\frac{1}{|\mathcal{B}_s|} \sum_{q \in \mathcal{B}_s} \ell_{\hat{y}_q^{\text{map}}}(\mathbf{x}_q), \qquad L_{\text{map}} = L_{\text{sdf}} + 0.5 L_{\text{eik}} + 1.0 L_{\text{sem}}. \tag{4.17}$$

Every valid labelled batch sample contributes to the semantic term. Geometric terms update shared features and the geometry decoder; semantic NLL updates those features and the semantic decoder. Thus a semantic target constrains the same local feature vector that supports geometric reconstruction. Temporal information accumulates through repeated optimization of these persistent features and through replay of observations from multiple frames.

Chapter 5

Experiments

THIS work provides a pipeline to build a semantic segmentation module using DINO features for PIN-SLAM. We present our experiments to show the capabilities of our method. The results of our experiments also support our key claims, which are: (i) It is capable of effectively using vision foundation model for semantic segmentation. (ii) The ability of our approach to work effectively within a implict multi-sensor based SLAM pipline for building semantic mapping. (iii) Our semantic segmentation method provides a solid foundation for further improving odometry results.

5.1 Evaluation Criteria

We evaluate methods mainly using mean intersection over union (mIoU) as the main segmentation metric, which is defined as the average IoU over all classes. Additionally, we also use the overall accuracy (ACC) to evaluate whether the categories of prediction is same with the ground truth label.

We evaluate pose estimation using absolute trajectory error (ATE) and relative pose error (RPE). A rigid $SE(3)$ transformation $(\mathbf{R}_S, \mathbf{t}_S)$ aligns the estimated positions $\hat{\mathbf{t}}_t$ with the reference positions $\mathbf{t}_t$ by least squares over the evaluated sequence. The ATE is

$$\mathrm{ATE}_{\mathrm{RMSE}} = \sqrt{\frac{1}{T}\sum_{t=1}^{T} \|\mathbf{R}_S \hat{\mathbf{t}}_t + \mathbf{t}_S - \mathbf{t}_t\|_2^2}. \tag{5.1}$$

RPE measures the error in relative motion. For reference poses $\mathbf{T}_t$ and estimated poses $\hat{\mathbf{T}}_t$, the error over Δ frames is

$$\mathbf{E}_{t,\Delta} = (\mathbf{T}_t^{-1}\mathbf{T}_{t+\Delta})^{-1}(\hat{\mathbf{T}}_t^{-1}\hat{\mathbf{T}}_{t+\Delta}). \tag{5.2}$$

We use the RMSE of its translation magnitude in meters and rotation angle in degrees, using $\Delta = 1$. ATE describes accumulated position error, whereas RPE describes local motion error.

5.2 Experiments Setup

We evaluate our work on SemanticKITTI and IPB dataset. SemanticKITTI provides dense point-wise annotations for the KITTI odometry sequences [2, 13]. The scans were recorded by a vehicle-mounted Velodyne HDL-64E in urban and residential environments. We follow the standard single-scan split: sequences 00–07, 09, and 10 for training, and sequence 08 for validation. These contain 19,130 and 4,071 scans, respectively. The original labels are mapped to the 19 evaluation classes. We use the paired front camera images during training and evaluate the LiDAR-only student on complete scans.

The IPB sequence used in this work contains outdoor road scenes with buildings, parked vehicles, sidewalks, trees, cyclists, and traffic signs. It provides LiDAR scans and four calibrated RGB views: front, left, rear, and right. The images have a resolution of 2064×1024. The point files contain XYZ coordinates and a time sequence, without reflectance. For our model, the intensity value and its availability flag are therefore set to 0. We use the first 500 frames of the 1,001 frame sequence for semantic segmentation task. Frames 0–399 are used for training and frames 400–499 for validation.

Table 5.1: Datasets and splits used for semantic segmentation task.

	SemanticKITTI	IPB
Training split	00–07, 09, 10	Frames 0–399
Validation split	08	Frames 400–499
Scans (train / val.)	19,130 / 4,071	400 / 100
Evaluation classes	19	19
Paired image views	Front camera	Front, left, rear, right
Reflectance	Available	Unavailable

We consider three segmentation methods with different uses of image information. 2DPASS [49] transfers 2D priors through multi-scale knowledge distillation and uses LiDAR alone at inference. DINO in the Room (DITR) [19] injects frozen DINOv2 features into a 3D segmentation backbone. LSK3DNet [12] performs LiDAR segmentation with large sparse kernels.

For the segmentation ablations, baseline denotes our DINOv3-based teacher–student model with unweighted cross-entropy. It already includes the fusion teacher and knowledge distillation. We evaluate class weighting, augmentation for rare classes, and a DINOv2 replacement of the visual encoder. For online mapping, the reference configuration is PIN-SLAM with geometric map learning. The semantic configuration additionally uses the student's predicted labels to supervise the semantic decoder and shared neural point features.

The student takes one LiDAR scan and returns a 19 class prediction for each retained point. Its input contains XYZ coordinates, intensity, and an intensity-availability flag. The entry encoding adds offsets from the voxel mean and voxel center. Four point–voxel stages then produce point-aligned features, which are concatenated for classification. Table 5.2 lists the dimensions.

The fusion teacher combines these student features with frozen DINOv3 ViT-L/16 features [36]. Four separate visual adapters reduce the descriptors before fusion at the corresponding student stages. The student, adapters, fusion layers, and classification heads are trained jointly. DINO remains frozen. The teacher operates on image-paired points; the student predicts labels for the retained LiDAR points. The DINOv2 comparison uses ViT-L/14 [29] with the same feature dimension and fusion architecture.

Table 5.2: Network dimensions. N_{ret} and N_{vis} are the retained and image-paired point counts.

Component	Configuration
Student input / entry encoding	$N_{\text{ret}} \times 5$ / 11 values per point
Entry MLP	$11 \to 64 \to 64$
Initial voxel size	0.1 m
Point–voxel stages	Four stages, each with 64 channels
Point readout grids	0.2, 0.4, 0.8, 1.6 m
Main classifier	$256 \to 128 \to 19$
Auxiliary classifiers	Four heads, each $64 \to 128 \to 19$
Frozen visual descriptors	$N_{\text{vis}} \times 1024$
Visual adapters / fused features	$1024 \to 64$ / 64 channels per stage

SemanticKITTI and IPB training uses 50 epochs with randomly initialized student and fusion modules. We use AdamW with learning rate 10^{-3}, weight decay 10^{-4}, and gradient accumulation over eight scans. The learning rate warms up for five epochs, followed by cosine decay. Table 5.3 gives the shared optimization settings.

The objective combines student cross-entropy, Lovasz–Softmax [3], auxiliary classification, fusion-teacher classification, and knowledge distillation. The student Lovasz and auxiliary coefficients are 0.10 and 0.25, and the teacher classification coefficient is 1.0. Distillation matches the main and four auxiliary predictions at temperature 2. Its weight is zero for the first five epochs and increases to 0.10 at epoch ten. Class weighting is applied to the cross-entropy terms.

As for the training augmentation includes random yaw, horizontal flips, scaling in $[0.95, 1.05]$, and a shared Gaussian translation with standard deviation

0.1 m. The point dropout rate is sampled between 0 and 0.2, and frame-level intensity dropout has probability 0.3.

Table 5.3: Training and evaluation settings for our segmentation model.

Parameter	Setting
Training epoch	50 epochs
Optimizer	AdamW
Batch size	1
Learning rate	10^{-3}

At inference, we use the LiDAR backbone and main classifier, restoring predictions to the original scan order. Images, the fusion teacher, and auxiliary classifiers are used during training. Our training and student inference use an NVIDIA GeForce RTX 4060 Laptop GPU with PyTorch, CUDA 12.6.

As for the online mapping setup in PIN-SLAM, the pretraining student model supplies prediction semantic labels to PIN-SLAM. Point indices are carried through scan pre-processing so that the labels remain aligned with the map input. KITTI experiments use the SemanticKITTI pretraining student model; IPB experiments use the pretraining student model. The map uses a 0.4 m neural point voxel size, 8D features, and up to 6 neighbours per query. Tracking points are downsampled with a 0.6 m voxel size.

Map learning combines the SDF objective, Eikonal regularization with weight 0.5, and semantic supervision with weight 1.0. In the semantic configuration, the semantic decoder and SDF decoder use the shared neural point features. Pose estimation follows PIN-SLAM's point-to-SDF registration [30]. The geometry-only and semantic configurations share the geometric settings.

5.3 Semantic Segmentation Results

SemanticKITTI evaluates segmentation performance using single scan setting. In this method, a single scan as input, moving and non-moving are mapped to a single class. The report results on test sets are from the official blind test competition website of SemanticKITTI. Here, Baseline denotes the DINOv3-assisted model without class weighting or the additional targeted augmentation. The notation $+W$ denotes class-weighted cross-entropy, and $+A$ denotes additional data augmentation targeting weakly performing classes. The baseline already includes standard data augmentation.

Tab. 5.4 shows our performance under the single scan setting on validation set and test sets. It compares the effects of class weighting and targeted data augmentation. Class weighting is motivated by the imbalance in semantic class fre-

quencies. Assigning larger cross-entropy weights to less frequent classes increases their contribution to the supervised training objective. On validation sequence 08, class weighting increases mIoU from 61.83% to 64.13%. The largest improvements occur for bicycle (+11.82), other-vehicle (+9.33), and truck (+8.86). These gains are consistent with the intended rebalancing of class contributions during training. The higher mIoU is accompanied by a small decrease in overall accuracy, from 91.95% to 91.87%.

We next examine whether additional augmentation can improve classes that remain difficult to segment under the weighted training configuration. Adding targeted augmentation further increases mIoU to 64.98%, yielding an additional gain of 0.85 percentage points. Bicycle and other-ground benefit particularly, with IoU increasing from 39.53% to 46.07% and from 2.50% to 6.02%, respectively.

On test sequences 11–21, Baseline + W achieves 63.5% mIoU, compared with 62.7% for Baseline + W + A. The largest deterioration occurs for motorcyclist, whose IoU drops from 37.2% to 17.5%, outweighing the net gains across the remaining classes. Thus, the additional augmentation improves validation performance, but this benefit does not extend to the test set.

Table 5.4: Semantic segmentation results on the SemanticKITTI validation and test sets. All values are percentages.

Method	**mIoU**	**Acc.**	car	bicycle	motorcycle	truck	other-vehicle	person	bicyclist	motorcyclist	road	parking	sidewalk	other-ground	building	fence	vegetation	trunk	terrain	pole	traffic-sign
Validation set: sequence 08																					
Baseline	61.83	91.95	96.51	27.71	57.77	75.14	57.37	65.96	79.99	0.00	92.69	43.83	79.33	0.93	**90.58**	**61.07**	88.84	68.25	**76.81**	**63.34**	48.57
Baseline + W	64.13	91.87	**96.79**	39.53	61.71	**84.00**	**66.70**	69.69	86.16	0.04	93.15	**46.63**	79.50	2.50	89.34	56.72	**88.91**	68.66	76.76	60.72	51.07
Baseline + W + A	**64.98**	91.51	96.70	**46.07**	**67.31**	81.01	65.74	**71.42**	**86.17**	**1.45**	**93.33**	45.84	**79.59**	**6.02**	90.02	58.73	87.71	**70.02**	73.77	62.27	**51.40**
Test set: sequences 11–21																					
Baseline + W	**63.5**	**90.3**	96.1	49.7	49.9	49.2	51.9	61.5	65.8	**37.2**	**89.4**	**58.7**	**72.0**	25.7	**91.5**	**66.3**	84.5	**69.0**	69.0	57.6	**61.2**
Baseline + W + A	62.7	**90.3**	**96.3**	**50.1**	**51.5**	**50.7**	**53.9**	**62.8**	**66.8**	17.5	89.0	56.5	71.4	**26.0**	**91.5**	66.2	**85.1**	68.4	**69.4**	**57.8**	59.7

W: class weighting; A: weak-class data augmentation. Bold values indicate the best result within each split. For the test set, Acc. is the reported mean accuracy.

We also compare the numerical experiments of our work with the existing method in Tab.5.4. It demonstrates the class wise IoU, Frames Per Second (FPS), and mean IoU for different approaches. It compares our model with representative semantic segmentation methods on the SemanticKITTI test set. Our model (Baseline + W) achieves 63.5% mIoU, outperforming KPConv, FusionNet, and KPRNet by 4.7, 2.2, and 0.4 percentage points, respectively, and remaining within 0.3 percentage points of Lite-HDSeg. For the truck class, it achieves 49.2% IoU, compared with 33.4% for KPConv and 41.8% for FusionNet. Although 2DPASS and LSK3DNet achieve higher overall mIoU, our student operates at 16.5 FPS in the evaluated setup and requires only LiDAR input at inference. Together, these results support its use as a practical semantic front end for the proposed mapping

system, with visual features used during training and no camera input required during deployment. Figure5.5 shows our prediction results on SemanticKITTI sequence 11-21.



Figure 5.1: Our semantic segmentation prediction results on SemanticKITTI sequences 11–21.

We investigate the effect of visual inputs to the fusion teacher by comparing RGB images, DINOv2 features, and DINOv3 features with class weighting en-

Table 5.5: Comparison on the SemanticKITTI single scan test set (sequences 11–21). IoU values are percentages.

Method	Input	**mIoU**	car	bicycle	motorcycle	truck	other-vehicle	person	bicyclist	motorcyclist	road	parking	sidewalk	other-ground	building	fence	vegetation	trunk	terrain	pole	traffic-sign	FPS (Hz)
LatticeNet [35]	L	52.9	92.9	16.6	22.2	26.6	21.4	35.6	43.0	46.0	90.0	59.4	74.1	22.0	88.2	58.8	81.7	63.6	63.1	51.9	48.4	7
KPConv [41]	L	58.8	96.0	30.2	42.5	33.4	44.3	61.5	61.6	11.8	88.8	61.3	72.7	31.6	90.5	64.2	84.8	69.2	69.1	56.4	47.4	–
FusionNet [50]	L	61.3	95.3	47.5	37.7	41.8	34.5	59.5	56.8	11.9	91.8	68.8	77.1	30.8	92.5	69.4	84.5	69.8	68.5	60.4	66.5	–
Cylinder3D [57]	L	61.8	96.1	54.2	47.6	38.6	45.0	65.1	63.5	13.6	91.2	62.2	75.2	18.7	89.6	61.6	85.4	69.7	69.3	62.6	64.7	–
KPRNet [20]	L	63.1	95.5	54.1	47.9	23.6	42.6	65.9	65.0	16.5	**93.2**	**73.9**	**80.6**	30.2	91.7	68.4	85.7	69.8	71.2	58.7	64.1	0.3
DeepTemporalSeg [10]	L	37.6	81.5	29.4	19.6	6.6	6.5	23.7	20.1	2.4	85.8	8.7	59.3	1.0	78.6	39.6	77.1	46.0	58.1	32.6	39.1	–
SqueezeSegV2 [45]	L	39.7	81.8	18.5	17.9	13.4	14.0	20.1	25.1	3.9	88.6	45.8	67.6	17.7	73.7	41.1	71.8	35.8	60.2	20.2	36.3	50
RangeNet53++KNN [26]	L	52.2	91.4	25.7	34.4	25.7	23.0	38.3	38.8	4.8	91.8	65.0	75.2	27.8	87.4	58.6	80.5	55.1	64.6	47.9	55.9	12
PolarNet [52]	L	54.3	93.8	40.3	30.1	22.9	28.5	43.2	40.2	5.6	90.8	61.7	74.4	21.7	90.0	61.3	84.0	65.5	67.8	51.8	57.5	16
3D-MiniNet-KNN [1]	L	55.8	90.5	42.3	42.1	28.5	29.4	47.8	44.1	14.5	91.6	64.2	74.5	25.4	89.4	60.8	82.8	60.8	66.7	48.0	56.6	28
SqueezeSegV3-53+kNN [48]	L	55.9	92.5	38.7	36.5	29.6	33.0	45.6	46.2	20.1	91.7	63.4	74.8	26.4	89.0	59.4	82.0	58.7	65.4	49.6	58.9	6
SalsaNext [8]	L	59.5	91.9	48.3	38.6	38.9	31.9	60.2	59.0	19.4	91.7	63.7	75.8	29.1	90.2	64.2	81.8	63.6	66.5	54.3	62.1	24
Lite-HDSeg [34]	L	63.8	92.3	40.0	55.4	37.7	39.6	59.2	71.6	54.1	93.0	68.2	78.3	29.3	91.5	65.0	78.2	65.8	65.1	59.5	67.7	20
2DPASS [49]	L	72.9	97.0	63.6	63.4	**61.1**	61.5	77.9	**81.3**	74.1	89.7	67.4	74.7	40.0	**93.5**	**72.9**	86.2	73.9	71.0	65.0	70.4	16.1
DITR [19]	L+C	74.4	**97.3**	60.3	68.8	58.8	**66.3**	79.0	77.3	**87.3**	92.3	72.8	79.3	37.7	92.9	71.3	85.6	**76.0**	71.1	**67.8**	71.4	–
LSK3DNet [12]	L	**75.6**	**97.3**	**71.4**	**75.6**	61.0	64.2	**81.2**	80.6	85.2	92.2	70.2	78.9	**41.8**	92.7	72.0	**86.4**	72.7	**71.9**	67.0	**74.6**	11.2
Baseline + W (ours)	L	63.5	96.1	49.7	49.9	49.2	51.9	61.5	65.8	37.2	89.4	58.7	72.0	25.7	91.5	66.3	84.5	69.0	69.0	57.6	61.2	16.5†

L: LiDAR; C: RGB images at inference; W: class weighting. Bold values mark the highest IoU in each column. Speeds retain their respective timing settings; – indicates an unreported value.

abled. As shown in Table 5.6, DINOv3 achieves the highest mIoU on both splits. On the validation set, it reaches 64.13%, exceeding DINOv2 and RGB inputs by 0.15 and 0.81 percentage points, respectively. On the test set, it achieves 63.5%, outperforming both alternatives by 1.3 percentage points. These results support the choice of DINOv3 as the visual feature extractor during training.

Table 5.6: Semantic segmentation results with different visual inputs on the SemanticKITTI validation and test sets. All values are percentages.

Method	mIoU	Acc.	car	bicycle	motorcycle	truck	other-vehicle	person	bicyclist	motorcyclist	road	parking	sidewalk	other-ground	building	fence	vegetation	trunk	terrain	pole	traffic-sign
Validation set: sequence 08																					
DINOv2 + Baseline + W	63.98	91.91	96.05	40.17	60.56	86.58	58.03	69.38	84.61	0.01	93.25	47.76	79.35	1.68	90.41	60.11	88.73	69.39	76.63	61.29	51.62
RGB images + Baseline + W	63.32	91.55	96.15	38.69	61.28	79.61	58.08	68.53	83.66	0.21	92.89	47.56	79.00	1.84	90.65	60.88	87.88	68.62	74.57	61.57	51.47
DINOv3 + Baseline + W	64.13	91.87	96.79	39.53	61.71	84.00	66.70	69.69	86.16	0.04	93.15	46.63	79.50	2.50	89.34	56.72	88.91	68.66	76.76	60.72	51.07
Test set: sequences 11–21																					
DINOv2 + Baseline + W	62.2	90.3	96.2	45.9	49.0	46.6	51.1	62.3	66.8	17.3	89.4	58.3	72.0	25.8	91.8	67.2	84.3	69.1	68.5	56.7	64.3
RGB images + Baseline + W	62.2	90.3	96.2	45.9	49.0	46.6	51.1	62.3	66.8	17.3	89.4	58.3	72.0	25.8	91.8	67.2	84.3	69.1	68.5	56.7	64.3
DINOv3 + Baseline + W	63.5	90.3	96.1	49.7	49.9	49.2	51.9	61.5	65.8	37.2	89.4	58.7	72.0	25.7	91.5	66.3	84.5	69.0	69.0	57.6	61.2

W denotes class weighting. DINOv2, DINOv3, and RGB images identify the visual branch used during training. Acc. is the reported accuracy for each split.

On the manually annotated IPB validation split, the locally trained model achieves 56.87% mIoU and 93.79% overall accuracy.

5.4 Semantic Mapping and Trajectory Estimation

We compare the geometry only configuration with the configuration using semantic supervision on SemanticKITTI sequences 08 and 09 and the first 500 frames of IPB dataset. Trajectory accuracy is evaluated using translational RMSE in `evo`. APE is computed after full trajectory SE(3) alignment with fixed scale, while RPE is computed between consecutive frames without global alignment.

As shown in Table 5.8, adding semantic supervision reduces APE from 4.11 to 4.04 m on KITTI 08, from 0.89 to 0.81 m on KITTI 09, and from 0.41 to 0.31 m on IPB. These correspond to reductions of 1.88%, 9.31%, and 24.46%, respectively, with the largest improvement observed on IPB.

Semantic supervision updates the map features shared with the SDF decoder and can therefore influence subsequent geometric registration, while pose estimation remains geometry based. The results show improved overall agreement with the reference trajectory, accompanied by higher consecutive frame translational errors.

Table 5.7: IPB dataset semantic segmentation results comparing with different approach.

Method	mIoU ↑	Acc. ↑	car	bicycle	motorcycle	truck	other-vehicle	person	bicyclist	motorcyclist	road	parking	sidewalk	other-ground	building	fence	vegetation	trunk	terrain	pole	traffic-sign
LSK3DNet	9.63	35.13	14.08	1.73	0.74	0.00	0.00	2.94	0.00	0.00	48.76	0.04	21.75	0.00	25.31	23.13	10.90	0.00	8.94	17.91	6.82
DITR	35.87	83.20	70.68	16.00	43.04	0.00	0.01	53.55	0.54	0.00	77.72	0.77	70.77	0.00	90.58	27.07	79.55	0.00	43.66	46.49	61.01
Ours	**56.87**	**93.79**	78.76	56.69	65.29	0.00	34.21	77.03	22.18	0.00	84.85	8.30	82.09	0.00	95.25	59.26	95.51	79.84	82.15	71.95	87.09

Table 5.8: Trajectory accuracy evaluated with evo (translational RMSE). Lower values are better. APE uses full-trajectory SE(3) alignment with fixed scale. RPE uses all adjacent frame pairs without global alignment.

Dataset	Frames	Method	APE (m) ↓	RPE (m) ↓
KITTI 08	4071	geometry	4.11	0.02
KITTI 08	4071	geometry+semantic	4.04	0.02
KITTI 09	1591	geometry	0.89	0.03
KITTI 09	1591	geometry+semantic	0.81	0.03
IPB	500	geometry	0.41	0.01
IPB	500	geometry+semantic	0.31	0.02

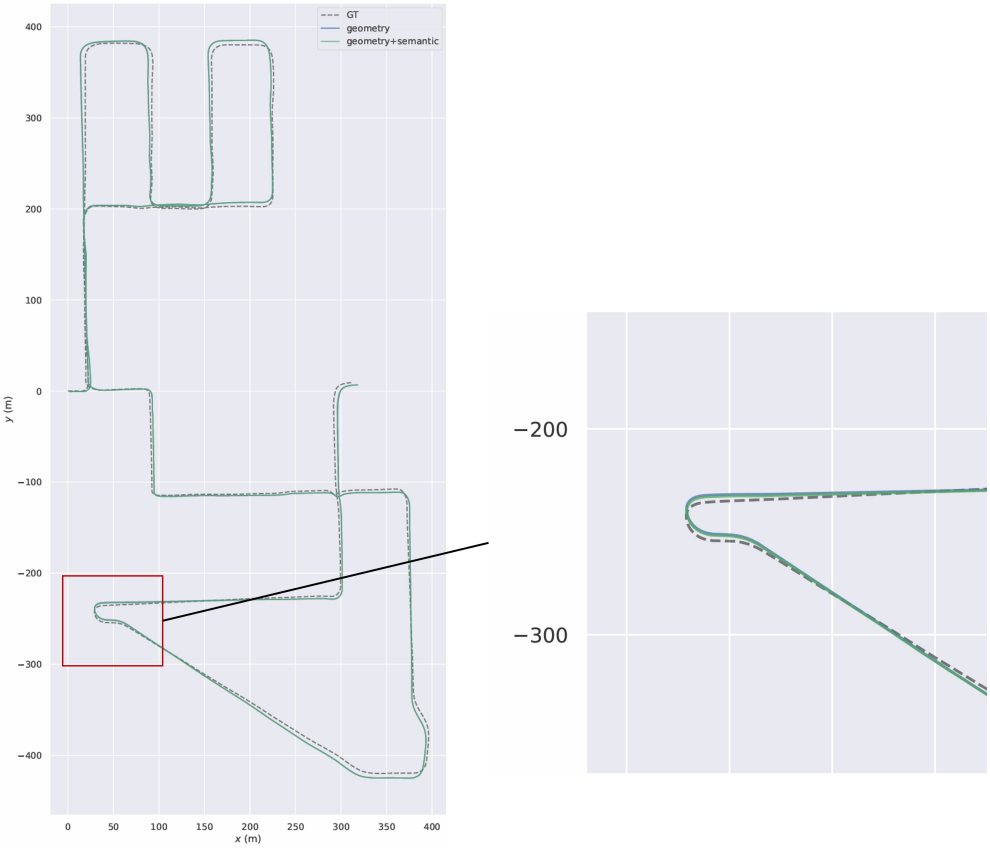


Figure 5.2: SemanticKITTI sequence 08 trajectory.

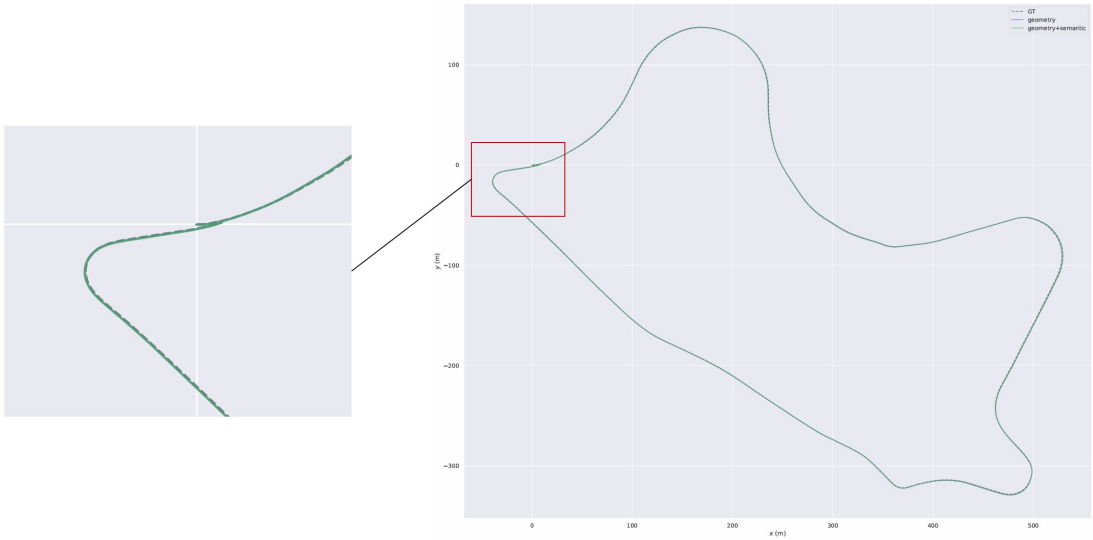


Figure 5.3: SemanticKITTI sequence 09 trajectory.

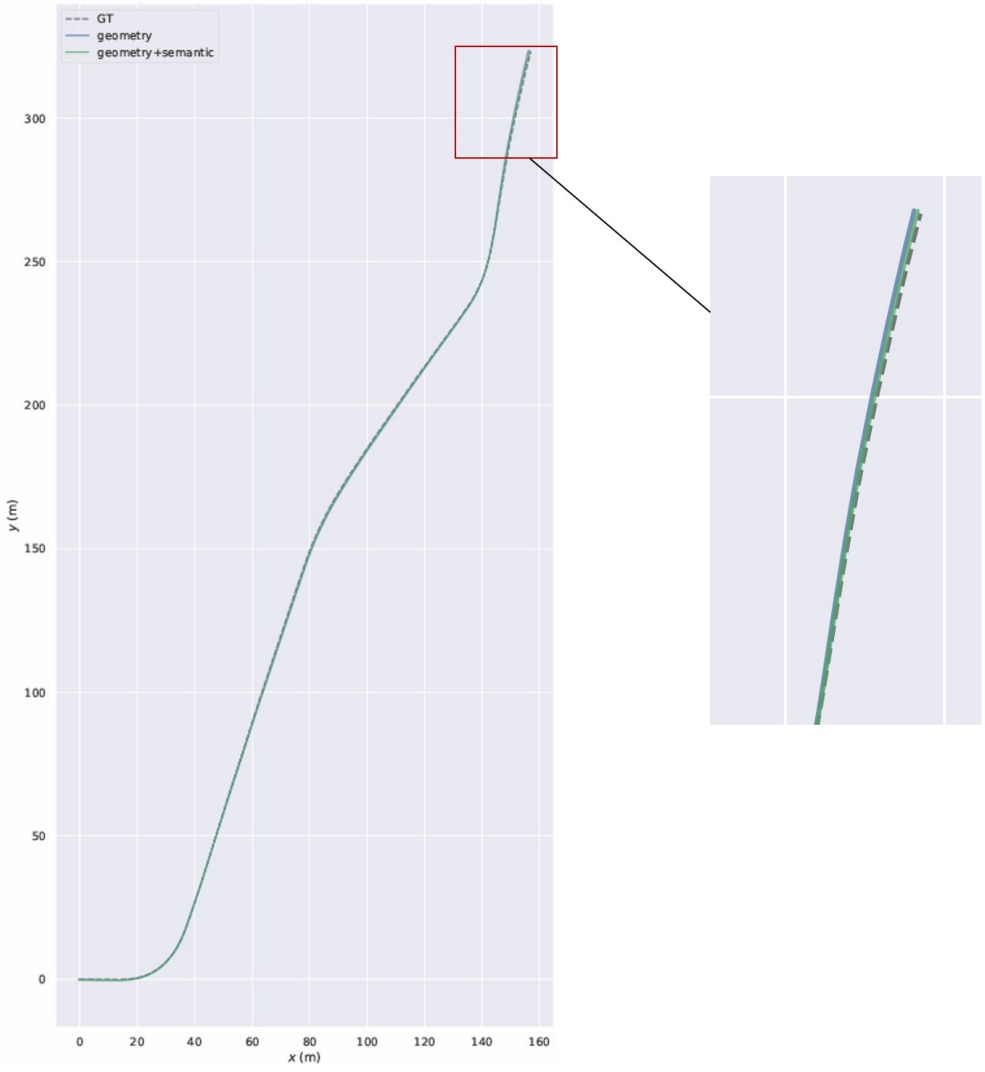


Figure 5.4: IPB trajectory.



Figure 5.5: IPB dataset semantic mapping result.

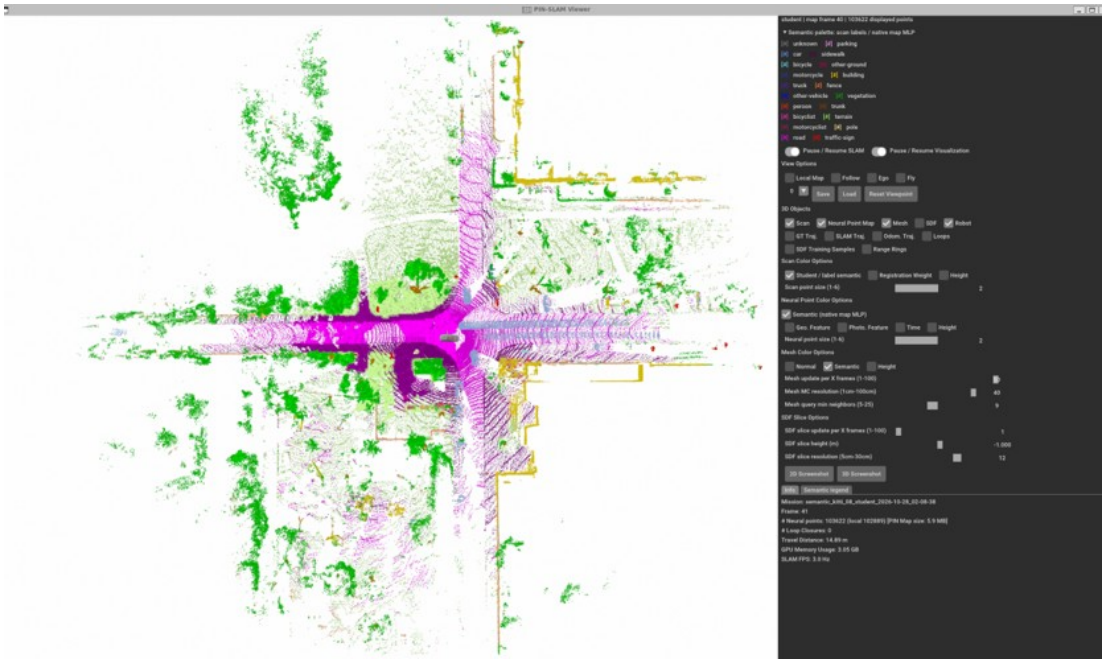


Figure 5.6: SemanticKITTI sequence 08 using pretrained semantic segmentation model building semantic mapping in PIN-SLAM.

Figure 5.6 shows the real-time visualization of semantic mapping in PIN-SLAM using our method. As new LiDAR scans are processed, our method can efficiently realize the map updating with semantic information, with different colors representing classes such as roads, vegetation, and buildings.

Chapter 6

Conclusion

This thesis addresses the integration of semantic scene understanding into an efficient implicit SLAM pipeline. The objective is to exploit vision foundation model features during training while enabling semantic segmentation from LiDAR scans alone at inference time, and to incorporate the resulting predictions into an incrementally updated map. The proposed approach takes advantage of DINOv3 feature during training of a semantic segmentation model. Once trained, the model predicts semantic labels directly from LiDAR input, without requiring images for segmentation. The segmentation pipeline is integrated into PIN-SLAM, where successive scans update the same map, allowing scene geometry and semantic categories to be queried and visualized together.

Our method on SemanticKITTI achieves a mean intersection over union (mIoU) of 64.13% on the validation set and 63.5% on the test set, with semantic segmentation inference running at 16.5 frames per second. On the IPB dataset validation split, the model trained on manually annotated local data achieves 56.87% mIoU. Compared with geometry-only mapping, the semantic configuration achieves lower aligned absolute pose error. The integrated system demonstrates live semantic map updates in PIN-SLAM using LiDAR input alone.

6.1 Summary of Key Contributions

In conclusion, the main contributions of this thesis are summarized as follows:

- It is capable of effectively using vision foundation model for semantic segmentation. We extracted dense image descriptors with a frozen DINOv3 encoder and associated them with LiDAR points using camera calibration. A multi-scale fusion teacher combines visual and LiDAR features and transfers semantic knowledge to the student through knowledge distillation. The trained student predicts 19 semantic classes from LiDAR input alone.

- We also showcase the ability of our approach to work effectively within a SLAM pipline for building semantic mapping. We integrated the pretrained LiDAR segmentation model into PIN-SLAM while maintaining the correspondence between retained points and their semantic predictions. These predictions supervise a semantic field through map features shared with the SDF decoder, while pose estimation continues to use PIN-SLAM's geometric registration.

- We used PIN-SLAM's mesh reconstruction tools to extract surfaces from the SDF and query semantic labels at the mesh vertices. Assigning colors according to these labels produces a surface representation in which different semantic classes can be visually distinguished.

- We manually annotated point-wise semantic labels for the IPB data and used separate training and validation frames. We organized the calibrated images, LiDAR scans, and point-wise reference labels for IPB and SemanticKITTI dataset. Consistent point correspondences and label conversion support student training and quantitative segmentation evaluation.

Bibliography

[1] Iñigo Alonso, Luis Riazuelo, Luis Montesano, and Ana C. Murillo. 3D-MiniNet: Learning a 2D representation from point clouds for fast and efficient 3D LIDAR semantic segmentation. *IEEE Robotics and Automation Letters*, 5(4):5432–5439, 2020.

[2] Jens Behley, Martin Garbade, Andres Milioto, Jan Quenzel, Sven Behnke, Cyrill Stachniss, and Jürgen Gall. SemanticKITTI: A dataset for semantic scene understanding of LiDAR sequences. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 9297–9307, 2019.

[3] Maxim Berman, Amal Rannen Triki, and Matthew B. Blaschko. The Lovász-Softmax loss: A tractable surrogate for the optimization of the intersection-over-union measure in neural networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 4413–4421, 2018.

[4] Berta Bescós, José M. Fácil, Javier Civera, and José Neira. DynaSLAM: Tracking, mapping, and inpainting in dynamic scenes. *IEEE Robot. Autom. Lett.*, 3(4):4076–4083, 2018.

[5] Alexandre Boulch, Joris Guerry, Bertrand Le Saux, and Nicolas Audebert. SnapNet: 3D point cloud semantic labeling with 2D deep segmentation networks. *Computers & Graphics*, 71:189–198, 2018.

[6] Mathilde Caron, Hugo Touvron, Ishan Misra, Hervé Jégou, Julien Mairal, Piotr Bojanowski, and Armand Joulin. Emerging properties in self-supervised vision transformers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 9650–9660, October 2021.

[7] Xieyuanli Chen, Andres Milioto, Emanuele Palazzolo, Philippe Giguère, Jens Behley, and Cyrill Stachniss. SuMa++: Efficient LiDAR-based semantic SLAM. In *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, pages 4530–4537, 2019.

[8] Tiago Cortinhal, George Tzelepis, and Eren Erdal Aksoy. SalsaNext: Fast, uncertainty-aware semantic segmentation of LiDAR point clouds. In *Advances in Visual Computing*, volume 12510 of *Lecture Notes in Computer Science*, pages 207–222. Springer, 2020.

[9] A.J. Davison, I.D. Reid, N.D. Molton, and O. Stasse. MonoSLAM: Real-time single camera SLAM. *IEEE Trans. on Pattern Analalysis and Machine Intelligence (TPAMI)*, 29, 2007.

[10] Ayush Dewan and Wolfram Burgard. DeepTemporalSeg: Temporally consistent semantic segmentation of 3D LiDAR scans. In *2020 IEEE International Conference on Robotics and Automation (ICRA)*, pages 2624–2630. IEEE, 2020.

[11] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations*, 2021.

[12] Tuo Feng, Wenguan Wang, Fan Ma, and Yi Yang. LSK3DNet: Towards effective and efficient 3D perception with large sparse kernels. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 14916–14927, June 2024.

[13] Andreas Geiger, Philip Lenz, and Raquel Urtasun. Are we ready for autonomous driving? the KITTI vision benchmark suite. In *2012 IEEE Conference on Computer Vision and Pattern Recognition*, pages 3354–3361, 2012.

[14] Benjamin Graham, Martin Engelcke, and Laurens van der Maaten. 3D semantic segmentation with submanifold sparse convolutional networks. In *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, pages 9224–9232, 2018.

[15] G. Grisetti, C. Stachniss, and W. Burgard. Improved Techniques for Grid Mapping with Rao-Blackwellized Particle Filters. *IEEE Trans. on Robotics (TRO)*, 23(1):34–46, 2007.

[16] Michael Grupp. evo: Python package for the evaluation of odometry and SLAM. `https://github.com/MichaelGrupp/evo`, 2017.

[17] Alexander Kirillov, Eric Mintun, Nikhila Ravi, Hanzi Mao, Chloe Rolland, Laura Gustafson, Tete Xiao, Spencer Whitehead, Alexander C. Berg, Wan-Yen Lo, Piotr Dollar, and Ross Girshick. Segment anything. In *Proceedings*

of the IEEE/CVF International Conference on Computer Vision (ICCV), pages 4015–4026, October 2023.

[18] G. Klein and D. Murray. Parallel Tracking and Mapping for Small AR Workspaces. In *Proc. of the Int. Symposium on Mixed and Augmented Reality (ISMAR)*, 2007.

[19] Karim Knaebel, Kadir Yilmaz, Daan de Geus, Alexander Hermans, David Adrian, Timm Linder, and Bastian Leibe. DINO in the room: Leveraging 2D foundation models for 3D segmentation. In *Proc. Int. Conf. 3D Vis. (3DV)*, 2026.

[20] Deyvid Kochanov, Fatemeh Karimi Nejadasl, and Olaf Booij. KPRNet: Improving projection-based LiDAR semantic segmentation. *arXiv preprint arXiv:2007.12668*, 2020.

[21] Lingdong Kong, Youquan Liu, Runnan Chen, Yuexin Ma, Xinge Zhu, Yikang Li, Yuenan Hou, Yu Qiao, and Ziwei Liu. Rethinking range view representation for LiDAR segmentation. In *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, pages 228–240, 2023.

[22] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.

[23] Mingrui Li, Shuhong Liu, Heng Zhou, Guohao Zhu, Na Cheng, Tianchen Deng, and Hongyu Wang. SGS-SLAM: Semantic gaussian splatting for neural dense SLAM. In *Computer Vision – ECCV 2024*, volume 15089 of *Lecture Notes in Computer Science*, pages 163–179, Cham, Switzerland, 2024. Springer.

[24] J. McCormac, A. Handa, A.J. Davison, and S. Leutenegger. SemanticFusion: Dense 3D Semantic Mapping with Convolutional Neural Networks. In *Proc. of the IEEE Int. Conf. on Robotics and Automation (ICRA)*, 2017.

[25] John McCormac, Ronald Clark, Michael Bloesch, Andrew J. Davison, and Stefan Leutenegger. Fusion++: Volumetric object-level SLAM. In *Proc. Int. Conf. 3D Vis. (3DV)*, pages 32–41, 2018.

[26] Andres Milioto, Ignacio Vizzo, Jens Behley, and Cyrill Stachniss. RangeNet++: Fast and accurate LiDAR semantic segmentation. In *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, pages 4213–4220, 2019.

[27] Michael Montemerlo, Sebastian Thrun, Daphne Koller, and Ben Wegbreit. FastSLAM: A factored solution to the simultaneous localization and mapping

problem. In *Proc. 18th Nat. Conf. Artif. Intell. (AAAI)*, pages 593–598, Edmonton, AB, Canada, 2002.

[28] R. Mur-Artal, J.M.M. Montiel, and J. D Tardos. ORB-SLAM: a versatile and accurate monocular SLAM system. *IEEE Trans. on Robotics (TRO)*, 31(5):1147–1163, 2015.

[29] Maxime Oquab, Timothée Darcet, Théo Moutakanni, Huy V. Vo, Marc Szafraniec, Vasil Khalidov, Pierre Fernandez, Daniel Haziza, Francisco Massa, Alaaeldin El-Nouby, Mido Assran, Nicolas Ballas, Wojciech Galuba, Russell Howes, Po-Yao Huang, Shang-Wen Li, Ishan Misra, Michael Rabbat, Vasu Sharma, Gabriel Synnaeve, Hu Xu, Hervé Jégou, Julien Mairal, Patrick Labatut, Armand Joulin, and Piotr Bojanowski. DINOv2: Learning robust visual features without supervision. *Transactions on Machine Learning Research*, 2024.

[30] Yue Pan, Xingguang Zhong, Louis Wiesmann, Timo Posewsky, Jens Behley, and Cyrill Stachniss. PIN-SLAM: LiDAR SLAM using a point-based implicit neural representation for achieving global map consistency. *IEEE Trans. Robot.*, 40:4045–4064, 2024.

[31] C. R. Qi, H. Su, K. Mo, and L. J. Guibas. PointNet: Deep Learning on Point Sets for 3D Classification and Segmentation. In *Proc. of the IEEE Conf. on Computer Vision and Pattern Recognition (CVPR)*, 2017.

[32] C.R. Qi, K. Yi, H. Su, and L. J. Guibas. PointNet++: Deep Hierarchical Feature Learning on Point Sets in a Metric Space. In *Proc. of the Advances in Neural Information Processing Systems (NIPS)*, 2017.

[33] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. Learning transferable visual models from natural language supervision. In Marina Meila and Tong Zhang, editors, *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 8748–8763. PMLR, 2021.

[34] Ryan Razani, Ran Cheng, Ehsan Taghavi, and Bingbing Liu. Lite-HDSeg: LiDAR semantic segmentation using lite harmonic dense convolutions. In *2021 IEEE International Conference on Robotics and Automation (ICRA)*, pages 9550–9556. IEEE, 2021.

[35] Radu Alexandru Rosu, Peer Schütt, Jan Quenzel, and Sven Behnke. LatticeNet: Fast point cloud segmentation using permutohedral lattices. In *Proceedings of Robotics: Science and Systems*, July 2020.

[36] Oriane Siméoni, Huy V. Vo, Maximilian Seitzer, Federico Baldassarre, Maxime Oquab, Cijo Jose, Vasil Khalidov, Marc Szafraniec, Seungeun Yi, Michaël Ramamonjisoa, Francisco Massa, Daniel Haziza, Luca Wehrstedt, Jianyuan Wang, Timothée Darcet, Théo Moutakanni, Leonel Sentana, Claire Roberts, Andrea Vedaldi, Jamie Tolan, John Brandt, Camille Couprie, Julien Mairal, Hervé Jégou, Patrick Labatut, and Piotr Bojanowski. DINOv3. arXiv:2508.10104, August 2025.

[37] Randall C. Smith and Peter Cheeseman. On the representation and estimation of spatial uncertainty. *Int. J. Robot. Res.*, 5(4):56–68, December 1986.

[38] Jürgen Sturm, Nikolas Engelhard, Felix Endres, Wolfram Burgard, and Daniel Cremers. A benchmark for the evaluation of RGB-D SLAM systems. In *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 573–580, 2012.

[39] Haotian Tang, Zhijian Liu, Shengyu Zhao, Yujun Lin, Ji Lin, Hanrui Wang, and Song Han. Searching efficient 3D architectures with sparse point-voxel convolution. In *Computer Vision – ECCV 2020*, volume 12373 of *Lecture Notes in Computer Science*, pages 685–702, Cham, Switzerland, 2020. Springer.

[40] Lyne P. Tchapmi, Christopher B. Choy, Iro Armeni, JunYoung Gwak, and Silvio Savarese. SEGCloud: Semantic segmentation of 3D point clouds. In *Proc. Int. Conf. 3D Vis. (3DV)*, pages 537–547, 2017.

[41] Hugues Thomas, Charles R. Qi, Jean-Emmanuel Deschaud, Beatriz Marcotegui, François Goulette, and Leonidas J. Guibas. KPConv: Flexible and deformable convolution for point clouds. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 6411–6420, October 2019.

[42] Yue Wang, Yongbin Sun, Ziwei Liu, Sanjay E. Sarma, Michael M. Bronstein, and Justin M. Solomon. Dynamic graph CNN for learning on point clouds. *ACM Trans. Graph.*, 38(5):146:1–146:12, 2019.

[43] M. Weinmann, B. Jutzi, S. Hinz, and C. Mallet. Semantic point cloud interpretation based on optimal neighborhoods, relevant features and efficient

classifiers. *ISPRS Journal of Photogrammetry and Remote Sensing*, 105:286–304, 2015.

[44] L. Wiesmann, T. Läbe, L. Nunes, J. Behley, and C. Stachniss. Joint Intrinsic and Extrinsic Calibration of Perception Systems Utilizing a Calibration Environment. *IEEE Robotics and Automation Letters (RA-L)*, 9(10):9103–9110, 2024.

[45] Bichen Wu, Xuanyu Zhou, Sicheng Zhao, Xiangyu Yue, and Kurt Keutzer. SqueezeSegV2: Improved model structure and unsupervised domain adaptation for road-object segmentation from a LiDAR point cloud. In *2019 International Conference on Robotics and Automation (ICRA)*, pages 4376–4382. IEEE, 2019.

[46] Xiaoyang Wu, Daniel DeTone, Duncan Frost, Tianwei Shen, Chris Xie, Nan Yang, Jakob Engel, Richard Newcombe, Hengshuang Zhao, and Julian Straub. Sonata: Self-supervised learning of reliable point representations. In *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, pages 22193–22204, 2025.

[47] Xiaoyang Wu, Li Jiang, Peng-Shuai Wang, Zhijian Liu, Xihui Liu, Yu Qiao, Wanli Ouyang, Tong He, and Hengshuang Zhao. Point transformer V3: Simpler, faster, stronger. In *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, pages 4840–4851, 2024.

[48] Chenfeng Xu, Bichen Wu, Zining Wang, Wei Zhan, Peter Vajda, Kurt Keutzer, and Masayoshi Tomizuka. SqueezeSegV3: Spatially-adaptive convolution for efficient point-cloud segmentation. In *Computer Vision – ECCV 2020*, pages 1–19. Springer, 2020.

[49] Xu Yan, Jiantao Gao, Chaoda Zheng, Chao Zheng, Ruimao Zhang, Shuguang Cui, and Zhen Li. 2DPASS: 2D priors assisted semantic segmentation on LiDAR point clouds. In *Computer Vision – ECCV 2022*, pages 677–695. Springer, 2022.

[50] Feihu Zhang, Jin Fang, Benjamin Wah, and Philip Torr. Deep FusionNet for point cloud semantic segmentation. In *Computer Vision – ECCV 2020*, volume 12356 of *Lecture Notes in Computer Science*, pages 644–663. Springer International Publishing, 2020.

[51] J. Zhang and S. Singh. LOAM: Lidar Odometry and Mapping in Real-time. In *Proc. of Robotics: Science and Systems (RSS)*, 2014.

[52] Yang Zhang, Zixiang Zhou, Philip David, Xiangyu Yue, Zerong Xi, Boqing Gong, and Hassan Foroosh. PolarNet: An improved grid representation for online LiDAR point clouds semantic segmentation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 9601–9610, 2020.

[53] Hengshuang Zhao, Li Jiang, Jiaya Jia, Philip H. S. Torr, and Vladlen Koltun. Point transformer. In *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, pages 16259–16268, 2021.

[54] Na Zhao. *Towards Learning Scene Semantics on 3D Point Clouds*. Ph.d. dissertation, National University of Singapore, Singapore, 2020.

[55] Chunran Zheng, Wei Xu, Zuhao Zou, Tong Hua, Chongjian Yuan, Dongjiao He, Bingyang Zhou, Zheng Liu, Jiarong Lin, Fangcheng Zhu, Yunfan Ren, Rong Wang, Fanle Meng, and Fu Zhang. FAST-LIVO2: Fast, direct LiDAR–inertial–visual odometry. *IEEE Trans. Robot.*, 41:326–346, 2025.

[56] Shuaifeng Zhi, Tristan Laidlow, Stefan Leutenegger, and Andrew J. Davison. In-place scene labelling and understanding with implicit scene representation. In *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, pages 15838–15847, 2021.

[57] Hui Zhou, Xinge Zhu, Xiao Song, Yuexin Ma, Zhe Wang, Hongsheng Li, and Dahua Lin. Cylinder3D: An effective 3D framework for driving-scene LiDAR semantic segmentation. *arXiv preprint arXiv:2008.01550*, 2020.

[58] Siting Zhu, Guangming Wang, Hermann Blum, Jiuming Liu, Liang Song, Marc Pollefeys, and Hesheng Wang. SNI-SLAM: Semantic neural implicit SLAM. In *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, pages 21167–21177, 2024.

[59] Xinge Zhu, Hui Zhou, Tai Wang, Fangzhou Hong, Yuexin Ma, Wei Li, Hongsheng Li, and Dahua Lin. Cylindrical and asymmetrical 3D convolution networks for LiDAR segmentation. In *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, pages 9939–9948, 2021.

List of Figures

1.1 The aim of this thesis lies in pushing intelligent agents towards human level intelligence on interaction and navigation 2

2.1 The illustration of the native range image. 6
2.2 The TSDF value is derived from the signed distance function (SDF) value. As shown in the figure, the white and grap squares represent the individual voxels in the TSDF map. The bule triangles represent the camera's field of view. A green cross-section line in the center of the figure represents the cross-section of an object. The depth map from a depth camera can display depth information for the object's cross-section. We need to calculate the SDF and TSDF values for the voxels, namely SDF(x) and TSDF(x) . . 10

3.1 Representative vision foundation model architectures 14
3.2 Visualization of dense features of DINOv3 by mapping the first three components of a PCA computed over the feature space to RGB. 15
3.3 Schematic of patch feature extraction with frozen DINOv3 ViT-L/16. The final patch descriptors retain their spatial indices and form the grid used for LiDAR–image association. 17
3.4 The overview of mapping 20
3.5 The explanation of how to calculate TP, FP, FN on prediction and ground truth label. In this case, LiDAR point clouds of SemanticKITTI sequence 09, 000888 frame was projected on it correspondence image, compare the ground truth label and prediction point by point. 22

3.6 The explanation of how to calculate ACC on prediction and ground truth label. In this case, LiDAR point clouds of SemanticKITTI sequence 09, 000888 frame was projected on it correspondence image. The color only means whether corret or not, do not mean any semantic class in this case. 1) shows the origin points; 2) the green color point means the true positive prediction point of all classes; 3) the red color points means the prediction point of all classes are different with ground truth labels. 23

4.1 Overview of the all pipeline. A) pretraining part connects frozen DINO dense visual featrues to the LiDAR backbone through a 2D-3D fusion teacher. B) the online part combines geometric registration with optional semantic weighting, then updates the map at the selected pose. 25
4.2 Semantic KITTI sequence 08/000001 with 19 classes 26
4.3 Overview of the teacher–student training pipeline for LiDAR semantic segmentation. The fusion teacher combines image features from a frozen DINOv3 encoder with LiDAR features from the student encoder. Groundtruth point labels supervise teacher and student model, while KL distillation aligns the student's class probabilities with those of the teacher on image paired points with valid labels. Teacher and student model share the LiDAR encoder and use separate classifiers. Only the LiDAR student is used at inference. 29
4.4 LiDAR-only student model architecture. The 11-dimensional point input is embedded into 64 channels, averaged within the initial voxels, and processed by four point–voxel blocks. Each block returns one 64-dimensional feature for every retained point. Concatenating the four outputs gives the 256-dimensional input to the main classifier. The dashed box describes the auxiliary classifiers used during training. 31

4.5 2D-3D fusion teacher model architecture. The fusion teacher combines visual features from a frozen DINOv3 encoder with 3D geometric features from the LiDAR student. Its inputs are 1024D DINO descriptors for image paired points and four 64D student point features. At each stage, a trainable adapter reduces the DINO descriptor to 64D. This descriptor is concatenated with the corresponding student feature, then processed by a linear layer, a sigmoid, and ReLU to produce a 64D fused feature. The four fused features are concatenated and passed to the main classifier, which predicts 19 class logits per image paired point. Each stage also has an auxiliary classifier. The teacher and student use the same classifier architectures with separate weights. During training, the teacher's predictions guide the student through knowledge distillation. 33

5.1 Our semantic segmentation prediction results on SemanticKITTI sequences 11–21. 42
5.2 SemanticKITTI sequence 08 trajectory. 45
5.3 SemanticKITTI sequence 09 trajectory. 46
5.4 IPB trajectory. 46
5.5 IPB dataset semantic mapping result. 47
5.6 SemanticKIITI sequence 08 using pretrained semantic segmentation model building semantic mapping in PIN-SLAM. 47

List of Tables

2.1 Overview of 3D semantic segmentation approaches. 8

3.1 Comparison of the DINO family 16

4.1 Main notation 27
4.2 Voxel sizes and output dimensions of the student encoder. Every stage returns one feature vector for each retained point. 32

5.1 Datasets and splits used for semantic segmentation task. 38
5.2 Network dimensions. N_{ret} and N_{vis} are the retained and image-paired point counts. 39
5.3 Training and evaluation settings for our segmentation model. . . . 40
5.4 Semantic segmentation results on the SemanticKITTI validation and test sets. All values are percentages. 41
5.5 Comparison on the SemanticKITTI single scan test set (sequences 11–21). IoU values are percentages. 43
5.6 Semantic segmentation results with different visual inputs on the SemanticKITTI validation and test sets. All values are percentages. 44
5.7 IPB dataset semantic segmentation results comparing with different approach. 45
5.8 Trajectory accuracy evaluated with evo (translational RMSE). Lower values are better. APE uses full-trajectory SE(3) alignment with fixed scale. RPE uses all adjacent frame pairs without global alignment. 45

List of Algorithms

Appendix A

Paper

Leveraging Vision Foundation Models for Semantic Implicit Multi-sensor SLAM

1st Shuyi Wu
University of BONN
Bonn, Germany
s84swu@uni-bonn.de

***Abstract*—In simultaneous localization and mapping problem beyond pose estimation and geometric reconstruction, a good semantic understanding of the scene is also crucial part to support navigation, interaction, and decision-making. This thesis develops a pipeline that leverages vision foundation models in semantic segmentation pipeline to build an efficient semantic implicit multi-sensor SLAM. Our approach takes advantage of DINOv3 features during training, and then conduct semantic segmentation only need LiDAR scans. During mapping, successive scans update the same map, allowing scene geometry and semantic categories to be queried and visualized together. As a result, our method on SemanticKITTI achieves a mean intersection over union (mIoU) of 64.13% on the validation set and 63.5% on the test set, with semantic segmentation inference running at 16.5 frames per second. On the IPB dataset validation split, the model trained on manually annotated local data achieves 56.87% mIoU. Compared with geometry-only mapping, the semantic configuration achieves lower aligned absolute pose error. The integrated system demonstrates live semantic map updates in PIN-SLAM using LiDAR input alone.**

***Index Terms*—Semantic SLAM, Semantic segmentation, Vision Foundation Model, DINOv3, PIN-SLAM**

I. Introduction

Simultaneous localization and mapping (SLAM) estimates sensor motion while constructing a representation of the surrounding environment. Geometry provides the spatial structure required for localization, but autonomous systems also need to distinguish meaningful categories such as road, sidewalk, vehicle, and vegetation. Associating these categories with a persistent map supports interpretation of the environment across successive observations.

PIN-SLAM [1] represents geometry with a point-based implicit neural map and supports geometric registration, surface reconstruction, and global map consistency. This representation provides a common spatial support on which semantic evidence can accumulate. The challenge is to learn useful point-wise semantics and integrate them into the map while maintaining correspondence between predictions and the measurements used for mapping.

Vision foundation models offer dense features learned from large image collections. DINOv3 [2] supplies visual features that can inform a LiDAR segmentation model during training. However, requiring images for every online prediction would retain a multimodal dependency in deployment. Following the multi-scale fusion-to-single knowledge distillation strategy of 2DPASS [3], we use paired images to train a 2D-3D fusion teacher and transfer its predictions to a LiDAR-only student. The teacher and student share a LiDAR encoder, while their classification heads have separate parameters. DINOv3 remains frozen throughout training.

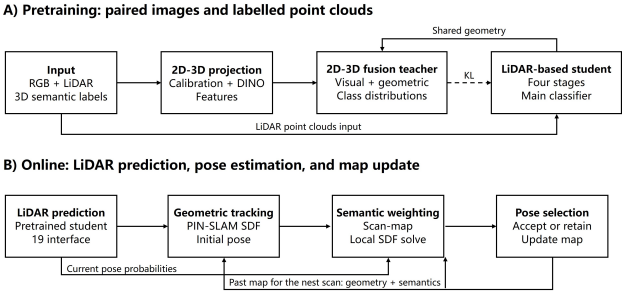


Fig. 1. Overview of the proposed pipeline. A) Offline training uses frozen DINOv3 features and a 2D-3D fusion teacher to train the LiDAR-only student. B) Online mapping estimates the current pose through geometric registration, then uses aligned student predictions to supervise the semantic decoder and map features shared with the SDF decoder.

The pretrained student model is connected to PIN-SLAM's neural point map. Semantic labels supervise the semantic decoder and the trainable map features shared with the signed distance field (SDF) decoder. Pose estimation continues to use PIN-SLAM's geometric registration; semantic supervision is applied when updating the map after the current pose has been estimated. This arrangement links semantic observations to the persistent geometric representation without requiring camera images at inference.

The main contributions are threefold. First, we develop DINOv3-assisted LiDAR semantic segmentation with calibrated visual features, a four-stage fusion teacher, and a LiDAR-only student. Second, we integrate the pretrain LiDAR-only student model into PIN-SLAM while preserving point correspondence, enabling semantic map queries, live visualization, and semantic colors on reconstructed mesh vertices. Third, we prepare calibrated data and manually annotated IPB labels, and evaluate segmentation, ablations, inference speed, and trajectory errors on SemanticKITTI [4] and IPB.

II. Related Work

A. 3D Semantic Segmentation

LiDAR semantic segmentation methods differ in how they represent irregular point clouds. RangeNet++ [5] predicts labels from spherical range images and returns predictions to the original points. Cylinder3D [6] uses cylindrical partitioning and asymmetrical convolutions to account for the variation of point density with range. Submanifold sparse convolutions [7] restrict computation to active spatial sites, while sparse point-voxel convolution [8] combines voxel aggregation with point-level processing. LSK3DNet [9] uses large sparse kernels with spatial-wise dynamic sparsity and channel-wise weight selection.

Point-based networks instead process points directly. PointNet [10] combines shared point-wise features with symmetric pooling, and Point Transformer V3 [11] uses serialized neighborhoods for scalable attention. Sonata [12] studies self-supervised point cloud representations. These approaches improve how geometric context is encoded or how representations are learned before downstream segmentation.

Cross-modal learning introduces information from paired images. 2DPASS [3] combines auxiliary modality fusion with multi-scale fusion-to-single knowledge distillation, allowing LiDAR-only inference. DINO in the Room (DITR) [13] studies injection and distillation of pretrained visual features into 3D segmentation. Such transfer still requires valid image–point correspondences during training, and these segmentation methods do not by themselves maintain a persistent semantic map. Building on this direction, our approach uses frozen DINOv3 features with multi-scale fusion-to-single knowledge distillation and connects the resulting LiDAR-only student to PIN-SLAM for persistent semantic LiDAR mapping.

B. Semantic SLAM

SemanticFusion [14] accumulates segmentation probabilities in a surfel map through geometric correspondences. SuMa++ [15] integrates LiDAR semantics into surfel-based SLAM and uses semantic information for moving-object filtering and scan matching. These systems illustrate that maintaining semantic map attributes and introducing semantic constraints into tracking are separate design choices.

Neural representations provide another way to associate scene properties. Semantic-NeRF [16] represents semantics, appearance, and geometry in an implicit scene model. SNI-SLAM [17] combines semantic features with online neural tracking and mapping, while SGS-SLAM [18] incorporates semantic information into a Gaussian representation. Here, PIN-SLAM's neural points support both an SDF and a semantic field. Semantic supervision changes their shared features, while online pose estimation remains based on geometric registration.

III. Approach

A. Problem Formulation and 2D-3D Correspondence

A training sample contains a LiDAR scan $\mathcal{P} = \{(\mathbf{x}_i^L, i_i, b_i^r)\}_{i=1}^N$, paired RGB images, point labels y_i, and camera–LiDAR calibration. The coordinate $\mathbf{x}_i^L \in \mathbb{R}^3$ is expressed in the LiDAR frame, i_i is intensity, and b_i^r indicates its availability and validity. SemanticKITTI learning IDs $1, \dots, 19$ become network targets $0, \dots, 18$; ignored learning ID zero becomes target -1. All segmentation heads use this same 19-class order.

We use i for scan points, j for map nodes, $l = 1, \dots, 4$ for network stages, and k for cameras. A transform $\mathbf{T}_{AB}$ maps coordinates from frame B to frame A, with rotation $\mathbf{R}_{AB}$ and translation $\mathbf{t}_{AB}$. Following the camera projection formulation of Wiesmann et al. [27], a LiDAR point is transformed into the camera frame, projected into normalized image coordinates, and mapped to image pixels:

$$\begin{aligned} \mathbf{x}_i^{C_k} &= \mathbf{R}_{C_k L}\mathbf{x}_i^L + \mathbf{t}_{C_k L}, \\ \boldsymbol{\xi}_{ik} &= \mathrm{project}_k(\mathbf{x}_i^{C_k}), \\ \widetilde{\boldsymbol{\xi}}_{ik} &= \mathrm{distort}_k(\boldsymbol{\xi}_{ik}), \\ [u_{ik}, v_{ik}, 1]^\top &= \mathbf{K}_k[\widetilde{\boldsymbol{\xi}}_{ik}^\top, 1]^\top. \end{aligned} \tag{1}$$

Here, $\boldsymbol{\xi}_{ik}$ and $\widetilde{\boldsymbol{\xi}}_{ik}$ are the undistorted and distorted normalized coordinates, respectively. For a point $\mathbf{x}_i^{C_k} = [X_{ik}, Y_{ik}, Z_{ik}]^\top$ in the camera frame, the pinhole and equidistant fisheye projections are

$$\begin{aligned} \boldsymbol{\xi}_{ik} &= \mathrm{project}_k(\mathbf{x}_i^{C_k}) \\ &= \begin{cases} \left[\dfrac{X_{ik}}{Z_{ik}}, \dfrac{Y_{ik}}{Z_{ik}}\right]^\top, & \text{pinhole}, \\ \dfrac{\theta_{ik}}{\rho_{ik}}[X_{ik}, Y_{ik}]^\top, & \text{equidistant fisheye}, \end{cases} \end{aligned} \tag{2}$$

where $\rho_{ik} = \sqrt{X_{ik}^2 + Y_{ik}^2}$ is the distance to the optical axis and $\theta_{ik} = \mathrm{atan2}(\rho_{ik}, Z_{ik})$ is the angle to the positive optical axis.

Radial and tangential distortion are applied in the normalized image plane:

$$\begin{aligned} \widetilde{\boldsymbol{\xi}}_{ik} &= \mathrm{distort}_k(\boldsymbol{\xi}_{ik}) \\ &= \boldsymbol{\xi}_{ik}\left(1 + \sum_{n=1}^{N_d} \kappa_{k,n} r_{ik}^{2n}\right)^{\gamma_k} \\ &\quad + 2\boldsymbol{\xi}_{ik}\boldsymbol{\xi}_{ik}^\top \mathbf{p}_k + r_{ik}^2 \mathbf{p}_k. \end{aligned} \tag{3}$$

Here, $r_{ik}^2 = \boldsymbol{\xi}_{ik}^\top \boldsymbol{\xi}_{ik}$, $\{\kappa_{k,n}\}_{n=1}^{N_d}$ are the radial distortion coefficients, and N_d is their number. The vector $\mathbf{p}_k = [p_{k,2}, p_{k,1}]^\top$ contains the tangential distortion coefficients. The choice $\gamma_k = 1$ gives the Brown distortion model, while $\gamma_k = -1$ gives the division model.

With zero skew, the intrinsic matrix and the mapping from distorted normalized coordinates to pixels are

$$\begin{aligned} \mathbf{K}_k &= \begin{bmatrix} f_{x,k} & 0 & c_{x,k} \\ 0 & f_{y,k} & c_{y,k} \\ 0 & 0 & 1 \end{bmatrix}, \\ \begin{bmatrix} u_{ik} \\ v_{ik} \end{bmatrix} &= \begin{bmatrix} f_{x,k} & 0 \\ 0 & f_{y,k} \end{bmatrix} \widetilde{\boldsymbol{\xi}}_{ik} + \begin{bmatrix} c_{x,k} \\ c_{y,k} \end{bmatrix}. \end{aligned} \tag{4}$$

The parameters $f_{x,k}$ and $f_{y,k}$ are the focal lengths in pixels along the horizontal and vertical image axes, and $(c_{x,k}, c_{y,k})$ is the principal point in pixels.

SemanticKITTI uses rectified images, and IPB uses undistorted images. With the corresponding intrinsics, projection into these images reduces to

$$d_{ik}[u_{ik}, v_{ik}, 1]^\top = \mathbf{K}_k[\mathbf{I}_3\ \mathbf{0}]\mathbf{T}_{C_kL}\overline{\mathbf{x}}_i^L, \tag{5}$$

where $\overline{\mathbf{x}}_i^L = [(\mathbf{x}_i^L)^\top, 1]^\top$ is the homogeneous point and d_{ik} is its camera depth. Lens distortion is already removed in these images. For SemanticKITTI, the supplied calibration projects points into the rectified `image_2` view as $d_i[u_i, v_i, 1]^\top = \mathbf{P}_2\mathbf{T}_{C_0L}\overline{\mathbf{x}}_i^L$. Here, $\mathbf{T}_{C_0L}$ maps points to the reference camera frame, and $\mathbf{P}_2 \in \mathbb{R}^{3\times4}$ includes the intrinsic projection and the offset of camera 2. For IPB, points are projected into the front, left, rear, and right views using their respective calibrations. The LiDAR-to-camera transformations are obtained from the sensor extrinsics in a common reference frame. If several views provide valid projections, we select the view whose projected point lies closest to the image center in normalized image coordinates, assigning one image correspondence to each matched point.

Frozen DINOv3 ViT-L/16 extracts 1024-dimensional patch descriptors. Each image is resized with preserved aspect ratio and a long side of 1248 pixels. After RGB normalization, zero padding on the right and bottom makes both dimensions divisible by 16. For padded size $H' \times W'$, the final normalized patch tokens form a grid $\mathbf{F}_{\text{grid}}^{2D} \in \mathbb{R}^{H_p\times W_p\times1024}$, where $H_p = H'/16$ and $W_p = W'/16$.

For the selected view, let (u_i, v_i) denote the original image position and s_x, s_y the actual resize factors after rounding the image dimensions. The continuous token coordinates are

$$q_i^u = \frac{s_x(u_i + 1/2)}{16} - \frac{1}{2}, \qquad q_i^v = \frac{s_y(v_i + 1/2)}{16} - \frac{1}{2}. \tag{6}$$

SemanticKITTI descriptors are bilinearly sampled at (q_i^u, q_i^v). IPB uses the containing token, with horizontal and vertical indices $\lfloor q_i^u + 1/2 \rfloor$ and $\lfloor q_i^v + 1/2 \rfloor$, respectively. Sampling indices are clipped to the token grid bounds. Each descriptor $\mathbf{f}_i^{2D} \in \mathbb{R}^{1024}$ is stored with its original LiDAR point index. These indices select the corresponding LiDAR features and labels when constructing the paired training subset, preserving point-wise correspondence during fusion.

B. LiDAR-Only Student

The student receives five attributes per retained point: augmented XYZ coordinates, remission $\widetilde{i}_i$, and a remission validity indicator b_i^r. With base voxel size $h_0 = 0.10$ m, let $\overline{\mathbf{x}}_{v(i)}$ and $\mathbf{c}_{v(i)}$ denote the mean coordinate and center of the point's base voxel. The entry encoding is

$$\begin{aligned}\mathbf{f}_{i,\text{entry}}^{3D} &= \text{MLP}_{11\to64\to64}\big([\mathbf{a}_i; \mathbf{x}_i^{\text{aug}} - \overline{\mathbf{x}}_{v(i)}; \mathbf{x}_i^{\text{aug}} - \mathbf{c}_{v(i)}]\big),\\ \mathbf{a}_i &= [x_i^{\text{aug}}, y_i^{\text{aug}}, z_i^{\text{aug}}, \widetilde{i}_i, b_i^r]^\top.\end{aligned} \tag{7}$$

Thus, the input comprises five point attributes and two 3D offsets. ReLU separates the two linear layers.

Fig. 3 shows the student architecture. Four point–voxel blocks each contain a sparse convolution branch and an MLP branch. The sparse branch uses two residual blocks with sparse convolutions [7]. At each stage, points gather their current voxel features, and the branch features are averaged into the next, larger voxels. Their sum feeds the following block. The point-aligned stage output is read from the pooled MLP branch. Output voxel sizes are 0.20, 0.40, 0.80, and 1.60 m, and every stage returns $N_{\text{ret}} \times 64$ features in the retained point order.

The main classifier concatenates the four stage outputs and uses a $256 \to 128 \to 19$ MLP. Each stage also has a $64 \to 128 \to 19$ auxiliary classifier during training. ReLU is used between the linear layers. At deployment, only the encoder and main classifier are retained, predictions are restored to the original scan order.

C. 2D-3D Fusion Teacher

Fig. 2 summarizes the training pipeline. The fusion teacher reuses the student's LiDAR features instead of introducing a second LiDAR encoder. For each image-paired point, a separate adapter at stage l reduces the frozen 1024D descriptor to 64D. Fusion combines this descriptor with the corresponding 64D student feature:

$$\begin{aligned}\widetilde{\mathbf{f}}_{l,i}^{2D} &= \text{ReLU}\big(\text{LN}(\mathbf{A}_l\mathbf{f}_i^{2D} + \mathbf{b}_l^A)\big),\\ \widetilde{\mathbf{f}}_{l,i}^{\text{fuse}} &= \mathbf{B}_l[\widetilde{\mathbf{f}}_{l,i}^{2D}; \mathbf{f}_{l,i}^{3D}] + \mathbf{b}_l^B,\\ \mathbf{f}_{l,i}^{\text{fuse}} &= \text{ReLU}\Big(\widetilde{\mathbf{f}}_{l,i}^{\text{fuse}} \odot \sigma(\mathbf{Q}_l\widetilde{\mathbf{f}}_{l,i}^{\text{fuse}} + \mathbf{b}_l^Q)\Big).\end{aligned} \tag{8}$$

Here, LN is LayerNorm, σ is sigmoid, and $\odot$ is element-wise multiplication. The four stages use the same frozen descriptor with independent adapters and fusion weights. The teacher has the same main and auxiliary classifier dimensions as the student, with separate parameters, and predicts only for image-paired points. Fig. 4 shows the architecture of 2D-3D fusion teacher model.

D. Supervision and Knowledge Distillation

Let $\mathcal{A}$ contain all valid annotated points and $\mathcal{I}_{\text{pair}}$ the image-paired points with valid labels. For logits $\mathbf{Z}$ and probabilities $\mathbf{p}_i = \text{softmax}(\mathbf{z}_i)$, weighted cross-entropy is

$$\begin{aligned}\text{CE}(\mathbf{Z}, \mathbf{y}; \mathcal{B}) &= -\frac{\sum_{i\in\mathcal{B}} w_{y_i} \log p_{i,y_i}}{\sum_{i\in\mathcal{B}} w_{y_i}},\\ w_c &= \left(\frac{n_{\max}}{n_c}\right)^{1/3},\end{aligned} \tag{9}$$

where n_c is the fixed SemanticKITTI class count and $n_{\max} = \max_c n_c$. The student main head additionally uses Lovász–

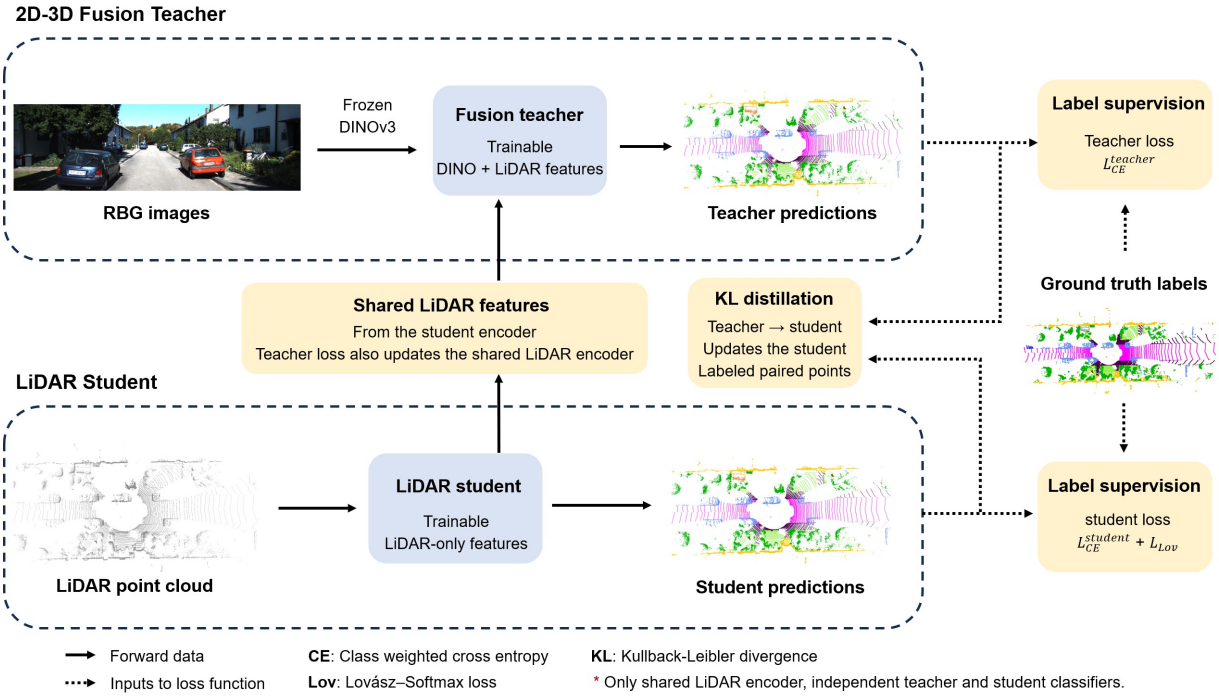


Fig. 2. Teacher–student training for LiDAR semantic segmentation. Frozen DINOv3 features and shared LiDAR features form the 2D-3D fusion teacher. Both branches receive label supervision; detached teacher distributions guide the student through KL distillation. Only the LiDAR student is used at inference.

Softmax loss [19] over classes present in the valid annotations. Head zero denotes the main classifier:

$$\begin{aligned} L_{3D} &= \mathrm{CE}(\mathbf{Z}_0^{3D}, \mathbf{y}; \mathcal{A}) + 0.10 L_{\mathrm{Lov}} \\ &\quad + \frac{0.25}{4} \sum_{l=1}^{4} \mathrm{CE}(\mathbf{Z}_l^{3D}, \mathbf{y}; \mathcal{A}), \\ L_{\mathrm{fuse}} &= \mathrm{CE}(\mathbf{Z}_0^{\mathrm{fuse}}, \mathbf{y}; \mathcal{I}_{\mathrm{pair}}) \\ &\quad + \frac{1}{4} \sum_{l=1}^{4} \mathrm{CE}(\mathbf{Z}_l^{\mathrm{fuse}}, \mathbf{y}; \mathcal{I}_{\mathrm{pair}}). \end{aligned} \tag{10}$$

The student, visual adapters, fusion layers, and classifiers are trained jointly. DINOv3 remains frozen. Student features are not detached before fusion, so the teacher's supervised loss also updates the shared LiDAR encoder.

Knowledge distillation matches the main and four auxiliary distributions at temperature $\tau = 2$. Teacher probabilities are detached specifically for this loss:

$$\begin{aligned} \mathbf{p}_{h,i}^{3D}(\tau) &= \mathrm{softmax}(\mathbf{z}_{h,i}^{3D}/\tau), \\ \overline{\mathbf{p}}_{h,i}^{\mathrm{fuse}}(\tau) &= \mathrm{sg}\big(\mathrm{softmax}(\mathbf{z}_{h,i}^{\mathrm{fuse}}/\tau)\big). \end{aligned} \tag{11}$$

For $M = |\mathcal{I}_{\mathrm{pair}}| > 0$, the distillation objective is

$$L_{\mathrm{KD}} = \frac{\tau^2}{5M} \sum_{h=0}^{4} \sum_{i \in \mathcal{I}_{\mathrm{pair}}} D_{\mathrm{KL}}\big(\overline{\mathbf{p}}_{h,i}^{\mathrm{fuse}}(\tau) \,\|\, \mathbf{p}_{h,i}^{3D}(\tau)\big). \tag{12}$$

Paired points have equal distillation weight; class-frequency weights apply to cross-entropy. Training uses

$$\begin{aligned} L_{\mathrm{train}} &= L_{3D} + L_{\mathrm{fuse}} + \lambda_{\mathrm{KD}}(n_{\mathrm{ep}}) L_{\mathrm{KD}}, \\ \lambda_{\mathrm{KD}}(n_{\mathrm{ep}}) &= 0.10 \ \mathrm{clip}\left(\frac{n_{\mathrm{ep}} - 5}{5}, 0, 1\right). \end{aligned} \tag{13}$$

Here, sg denotes the stop-gradient operator, n_{ep} is the epoch number, and $\mathrm{clip}(x, 0, 1)$ clips x to the interval $[0, 1]$.

Distillation starts after five epochs and reaches its full weight at epoch ten. Visual information therefore reaches the student through supervised fusion and prediction distillation, while the deployed student requires only LiDAR data.

E. Semantic Neural Point Map

Each PIN-SLAM map point stores a world position $\mathbf{n}_j \in \mathbb{R}^3$, an orientation $\mathbf{R}_j \in \mathrm{SO}(3)$, and an 8D trainable feature $\mathbf{f}_j^{\mathrm{map}}$. Separate geometry and semantic decoders share these features. Each decoder has one 64-unit ReLU hidden layer. The geometry decoder predicts an SDF; the semantic head has 20 channels, comprising reserved index zero and the 19 scene categories.

Let g_{θ_g} and s_{θ_s} denote the geometry and semantic decoders, respectively. With local decoder input $\mathbf{h}_j(\mathbf{x})$, interpolation weights $\alpha_j(\mathbf{x})$, and SDF scale β_s, decoding before interpo-

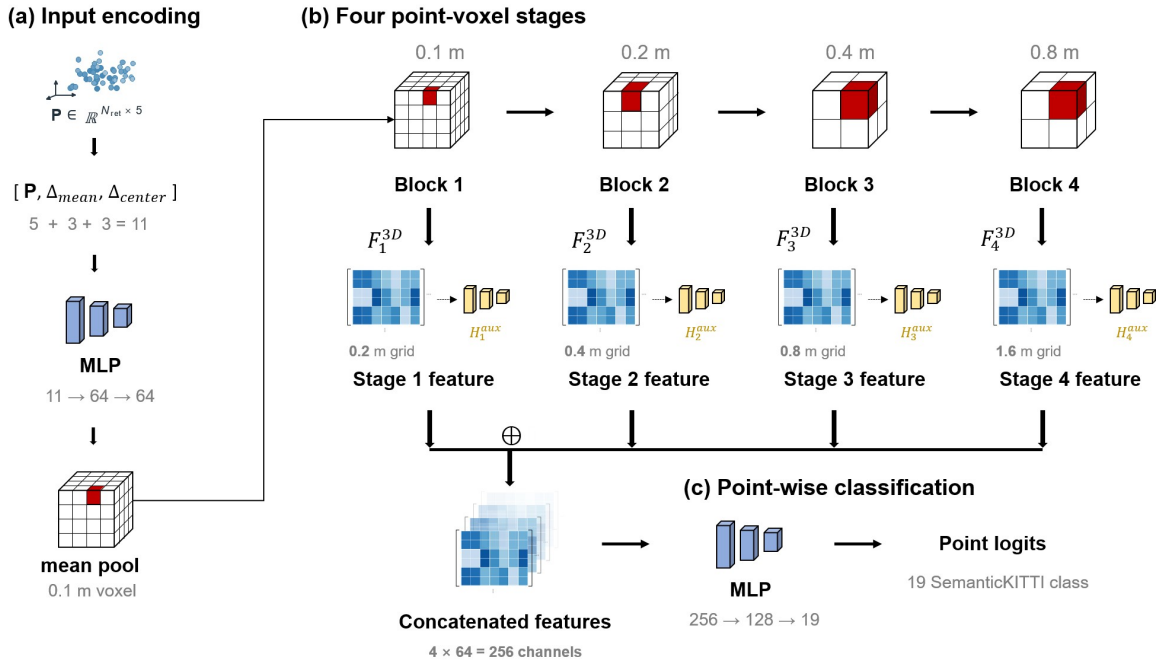


Fig. 3. LiDAR-only student model architecture. An 11-dimensional point input is embedded into 64 channels and processed by four point–voxel blocks. Concatenated stage outputs feed the main classifier; auxiliary classifiers supervise the individual stages during training.

lation gives

$$\begin{aligned} s_j(\mathbf{x}) &= \beta_s g_{\theta_g}(\mathbf{h}_j(\mathbf{x})), \\ s_M(\mathbf{x}) &= \sum_j \alpha_j(\mathbf{x}) s_j(\mathbf{x}), \\ \ell_{j,c}(\mathbf{x}) &= \log \operatorname{softmax}(s_{\theta_s}(\mathbf{h}_j(\mathbf{x})))_c, \\ \ell_c(\mathbf{x}) &= \sum_j \alpha_j(\mathbf{x}) \ell_{j,c}(\mathbf{x}). \end{aligned} \tag{14}$$

The map SDF s_M is in meters, and ℓ_c is the aggregated log score for class c. The semantic label is $\widehat{y}_M(\mathbf{x}) = \arg\max_{c\in\{0,\dots,19\}} \ell_c(\mathbf{x})$.

The frozen student processes each raw scan while geometric preprocessing and tracking proceed. Original row indices preserve correspondence through point selection. At the Mapper boundary, predictions for the same frame are attached to retained points. A student class $c \in \{0,\dots,18\}$ becomes map target $\widehat{y}_i^{\text{map}} = c + 1$; unusable predictions receive zero. The current pose is first estimated by geometric registration, after which the prediction labels from the student model supervise map learning.

For the valid labeled map samples $\mathcal{B}_s$, the semantic and complete map objectives are

$$\begin{aligned} L_{\text{sem}} &= -\frac{1}{|\mathcal{B}_s|} \sum_{q\in\mathcal{B}_s} \ell_{\widehat{y}_q^{\text{map}}}(\mathbf{x}_q), \\ L_{\text{map}} &= L_{\text{sdf}} + 0.5 L_{\text{eik}} + 1.0 L_{\text{sem}}. \end{aligned} \tag{15}$$

Geometric losses update the shared features and geometry decoder; the semantic loss L_{sem} updates the shared features and semantic decoder. Repeated optimization and replay of observations from multiple frames accumulate temporal information in the persistent map. PIN-SLAM's surface reconstruction tools extract the SDF surface, and querying semantic labels at mesh vertices assigns semantic colors. Semantic supervision can influence subsequent geometric registration through the shared map features; the pose objective itself remains geometric.

IV. Experiments

A. Datasets and Evaluation

SemanticKITTI [4] provides dense point-wise labels for KITTI odometry scans [20]. We use the standard single-scan split: sequences 00–07, 09, and 10 for training (19,130 scans), sequence 08 for validation (4,071 scans), and sequences 11–21 for blind test evaluation. Labels are mapped to 19 evaluation classes, with moving and non-moving instances assigned to the same class. Paired front camera images are available during training, while the student is evaluated on complete LiDAR scans.

The IPB sequence contains outdoor road scenes with buildings, parked vehicles, sidewalks, trees, cyclists, and traffic signs. It provides LiDAR and four calibrated RGB views at 2064×1024 resolution. Of the 1,001 frames, the first 500 receive manually annotated point labels for this task: frames 0–399 form the training set and frames 400–499 the validation set. The same 19-class label space is used. IPB has no reflectance, so intensity and its availability flag are both set to zero.

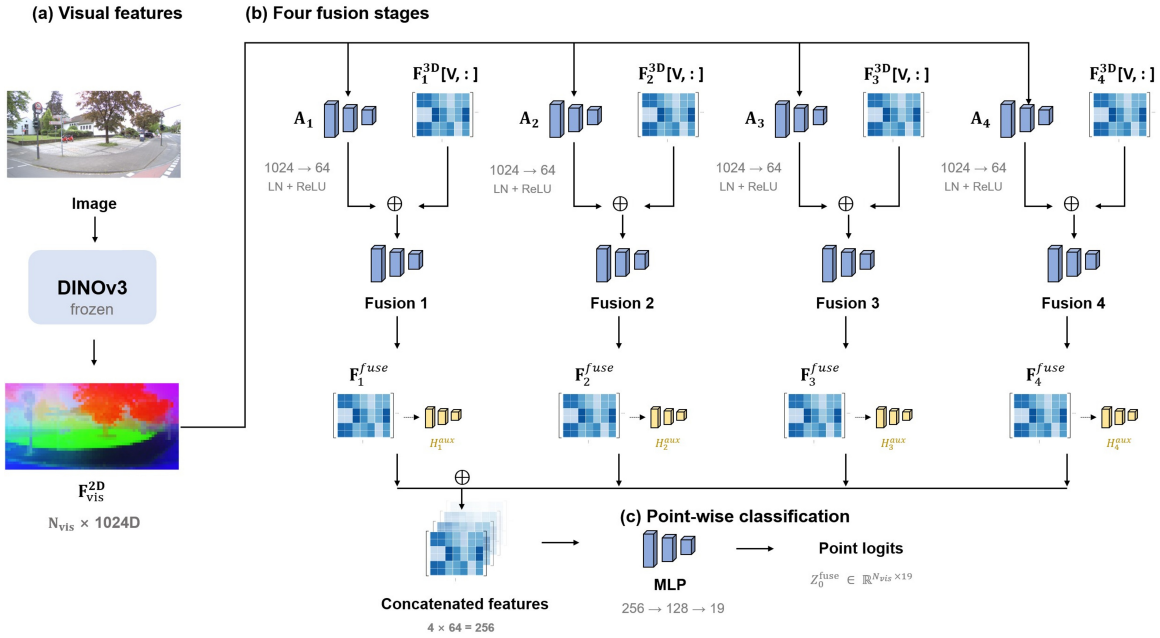


Fig. 4. 2D-3D fusion teacher model architecture. The fusion teacher combines visual features from a frozen DINOv3 encoder with 3D geometric features from the LiDAR student. Its inputs are 1024D DINO descriptors for image paired points and four 64D student point features. At each stage, a trainable adapter reduces the DINO descriptor to 64D. This descriptor is concatenated with the corresponding student feature, then processed by a linear layer, a sigmoid, and ReLU to produce a 64D fused feature. The four fused features are concatenated and passed to the main classifier, which predicts 19 class logits per image paired point. Each stage also has an auxiliary classifier. The teacher and student use the same classifier architectures with separate weights. During training, the teacher's predictions guide the student through knowledge distillation.

Segmentation is evaluated using mean intersection over union (mIoU):

$$\mathrm{mIoU} = \frac{1}{|\mathcal{C}_+|} \sum_{c \in \mathcal{C}_+} \frac{\mathrm{TP}_c}{\mathrm{TP}_c + \mathrm{FP}_c + \mathrm{FN}_c}, \tag{16}$$

where TP_c, FP_c, and FN_c denote the true-positive, false-positive, and false-negative point counts for class c. The set $\mathcal{C}_+$ contains the evaluated classes with a nonzero union. mIoU is reported as a percentage, and overall accuracy is additionally reported for the validation sets.

Trajectories are evaluated with evo [21]. Let $T_i, \widehat{T}_i \in \mathrm{SE}(3)$ denote the corresponding reference and estimated poses, mapping sensor coordinates to world coordinates, and let N be the number of corresponding poses. Absolute pose error (APE) is reported as translational RMSE after full-trajectory SE(3) alignment with fixed scale:

$$\mathrm{APE}_{\mathrm{RMSE}} = \sqrt{\frac{1}{N} \sum_{i=1}^{N} \left\| \mathrm{trans}\left(T_i^{-1} S \widehat{T}_i \right) \right\|_2^2}, \tag{17}$$

where $S \in \mathrm{SE}(3)$ is the fitted rigid alignment and $\mathrm{trans}(\cdot)$ extracts the translation component.

Relative pose error (RPE) is evaluated over adjacent frame pairs, $\Delta = 1$, without global alignment:

$$E_i = \left(T_i^{-1} T_{i+1} \right)^{-1} \left(\widehat{T}_i^{-1} \widehat{T}_{i+1} \right),$$
$$\mathrm{RPE}_{\mathrm{RMSE}} = \sqrt{\frac{1}{N-1} \sum_{i=1}^{N-1} \| \mathrm{trans}(E_i) \|_2^2}. \tag{18}$$

APE and RPE characterize globally aligned trajectory agreement and local motion error, respectively.

B. Implementation Details

For both datasets, the student and fusion modules are randomly initialized and trained for 50 epochs with AdamW, learning rate 10^{-3}, weight decay 10^{-4}, batch size one, and gradient accumulation over eight scans. Learning rate warm-up lasts five epochs and is followed by cosine decay. Training uses random yaw, horizontal flips, scaling in $[0.95, 1.05]$, and a shared Gaussian translation with standard deviation 0.1 m. Point dropout is sampled between zero and 0.2, and frame-level intensity dropout has probability 0.3. Training and student inference use an NVIDIA GeForce RTX 4060 Laptop GPU with PyTorch and CUDA 12.6.

For mapping, PIN-SLAM uses 0.4 m neural point voxels, 8D features, and at most six neighbors per query. Tracking points are downsampled with 0.6 m voxels. The geometry-only and geometry+semantic configurations share these geometric settings and use point-to-SDF registration. The semantic configuration adds the pretrained student's labels and the map supervision in (15).

TABLE I
SEMANTICKITTI SEGMENTATION ABLATIONS

Configuration	Val. mIoU (%)	Test mIoU (%)
DINOv3 Baseline	61.83	—
DINOv3 Baseline $+W$	64.13	**63.5**
DINOv3 Baseline $+W + A$	**64.98**	62.7
DINOv2 Baseline $+W$	63.98	62.2
RGB images Baseline $+W$	63.32	62.2

Validation: sequence 08. Test: sequences 11–21. W: class weighting; A: additional targeted augmentation. A dash denotes an unreported result.

TABLE II
REPRESENTATIVE SEMANTICKITTI TEST RESULTS

Method	Inference input	mIoU (%)
KPConv [23]	L	58.8
FusionNet [24]	L	61.3
KPRNet [25]	L	63.1
Lite-HDSeg [26]	L	63.8
2DPASS [3]	L	72.9
DITR [13]	L+C	74.4
LSK3DNet [9]	L	75.6
Baseline $+W$ (ours)	L	63.5

Single-scan test sequences 11–21. L: LiDAR; C: RGB images. Input denotes the evaluated inference configuration. The DITR entry uses image-feature injection.

C. SemanticKITTI Results and Ablations

Table I combines the effects of class weighting, targeted augmentation, and visual inputs. Baseline already contains the DINOv3 fusion teacher, knowledge distillation, and standard augmentation; it uses unweighted cross-entropy. $+W$ adds class weighting, and $+A$ adds targeted augmentation for weakly performing classes. The DINOv2 comparison uses ViT-L/14 [22] with the same descriptor dimension and fusion architecture.

Class weighting increases validation mIoU from 61.83% to 64.13%, a gain of 2.30 percentage points. The largest class improvements are bicycle (27.71% to 39.53%), other-vehicle (57.37% to 66.70%), and truck (75.14% to 84.00%). Overall accuracy changes from 91.95% to 91.87%, showing that the increase in mean class performance does not require an increase in the fraction of all correctly classified points.

Additional targeted augmentation raises validation mIoU to 64.98%. Bicycle improves from 39.53% to 46.07%, and other-ground from 2.50% to 6.02%. On the test set, however, mIoU decreases from 63.5% to 62.7%, while motorcyclist IoU falls from 37.2% to 17.5%. These results indicate an overfitting tendency in the augmented configuration. Baseline $+W$ is therefore retained as the main configuration.

DINOv3 gives the highest reported mIoU among the compared visual inputs. With class weighting, validation mIoU is 64.13%, compared with 63.98% for DINOv2 and 63.32% for RGB images. Test mIoU is 63.5% for DINOv3 and 62.2% for each alternative.

Table II places the main configuration among representative methods. Its 63.5% test mIoU exceeds KPConv, FusionNet, and KPRNet by 4.7, 2.2, and 0.4 percentage points, respectively, and is 0.3 percentage points below Lite-HDSeg. Higher overall mIoU is reported for 2DPASS, DITR, and LSK3DNet. The student runs at 16.5 FPS in the evaluated setup and uses LiDAR alone. This inference rate supports its use as an efficient semantic segmentation component in PIN-SLAM, providing semantic labels for incremental map updates.

TABLE III
TRAJECTORY ACCURACY WITH SEMANTIC SUPERVISION

Dataset	Frames	Method	APE (m) ↓	RPE (m) ↓
KITTI 08	4,071	geometry	4.11	0.02
KITTI 08	4,071	geometry+semantic	4.04	0.02
KITTI 09	1,591	geometry	0.89	0.03
KITTI 09	1,591	geometry+semantic	0.81	0.03
IPB	500	geometry	0.41	0.01
IPB	500	geometry+semantic	0.31	0.02

APE and RPE report translational RMSE. APE uses full-trajectory $\mathrm{SE}(3)$ alignment with fixed scale. RPE uses all adjacent frame pairs ($\Delta = 1$) without global alignment. Lower values are better.

D. IPB Semantic Segmentation

The model trained on manually annotated local IPB data achieves 56.87% mIoU and 93.79% overall accuracy on frames 400–499. Selected class IoUs are 95.25% for building, 95.51% for vegetation, 84.85% for road, 82.09% for sidewalk, 77.03% for person, and 87.09% for traffic-sign. The results demonstrate semantic prediction for the local sensor configuration, including missing reflectance.

E. Semantic Mapping and Trajectory Estimation

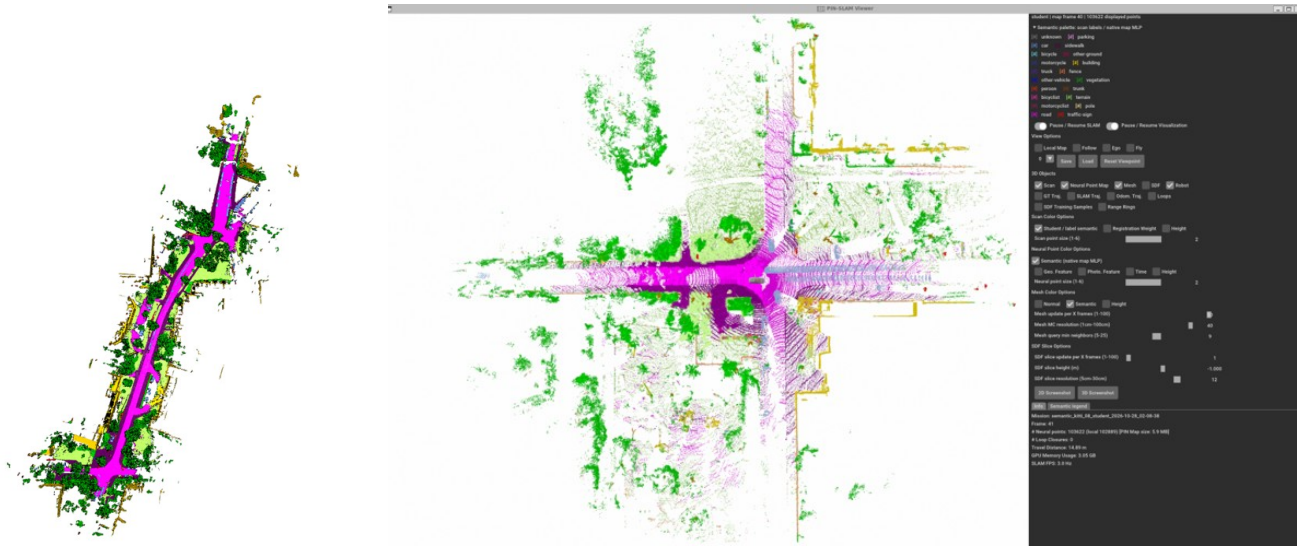
We compare geometry-only mapping and geometry+semantic mapping on KITTI sequences 08 and 09 and the first 500 IPB frames. Table III compares the translational APE and RPE of geometry-only mapping and geometry+semantic mapping. APE decreases from 4.11 to 4.04 m on KITTI 08, from 0.89 to 0.81 m on KITTI 09, and from 0.41 to 0.31 m on IPB.

Semantic supervision updates the map features shared with the SDF decoder and used for subsequent geometric registration. The semantic configuration reduces aligned APE on all three evaluated sequences.

Fig. 5 shows the accumulated IPB semantic map and the PIN-SLAM visualization on SemanticKITTI sequence 08. Successive scans update the same map, with colors distinguishing scene categories such as roads, vegetation, and buildings. Geometry and semantic labels can be queried together, and reconstructed mesh vertices can receive semantic colors. These visualizations demonstrate incremental semantic map construction and the joint representation of scene geometry and semantic categories.

V. DISCUSSION

Class weighting improves semantic segmentation by increasing the contribution of less frequent classes to the training objective. As shown in Table I, it raises validation mIoU on sequence 08 from 61.83% to 64.13%, with particularly



(a) IPB semantic map

(b) SemanticKITTI sequence 08 in PIN-SLAM

Fig. 5. Semantic mapping results using the pretrained LiDAR student. The IPB map accumulates semantic observations over the sequence, while the PIN-SLAM view shows the integrated semantic visualization on SemanticKITTI. Colors encode semantic categories.

strong improvements for bicycle, other-vehicle, and truck. These results demonstrate the effectiveness of class weighting in addressing class imbalance and improving mean class performance.

Additional targeted augmentation for weakly performing classes further increases validation mIoU on sequence 08 from 64.13% to 64.98%. However, mIoU on the official test sequences 11–21 decreases from 63.5% to 62.7%, indicating an overfitting tendency in the augmented configuration. The improvement on the validation sequence does not translate into better generalization to the test sequences. Therefore, Baseline $+W$ is retained as the main configuration, providing stronger test performance without the additional targeted augmentation.

Integrating the semantic segmentation module into PIN-SLAM also improves odometry accuracy in terms of APE. As shown in Table III, semantic map supervision reduces APE. These reductions demonstrate that using semantic predictions to supervise the map features shared with the SDF decoder benefits globally aligned trajectory accuracy.

VI. Conclusion

This work integrates DINOv3-assisted LiDAR semantic segmentation with the neural point map of PIN-SLAM. Calibrated frozen visual descriptors and a multi-scale 2D-3D fusion teacher support joint supervised learning and knowledge distillation, producing a student that predicts 19 semantic classes from LiDAR alone. During mapping, aligned point labels supervise a semantic decoder and persistent features shared with the SDF decoder, enabling incremental semantic map updates and visualization while retaining geometric pose estimation.

The main configuration achieves 64.13% validation and 63.5% test mIoU on SemanticKITTI, with 16.5 FPS segmentation inference, and the locally trained IPB model achieves 56.87% validation mIoU. Semantic map supervision reduces aligned APE on the evaluated sequences while slightly increasing consecutive-frame RPE. These results demonstrate efficient LiDAR-only semantic inference and incremental semantic mapping in PIN-SLAM, with visual knowledge transferred from DINOv3 during training.

References

[1] Y. Pan, X. Zhong, L. Wiesmann, T. Posewsky, J. Behley, and C. Stachniss, "PIN-SLAM: LiDAR SLAM using a point-based implicit neural representation for achieving global map consistency," *IEEE Trans. Robot.*, vol. 40, pp. 4045–4064, 2024.

[2] O. Siméoni *et al.*, "DINOv3," arXiv:2508.10104, Aug. 2025.

[3] X. Yan *et al.*, "2DPASS: 2D priors assisted semantic segmentation on LiDAR point clouds," in *Proc. ECCV*, 2022, pp. 677–695.

[4] J. Behley *et al.*, "SemanticKITTI: A dataset for semantic scene understanding of LiDAR sequences," in *Proc. IEEE/CVF ICCV*, 2019, pp. 9297–9307.

[5] A. Milioto, I. Vizzo, J. Behley, and C. Stachniss, "RangeNet++: Fast and accurate LiDAR semantic segmentation," in *Proc. IEEE/RSJ IROS*, 2019, pp. 4213–4220.

[6] X. Zhu *et al.*, "Cylindrical and asymmetrical 3D convolution networks for LiDAR segmentation," in *Proc. IEEE/CVF CVPR*, 2021, pp. 9939–9948.

[7] B. Graham, M. Engelcke, and L. van der Maaten, "3D semantic segmentation with submanifold sparse convolutional networks," in *Proc. IEEE CVPR*, 2018, pp. 9224–9232.

[8] H. Tang *et al.*, "Searching efficient 3D architectures with sparse point-voxel convolution," in *Proc. ECCV*, vol. 12373, 2020, pp. 685–702.

[9] T. Feng, W. Wang, F. Ma, and Y. Yang, "LSK3DNet: Towards effective and efficient 3D perception with large sparse kernels," in *Proc. IEEE/CVF CVPR*, 2024, pp. 14916–14927.

[10] C. R. Qi, H. Su, K. Mo, and L. J. Guibas, "PointNet: Deep learning on point sets for 3D classification and segmentation," in *Proc. IEEE CVPR*, 2017.

[11] X. Wu *et al.*, "Point transformer V3: Simpler, faster, stronger," in *Proc. IEEE/CVF CVPR*, 2024, pp. 4840–4851.

[12] X. Wu *et al.*, "Sonata: Self-supervised learning of reliable point representations," in *Proc. IEEE/CVF CVPR*, 2025, pp. 22193–22204.

[13] K. Knaebel *et al.*, "DINO in the room: Leveraging 2D foundation models for 3D segmentation," in *Proc. Int. Conf. 3D Vis. (3DV)*, 2026.

[14] J. McCormac, A. Handa, A. J. Davison, and S. Leutenegger, "SemanticFusion: Dense 3D semantic mapping with convolutional neural networks," in *Proc. IEEE ICRA*, 2017.
[15] X. Chen, A. Milioto, E. Palazzolo, P. Giguère, J. Behley, and C. Stachniss, "SuMa++: Efficient LiDAR-based semantic SLAM," in *Proc. IEEE/RSJ IROS*, 2019, pp. 4530–4537.
[16] S. Zhi, T. Laidlow, S. Leutenegger, and A. J. Davison, "In-place scene labelling and understanding with implicit scene representation," in *Proc. IEEE/CVF ICCV*, 2021, pp. 15838–15847.
[17] S. Zhu *et al.*, "SNI-SLAM: Semantic neural implicit SLAM," in *Proc. IEEE/CVF CVPR*, 2024, pp. 21167–21177.
[18] M. Li *et al.*, "SGS-SLAM: Semantic gaussian splatting for neural dense SLAM," in *Proc. ECCV*, vol. 15089, 2024, pp. 163–179.
[19] M. Berman, A. R. Triki, and M. B. Blaschko, "The Lovász-Softmax loss: A tractable surrogate for the optimization of the intersection-over-union measure in neural networks," in *Proc. IEEE CVPR*, 2018, pp. 4413–4421.
[20] A. Geiger, P. Lenz, and R. Urtasun, "Are we ready for autonomous driving? The KITTI vision benchmark suite," in *Proc. IEEE CVPR*, 2012, pp. 3354–3361.
[21] M. Grupp, "evo: Python package for the evaluation of odometry and SLAM," 2017. [Online]. Available: https://github.com/MichaelGrupp/evo
[22] M. Oquab *et al.*, "DINOv2: Learning robust visual features without supervision," *Trans. Mach. Learn. Res.*, 2024.
[23] H. Thomas, C. R. Qi, J.-E. Deschaud, B. Marcotegui, F. Goulette, and L. J. Guibas, "KPConv: Flexible and deformable convolution for point clouds," in *Proc. IEEE/CVF ICCV*, 2019, pp. 6411–6420.
[24] F. Zhang, J. Fang, B. Wah, and P. Torr, "Deep FusionNet for point cloud semantic segmentation," in *Proc. ECCV*, vol. 12356, 2020, pp. 644–663.
[25] D. Kochanov, F. Karimi Nejadasl, and O. Booij, "KPRNet: Improving projection-based LiDAR semantic segmentation," arXiv:2007.12668, 2020.
[26] R. Razani, R. Cheng, E. Taghavi, and B. Liu, "Lite-HDSeg: LiDAR semantic segmentation using lite harmonic dense convolutions," in *Proc. IEEE ICRA*, 2021, pp. 9550–9556.
[27] L. Wiesmann, T. Läbe, L. Nunes, J. Behley, and C. Stachniss, "Joint intrinsic and extrinsic calibration of perception systems utilizing a calibration environment," *IEEE Robot. Autom. Lett.*, vol. 9, no. 10, pp. 9103–9110, 2024, doi: 10.1109/LRA.2024.3457385.

Appendix B

Poster

Leveraging Vision Foundation Models for Semantic Implicit Multi-Sensor SLAM

Shuyi Wu, Louis Wiesmann , and Cyrill Stachniss

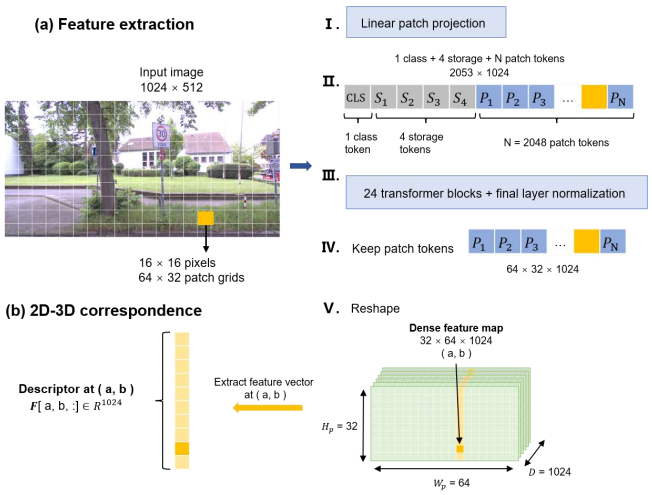
UNIVERSITÄT BONN

Abstract

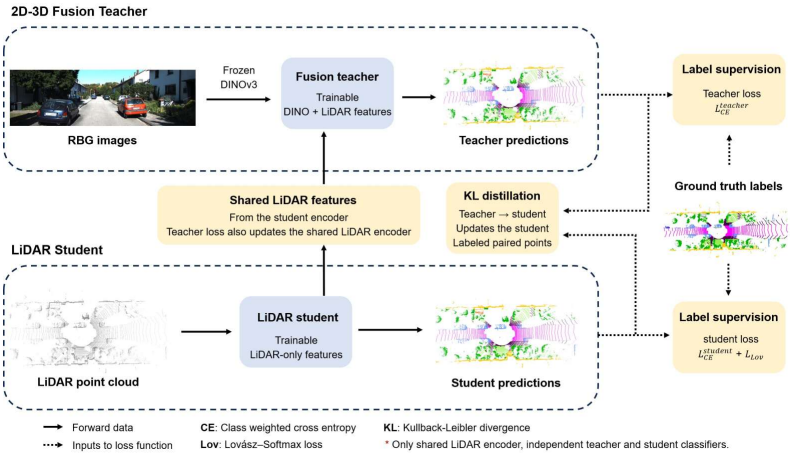
- Semantic SLAM, Semantic segmentation
- Why? Semantic information supports scene understanding and interaction with the environment.
- How? We jointly train a 2D-3D fusion teacher and a LiDAR-only student using frozen DINOv3 features and multi-scale fusion-to-single knowledge distillation.
- Results? The student achieves 63.5% SemanticKITTI test mIoU at 16.5 FPS for segmentation inference. Semantic map supervision also reduces aligned APE.

Method

- Calibrated 2D-3D correspondence associates LiDAR points with DINOv3 2D features
- Based on a multi-scale fusion-to-single knowledge distillation, 2D-3D fusion teacher and LiDAR-only student model share a LiDAR encoder and train jointly with supervised segmentation.



Schematic of patch feature extraction with frozen DINOv3 ViT-L/16. The final patch descriptors retain their spatial indices and form the grid used for LiDAR–image association.



Overview of pipeline for LiDAR semantic segmentation. The fusion teacher combines image features from a frozen DINOv3 encoder with LiDAR features from the student encoder. Ground truth point labels supervise teacher and student model, while KL distillation aligns the student's class probabilities with those of the teacher on image paired points with valid labels. Teacher and student model share the LiDAR encoder and use separate classifiers. Only the LiDAR student is used at inference.

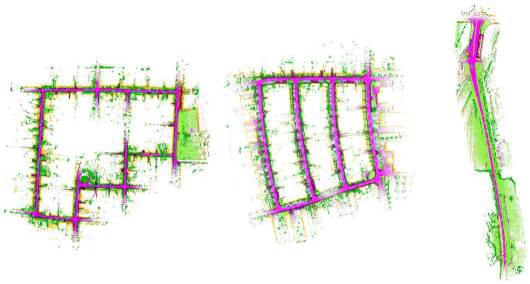
Experiments and Results

- It is capable of effectively using vision foundation model for semantic segmentation.
- We also showcase the ability of our approach to work effectively within a SLAM pipeline for building semantic mapping.
- Our method achieves 63.5% mIoU on official SemanticKITTI single-scan sequence 11-21.

REPRESENTATIVE SEMANTICKITTI TEST RESULTS

Method	Inference input	mIoU (%)
KPConv [23]	L	58.8
FusionNet [24]	L	61.3
KPRNet [25]	L	63.1
Lite-HDSeg [26]	L	63.8
2DPASS [3]	L	72.9
DITR [13]	L+C	74.4
LSK3DNet [9]	L	75.6
Baseline $+W$ (ours)	L	63.5

Single-scan test sequences 11–21. L: LiDAR; C: RGB images. Input denotes the evaluated inference configuration. The DITR entry uses image-feature injection.



Prediction results on SemanticKITTI sequence 15-17



Semantic mapping on IPB dataset